



F1Zone

Vizsgaremek

Szakma megnevezése:
Szoftverfejlesztő és -tesztelő

Készítette:
Bártfai Levente
Besze László

Témavezető:
Szalainé Török Edit
Méhes József György
Csukárdi Rajmund
Szabó Rózsa Terézia

2026

TARTALOM

1	Bevezetés	8
1.1	Projekt célja	8
1.2	A téma kiválasztásának indoklása	9
1.3	Célközönség	9
1.4	Csoport tagjai és munkamegosztás	9
1.4.1	Munkamegosztás	10
1.4.2	Közös fejlesztések.....	10
1.4.3	Fejlesztők egyéni hozzájárulásai.....	10
1.5	A program létrejötte	10
2	Fejlesztői dokumentáció	11
2.1	alkalmazás áttekintés.....	11
2.2	Használt technológiák.....	11
2.3	Felhasznált szoftverek	12
2.4	Domain és tárhely	12
2.5	Github	13
2.6	adatbázis.....	13
2.6.1	Adatbázismodell-diagramja	14
2.6.2	Adatbázis szerkezete	14
2.6.3	Kapcsolatok.....	18
2.7	Fő funkciók.....	19
2.8	Projekt struktúra	19
2.8.1	Fő könyvtárak.....	20
2.9	adatmodellek.....	21
2.9.1	Circuits (pályák).....	21
2.9.2	Driver_contracts (Versenyző szerződések).....	21
2.9.3	Drivers (Versenyzők).....	22

2.9.4	Engines (Motorok)	22
2.9.5	HomepageSettings (Fő oldali beállítások)	23
2.9.6	Races (Versenyek)	23
2.9.7	RaceSchedules (Versenynaptár)	23
2.9.8	Seasons (Szabályváltozás szezonjai)	24
2.9.9	Sponsors (Szponzorok)	24
2.9.10	Team_sponsors (Csapat szponzorjai)	24
2.9.11	Teams (Csapatok)	24
2.9.12	User (felhasználó)	25
2.9.13	UserFavorites (Felhasználó kedvence)	25
2.9.14	F1ZoneDbContext	25
2.10	Controllers (vezérlők)	27
2.10.1	GenericController	28
2.10.2	AuthController	29
2.10.3	CircuitsController	31
2.10.4	Driver_contractsController	32
2.10.5	DriversController	32
2.10.6	EnginesController	35
2.10.7	FavoritesController	35
2.10.8	HomepageSettingsController	37
2.10.9	PastRacesController	38
2.10.10	RaceSchedulesController	38
2.10.11	SeasonsController	39
2.10.12	SponsorsController	39
2.10.13	Team_sponsorsController	39
2.10.14	TeamsController	40
2.11	Services (Szolgáltatások)	40

2.11.1	GenericF1ZoneService	40
2.11.2	Interface (Interfész).....	42
2.11.3	IGenericF1ZoneService	42
2.12	F1Zone.BlazorApp/Program.cs.....	43
2.12.1	Blazor komponensek inicializálása	43
2.12.2	Böngészős adattárolás	43
2.12.3	Hitelesítés és jogosultságkezelés	43
2.12.4	API kommunikáció	44
2.13	F1Zone.API/Program.cs	44
2.13.1	Adatbázis-kapcsolat.....	44
2.13.2	Függőség-injektálás (Dependency Injection)	45
2.13.3	CORS Házirend	45
2.14	F1.razor.....	45
2.14.1	A főoldal felépítése	46
2.14.2	Működés:	49
2.15	Tracks.razor:.....	50
2.15.1	A pályák oldal felépítése:	50
2.15.2	Működés:	52
2.15.3	Fontosabb metódusok.....	53
2.15.4	Felhasznált adatmodell	53
2.16	Drivers.razor:	53
2.16.1	A versenyzők oldal felépítése:	54
2.16.2	Fontosabb metódusok:	56
2.16.3	Felhasznált adatmodellek:	56
2.17	DriverDetail.razor	57
2.17.1	A versenyző részletező oldal felépítése:.....	57
2.17.2	A versenyző statisztika oldal működése:	61

2.17.3	Fontosabb metódusok:	62
2.17.4	Felhasznált adatmodellek:	62
2.18	Schedule.razor	63
2.18.1	A versenynaptár oldal felépítése:.....	63
2.18.2	A versenynaptár oldal működése:.....	65
2.18.3	Fontosabb metódusok:	66
2.18.4	Felhasznált adatmodellek	66
2.19	News.razor	67
2.19.1	A hírek oldal felépítése:	67
2.19.2	A hírek oldal működése:	70
2.19.3	Fontosabb metódusok:	71
2.19.4	Felhasznált adatmodellek:	71
2.20	RegulationChanges.razor	72
2.20.1	A szabályváltozások oldal felépítése:	73
2.20.2	A szabályváltozások oldal működése:	74
2.20.3	Fontosabb metódusok	75
2.20.4	Felhasznált adatmodellek	75
2.21	Strategy.razor	76
2.21.1	A stratégia oldal felépítése.....	76
2.21.2	A stratégia oldal működése:.....	79
2.21.3	Fontosabb metódusok	80
2.21.4	Felhasznált adatmodellek	81
2.22	AddDriver.razor	82
2.22.1	Az új pilóta felvétele oldal felépítése:	82
2.22.2	Az új pilóta felvétele oldal működése	84
2.22.3	Fontosabb metódusok	84
2.22.4	Felhasznált adatmodellek	84

2.23	AdminDashboard.razor	85
2.23.1	Az adminisztrációs főoldal felépítése:	86
2.23.2	Az adminisztrációs főoldal működése:	87
2.23.3	Fontosabb metódusok:	87
2.23.4	Felhasznált adatmodellek:	88
2.24	AdminHome.razor	88
2.24.1	A főoldali beállítások oldal felépítése:	89
2.24.2	A főoldali beállítások oldal működése:	90
2.24.3	Fontosabb metódusok:	91
2.24.4	Felhasznált adatmodellek:	91
2.25	EditDriver.razor	92
2.25.1	Az oldal felépítése:	92
2.25.2	Az oldal működése:	94
2.25.3	Fontosabb metódusok:	94
2.25.4	Felhasznált adatmodell:	95
2.26	Login.razor	95
2.26.1	A bejelentkezési oldal felépítése:	95
2.26.2	A bejelentkezési oldal működése:	96
2.26.3	Fontosabb metódusok:	97
2.26.4	Felhasznált adatmodellek:	97
2.27	Register.razor	98
2.27.1	A regisztrációs oldal felépítése	98
2.27.2	A regisztrációs oldal működése	99
2.27.3	Fontosabb metódusok	100
2.27.4	Felhasznált adatmodellek	100
2.28	RaceDetails.razor	101
2.28.1	Az oldal felépítése	101

2.28.2	Az oldal működése	102
2.28.3	Fontosabb metódusok	103
2.28.4	Felhasznált adatmodell	103
2.29	RaceResult.razor	103
2.29.1	A versenyeredmény oldal felépítése	104
2.29.2	A versenyeredmény oldal működése	105
2.29.3	Fontosabb metódusok	106
2.29.4	Felhasznált adatmodellek	106
2.30	Képernyőtervek.....	107
2.31	Bejelentkezési adatok	111
2.32	Fejlesztési lehetőségek.....	112
2.33	Tesztelési terv	112
2.33.1	Tesztelési célok	113
2.33.2	Tesztelési típusok	113
2.33.3	Tesztelési környezet	114
2.33.4	Tesztesetek	115
3	Felhasználói dokumentáció	118
3.1	A Program céljának és lényegesebb funkcióinak összefoglalása	118
3.2	Szükséges hardvereszközök és szoftverek felsorolása	118
3.3	Telepítés és indítás lépéseinek ismertetése	118
3.4	A program részletes bemutatása	119
3.4.1	Látogatói (vendég) funkciók	119
3.4.2	Fiókkezelés és Hitelesítés	126
3.4.3	Regisztrált felhasználói funkciók	127
3.4.4	Adminisztrátor funkciók	128
3.4.5	Az oldalt segítő tájékoztatói opciók.....	131
4	Összefoglalás	133

4.1	Köszönetnyilvánítás	133
4.2	Elkészült projektünk értékelése.....	134

1 BEVEZETÉS

A projektünk keretében egy Formula–1 témájú full-stack webalkalmazást készítettünk, amely célja, hogy átfogó, naprakész és jól strukturált információkat nyújtson a sportág iránt érdeklődők számára. Az alkalmazás fókuszában a modern Formula-1 áll, az aktuális versenynaptár, a pilóták és csapataik adatai, miközben történelmi kontextust is biztosít a sportág jelentősebb technikai korszakainak (szabályváltozásainak) és a legendás versenypályák specifikációinak bemutatásával.

1.1 PROJEKT CÉLJA

A projekt célja egy olyan weboldal létrehozása volt, amely strukturált formában mutatja be a Formula–1 világot, fókuszálva a modern érára és az aktuális szezon eseményeire:

- a versenynaptárban szereplő, valamint a sportág történelmében ikonikus versenypályák paramétereit és jellemzőit,
- az aktuális bajnokságban részt vevő csapatok és pilóták legfontosabb adatait,
- az adott szezon versenynaptárát.

Kiemelt célunk volt, hogy az oldal ne csupán egy sablon legyen, hanem megfelelő kontextust is teremtsen a sportág megértéséhez. Bár az adatok feltöltésénél racionális okokból a jelenlegi Formula–1-et és az aktuális szezont helyeztük előtérbe, a rendszer architektúráját úgy alakítottuk ki, hogy bemutassa a történelmi mérföldköveket is. A jelentősebb technikai korszakok (szabályváltozások) és a klasszikus pályák adatainak integrálásával a felhasználók átláthatják, milyen keretek és adottságok határozzák meg a Forma–1 mai arculatát, anélkül, hogy elvesznének az évtizedes statisztikák sűrűjében.

1.2 A TÉMA KIVÁLASZTÁSÁNAK INDOKLÁSA

A Formula–1 egy világszerte ismert és összetett sportág, amely kiválóan alkalmas egy full-stack webalkalmazás megvalósítására. A téma lehetőséget biztosít nagy mennyiségű adat kezelésére, összetett adatbázis-tervezésre, valamint frontend és backend technológiák együttes alkalmazására. A sportág sajátosságaiból adódóan az adatok térben (különböző kontinensek versenypályái) és időben (aktuális szezonok, technikai korszakok és szabályváltozások) is tagoltak. Ez a sokrétűség valós szoftverfejlesztési kihívásokat teremt, az összetett lekérdezések, valamint az adatintegritás gyakorlati bemutatására. Emellett személyes érdeklődésünk is hozzájárult a téma kiválasztásához.

1.3 CÉLKÖZÖNSÉG

A weboldal célközönségét elsősorban a Formula–1 iránt érdeklődő rajongók alkotják, akik szeretnék mélyebb betekintést nyerni a sportág történetébe és működésébe. A rendszert úgy terveztük meg, hogy több felhasználói réteget is kiszolgáljon: egyaránt célcsoportot jelentenek az aktuális ERRE FIGYELJEK ITT ->futameredményeket, pilótákat és a versenynaptárat követő „alkalmi” nézők, valamint a specifikusabb adatok (például részletes pályaparaméterek, történelmi technikai korszakok és szabályváltozások) iránt érdeklődő elkötelezettebb rajongók. Továbbá az oldal hasznos platform lehet diákok és a sportág iránt újonnan érdeklődők számára is, akik információgyűjtés céljából keresnek egy megbízható, könnyen átlátható és logikusan rendszerezett adatforrást. Mivel az alkalmazás interaktív funkciókat is kínál (például kedvenc elemek mentése felhasználói fiókokhoz), a célközönség kiegészül azokkal a felhasználókkal is, akik személyre szabott élményre vágynak a sportág követése során.

1.4 CSOPORT TAGJAI ÉS MUNKAMEGOSZTÁS

A projektet két fős csoportban valósítottuk meg. A feladatokat a frontend és backend fejlesztés területén közel egyenlő arányban osztottuk fel.

1.4.1 MUNKAMEGOSZTÁS

1.4.2 KÖZÖS FEJLESZTÉSEK

- A projekt koncepciójának és funkcióinak megtervezése
- Az adatmodell és az alkalmazás főbb struktúráinak közös kialakítása
- A frontend és backend közötti kommunikáció megtervezése

1.4.3 FEJLESZTŐK EGYÉNI HOZZÁJÁRULÁSAI

- **Besze László**
Főként az adatbázis felépítéséért és a backend logikai működéséért felelt. Az adatkezeléssel és az adatszerkezet kialakításával kapcsolatos megoldások jelentős része az ő munkájához köthető.
- **Bártfai Levente**
Elsősorban a frontend fejlesztéséért és a felhasználói felület megtervezéséért volt felelős. Az alkalmazás megjelenését, használhatóságát és vizuális kialakítását ő határozta meg, valamint kreatív ötletekkel járult hozzá a projekthez.
- Fontos kiemelni, hogy mindkét fő részt vett a BACKEND és FRONTEND fejlesztésében.

1.5 A PROGRAM LÉTREJÖTTE

- A program az iskolai tanulmányaink befejezésének utolsó év végi vizsgájához készült. Ahová egy szabadon választott témában kellett projektet készíteni, amihez mi a Forma 1-et választottuk, mint témát.

2 FEJLESZTŐI DOKUMENTÁCIÓ

2.1 ALKALMAZÁS ÁTTEKINTÉS

A F1Zone egy adatvezérelt webalkalmazás, amely Blazor Web App technológiával készült és szorosan együttműködik egy MSSQL adatbázissal. A kliensoldal aszinkron hívásokon keresztül kéri le az adatokat(pl. adott évtized szabályai, jelenlegi pilóták listája) a szerveroldalról.

2.2 HASZNÁLT TECHNOLOGIÁK

- Frontend (Kliensoldal): A felhasználói felület kialakítása *ASP.NET Core Blazor* keretrendszerben készült. Ez a technológia lehetővé teszi a gazdag, interaktív webes felületek létrehozását C# alapon. A megjelenés formázását és reszponzivitását *HTML5* és *CSS3* technológiák biztosítják.
- Backend (szerveroldal) és Adatbázis: A szerveroldali logika C# nyelven, a *.NET keretrendszerre* építve valósult meg. Az adatok tárolásáért egy relációs adatbázis-kezelő rendszer, a *Microsoft SQL Server (MSSQL)* felel. A backend és az adatbázis közötti kommunikációt az *Entity Framework Core ORM* (Object-Relational Mapping) eszköz biztosítja.
- Fejlesztői eszközök:
 - Kódszerkesztő: Visual Studio 2022
 - Adatbázis-menedzsment: SQL Server Management Studio (SSMS)
 - Verziókövetés: Git és Github
 - Operációs rendszer: Windows 11 ^{64x}
- Felhasznált programozási nyelvek:
 - C#: A projekt fő programozási nyelve. A választásunk mert ez egy modern objektumorientált nyelv, amely kiválóan támogatja a skálázható, biztonságos webalkalmazások fejlesztését a .NET ökoszisztémában

2.3 FELHASZNÁLT SZOFTVEREK

- **Visual Studio 2022**

A projekt kliensoldali (Blazor Web App) és szerveroldali (ASP.NET Core Web API) rétegének fejlesztése egyaránt C# nyelven, a Visual Studio 2022 integrált fejlesztőkörnyezetben valósult meg. Ezt a platformot azért választottuk, mert beépített eszközeivel – mint például a fejlett lépésenkénti hibakereső (debugger), az integrált NuGet csomagkezelő, valamint a zökkenőmentes projektkezelés – rendkívül hatékonyá teszi mind a backend, mind a frontend fejlesztési munkafolyamatokat.

- **SQL Server Management Studio 21**

Az adatbázis tervezéséhez, adminisztrációjához és karbantartásához a Microsoft SQL Server (MSSQL) relációs adatbázis-kezelőt, illetve az ehhez tartozó SQL Server Management Studio (SSMS) 21-es verzióját alkalmaztuk. Az MSSQL mellett a .NET keretrendszerrel (és az Entity Framework Core-ral) való integrációja miatt döntöttünk. A rendszer kiváló teljesítményt, megbízható adatkezelést és rugalmas skálázhatóságot garantál az F1Zone jelenlegi és az esetleges történelmi adatállományainak kezelésekor. Az SSMS grafikus felülete nagyban megkönnyítette a lekérdezések gyors tesztelését, a táblaszerkezetek vizuális módosítását és a teljesítmény optimalizálását.

2.4 DOMAIN ÉS TÁRHELY

- A projekt domain címe: www.f1zone.hu
- A domain vásárlás a rackhost oldalán történt.

2.5 GITHUB

A csapatmunka és a projekt gördülékeny lebonyolítása érdekében a fejlesztési folyamatokat és a verziókövetést a GitHub platformon koordináltuk. Az egységes hozzáférés biztosítása céljából minden csapattag a hivatalos, intézményi (@ganziskola.hu végződésű) e-mail-címével regisztrált a rendszerbe.

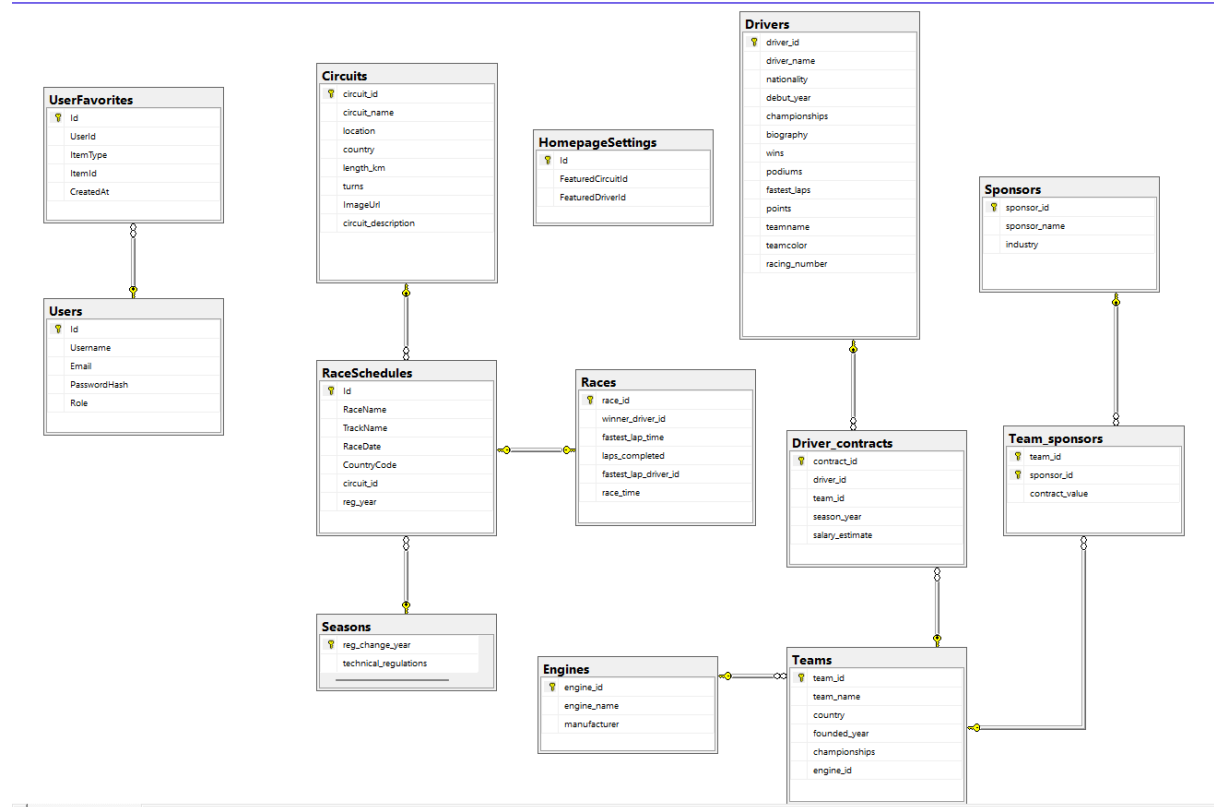
A fejlesztéshez létrejött egy külön repository vizsgaremek néven, amely az alábbi címen érhető el: <https://github.com/beszelaszlohihi/projektmunka>

Ez a központi tárhely biztosítja a projekt során keletkezett összes állomány – beleértve a C# forráskódot, az MSSQL adatbázis-szkripteket és a dokumentációt is – egységes, biztonságos és strukturált tárolását.

2.6 ADATBÁZIS

Az f1_history egy olyan adatbázis, amelyet Forma-1-es adatok tárolására terveztek, mint a pályák vagy a pilóták adatai. Az adatbázis ezek mellett tárolja a versenyek, szponzorok, motorok és a szabályváltozások adatait.

2.6.1 ADATBÁZISMODELL-DIAGRAMJA



Az adatbázisunk három részre bontva valósult meg az egyszerűség érdekében:

- Felhasználó és a hozzá kapcsolható „kedvenc” funkció.
- Pályák és a versenyek adatai, szabálmódosítások, illetve versenynaptár.
- Pilóták adatai csapataikkal és azoknak is az adataik.

2.6.2 ADATBÁZIS SZERKEZETE

- Engines tábla:

Leírás: A csapatok által használt erőforrásokat és azok gyártóit listázza.

Mező neve	Adattípus	Megkötések
engine_id	INT	PK, NOT NULL
engine_name	NVARCHAR(100)	NOT NULL
manufacturer	NVARCHAR(100)	-

○ Seasons tábla:

Leírás: A Forma–1 történelmének jelentősebb technikai korszakait és a hozzájuk tartozó szabályrendszereket rögzíti az évszám alapján.

<i>Mező neve</i>	<i>Adattípus</i>	<i>Megkötések</i>
reg_change_year	INT	PK, NOT NULL
technical_regulations	NVARCHAR(MAX)	-

○ Circuits tábla:

Leírás: A versenypályák statikus, fizikai adatait (hossz, kanyarok száma) és vizuális információit (leírás, kép URL) tartalmazza.

<i>Mező neve</i>	<i>Adattípus</i>	<i>Megkötések</i>
circuit_id	INT	PK, NOT NULL
circuit_name	NVARCHAR(100)	NOT NULL
location	NVARCHAR(100)	-
country	NVARCHAR(50)	-
length_km	DECIMAL(5,3)	-
turns	INT	-
ImageUrl	NVARCHAR(MAX)	-
circuit_description	NVARCHAR(MAX)	-

○ Races tábla:

Leírás: A már lezajlott naptári események konkrét versenyeredményeit rögzíti (győztes, leggyorsabb kört futó pilóta, futamidő).

<i>Mező neve</i>	<i>Adattípus</i>	<i>Megkötések</i>
race_id	INT	PK, NOT NULL
winner_driver_id	INT	-
fastest_lap_time	time(7)	-
laps_completed	INT	-
fastest_lap_driver_id	INT	-
race_time	time(7)	-

- Sponsors tábla:

Leírás: A sportágban jelen lévő támogató cégek, szponzorok és partnerek alapadatait (név, iparág) tartalmazza.

<i>Mező neve</i>	<i>Adattípus</i>	<i>Megkötések</i>
sponsor_id	INT	PK, NOT NULL
sponsor_name	NVARCHAR(100)	NOT NULL
industry	NVARCHAR(50)	-

- Team_sponsors tábla:

Leírás: Egy klasszikus kapcsolótábla, amely összerendeli a csapatokat a saját szponzoriakkal, és nyilvántartja az adott szponzori szerződés értékét.

<i>Mező neve</i>	<i>Adattípus</i>	<i>Megkötések</i>
team_id	INT	PK, FK (ref. Teams)
sponsor_id	INT	PK, FK (ref. Sponsors)
contract_value	DECIMAL(15,2)	-

- Driver_contracts tábla:

Leírás: A pilóták és a csapatok közötti kapcsolatot írja az adott szezonig, kiegészítve a becsült fizetési adatokkal.

<i>Mező neve</i>	<i>Adattípus</i>	<i>Megkötések</i>
contract_id	INT	PK, IDENTITY(1,1)
driver_id	INT	FK (ref. Drivers)
team_id	INT	FK (ref. Teams)
season_year	INT	-
salary_estimate	DECIMAL(15,2)	-

○ Drivers tábla:

Leírás: A pilóták személyes profilját, biográfiáját és a karrierjük legfontosabb statisztikáit (bajnoki címek, győzelmek, szerzett pontok) gyűjti össze.

<i>Mező neve</i>	<i>Adattípus</i>	<i>Megkötések</i>
driver_id	INT	PK, NOT NULL
driver_name	NVARCHAR(100)	NOT NULL
nationality	NVARCHAR(50)	-
debut_year	INT	-
championships	INT	-
biography	NVARCHAR(MAX)	-
wins	INT	-
podiums	INT	-
fastest_laps	INT	-
points	DECIMAL(10,1)	-
teamname	NVARCHAR(50)	-
teamcolor	NVARCHAR(10)	-
racing_number	INT	-

○ Teams tábla:

Leírás: A konstruktőrök (csapatok) alapadatait és történelmi eredményeit tárolja, valamint megmutatja, milyen motort használnak.

<i>Mező neve</i>	<i>Adattípus</i>	<i>Megkötések</i>
team_id	INT	PK, NOT NULL
team_name	NVARCHAR(100)	NOT NULL
country	NVARCHAR(50)	-
founded_year	INT	-
championships	INT	-
engine_id	INT	FK (ref. Engines)

○ HomepageSettings tábla:

Leírás: A weboldal főoldalának dinamikus beállításait tárolja például, hogy éppen melyik pálya vagy pilóta legyen kiemelve az oldalon.

<i>Mező neve</i>	<i>Adattípus</i>	<i>Megkötések</i>
Id	INT	PK, NOT NULL
FeaturedCircuitId	INT	NOT NULL
FeaturedDriverId	INT	NOT NULL

- RaceSchedules tábla:

Leírás: A versenynaptár "gerince". Definiálja a futamok idejét, nevét, és összeköti az adott eseményt a helyszínnel (Circuits) és az érvényben lévő szabályrendszerrel (Seasons).

Mező neve	Adattípus	Megkötések
Id	INT	PK, NOT NULL
RaceName	NVARCHAR(100)	NOT NULL
TrackName	NVARCHAR(100)	NOT NULL
RaceDate	DATETIME	NOT NULL
CountryCode	NVARCHAR(10)	NOT NULL
circuit_id	INT	FK(ref. Circuits)
reg_year	INT	FK(ref. Seasons)

- UserFavorites tábla:

Leírás: Nyilvántartja a felhasználók által kedvelt eleme(ke)t (kedvenc pilótát, pályát), összekötve a felhasználót a kiválasztott entitással.

Mező neve	Adattípus	Megkötések
Id	INT	PK, NOT NULL
UserId	INT	FK(ref. Users), NOT NULL
ItemType	NVARCHAR(50)	-
ItemId	INT	-
CreatedAt	DATETIME	-

2.6.3 KAPCSOLATOK

- Versenynaptár (*RaceSchedules* tábla) kapcsolatai:
 - Minden naptári esemény hivatkozik egy versenypályára (*Circuits* tábla) a circuit_id változóval. Ezzel határozzuk meg a futam pontos helyszínét.
 - Minden esemény hivatkozik egy technikai korszakra/szezonra (*Seasons* tábla) a reg_year oszlopon keresztül. Ez alapján tudjuk, milyen szabályrendszer volt érvényben az adott futamon.
- Futameredmények (*Races* tábla) kapcsolatai:
 - A tábla kulcsa (race_id) egyben hivatkozás is a naptárra (*RaceSchedules* tábla). Ez egy 1:1 kapcsolat, ami biztosítja, hogy csak létező, kiírt eseményhez lehessen eredményt rögzíteni.

- Szerződések (*Driver_contracts* tábla) kapcsolatai:
 - Két hivatkozás található a táblában: a *driver_id* hivatkozik a pilótára (*Drivers* tábla), a *team_id* pedig a csapatra (*Teams* tábla). Ez rögzíti, hogy ki melyik istállónál versenyez.
- Csapatok (*Teams* tábla) kapcsolatai:
 - Minden csapat hivatkozik egy motorgyártóra (*Engines* tábla) az *engine_id* változóval, ami a használt erőforrást mutatja.
- Csapat szponzorok (*Team_sponsors* tábla) kapcsolatai:
 - Kapcsolótáblaként két hivatkozás van benne: a *team_id* hivatkozik a csapatra (*Teams* tábla), a *sponsor_id* pedig a szponzorra (*Sponsors* tábla). Ez teszi lehetővé a több-a-többhöz kapcsolatot a támogatók és az istállók között.
- Felhasználói kedvencek (*UserFavorites* tábla) kapcsolatai:
 - A *UserId* változó hivatkozik a felhasználók táblájára (*Users* tábla), így tudjuk egyértelműen azonosítani, hogy a kimentett adat melyik regisztrált fiókhoz tartozik.

2.7 FŐ FUNKCIÓK

- **Nem bejelentkezett látogatók számára:** Versenyzők, pályák, hírek, szabályok oldalak megtekintése
- **Bejelentkezett látogatók számára:** Az előbb említetteken kívül, versenyzők „kedvencezése”, hírek oldalon gomb, ami a kedvenc pilóta híreit listázza csak, stratégia oldal elérhetősége
- **Adminisztrátor számára:** Fő oldalon a „Legutóbbi futam”, illetve a „Driver of the Day” módosítása, Adatok szerkesztése oldal láthatósága

2.8 PROJEKT STRUKTÚRA

A megoldás egy Blazor Web App alkalmazásból, egy ASP.NET Core Web API-ból és egy közös könyvtárból áll

2.8.1 FŐ KÖNYVTÁRAK

- **F1Zone.API:**

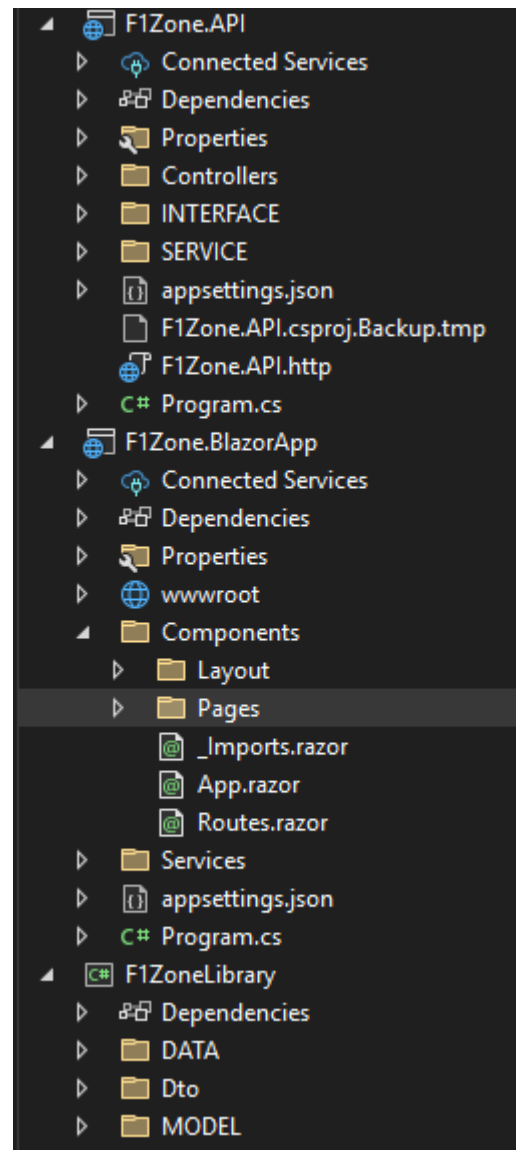
- **Controllers:** Az alkalmazás backendjének a vezérlői.
- **Interface:** Az adatok lekérdezéséhez szükséges metódusok szabályait (szerződéseit) definiálja.
- **Services:** Az Interface mappában található metódusokat megvalósító, a külső API-val tényleg kommunikáló üzleti logikát tartalmazza.

- **F1Zone.BlazorApp:**

- **Pages:** Razor komponensek (oldalak)

- **F1ZoneLibrary:**

- **DATA:** Az adatbázis kapcsolatért és a táblák leképzéseért felelős konfigurációkat tartalmazza.
- **Dto:** Azokat az adatátviteli osztályokat tárolja, amelyek a backend és a kliens közötti biztonságos, optimalizált
- **MODEL:** Az alkalmazás alapvető üzleti objektumait és entitásait definiáló C# osztályokat tartalmazza, amelyek közvetlen leképzései az adatbázis tábláinak, a rendszer adatmodelljének alapját képezik.



2.9 ADATMODELLEK

A rendszer a következő fő entitásokat használja.

2.9.1 CIRCUITS (PÁLYÁK)

```
5 references
public class Circuits
{
    [Key]
    5 references
    public int circuit_id { get; set; }
    2 references
    public string circuit_name { get; set; }
    0 references
    public string location { get; set; }
    1 reference
    public string country { get; set; }
    2 references
    public decimal length_km { get; set; }
    1 reference
    public int turns { get; set; }
    0 references
    public string? imageUrl { get; set; } //képnek a linkje
    0 references
    public string circuit_description { get; set; }
}
```

2.9.2 DRIVER_CONTRACTS (VERSENYZŐ SZERZŐDÉSEK)

```
3 references
public class Driver_contracts
{
    [Key]
    0 references
    public int contract_id { get; set; }
    1 reference
    public int driver_id { get; set; }
    0 references
    public int team_id { get; set; }
    1 reference
    public decimal salary_estimate { get; set; }
}
```

2.9.3 DRIVERS (VERSENYZŐK)

```
16 references
public class Drivers
{
    [Key]
    5 references
    public int driver_id { get; set; }
    13 references
    public string driver_name { get; set; } = string.Empty;
    7 references
    public string? nationality { get; set; }
    5 references
    public int? debut_year { get; set; }
    7 references
    public int? championships { get; set; }
    7 references
    public string? biography { get; set; }
    7 references
    public int? wins { get; set; }
    6 references
    public int? podiums { get; set; }
    6 references
    public int? fastest_laps { get; set; }
    7 references
    public decimal? points { get; set; }
    10 references
    public string? teamname { get; set; }
    4 references
    public string? teamcolor { get; set; }
    4 references
    public int? racing_number { get; set; }
}
```

2.9.4 ENGINES (MOTOROK)

```
3 references
public class Engines
{
    [Key]
    1 reference
    public int engine_id { get; set; }
    0 references
    public string engine_name { get; set; }
    1 reference
    public string manufacturer { get; set; }
}
```

2.9.5 HOMEPAGESETTINGS (FŐ OLDALI BEÁLLÍTÁSOK)

```
3 references
public class HomepageSettings
{
    [Key]
    1 reference
    public int Id { get; set; }
    2 references
    public int FeaturedCircuitId { get; set; } //fő oldalon a pálya id-jét tárolja
    2 references
    public int FeaturedDriverId { get; set; } //fő oldalon a pilóta id-jét tárolja
}
```

2.9.6 RACES (VERSENYEK)

```
7 references
public class Races
{
    [Key]
    0 references
    public int race_id { get; set; }
    1 reference
    public int winner_driver_id { get; set; }
    1 reference
    public TimeSpan fastest_lap_time { get; set; } // óra:perc:mp:ezredmásodperc
    0 references
    public int laps_completed { get; set; }
    1 reference
    public int fastest_lap_driver_id { get; set; }
    1 reference
    public TimeSpan race_time { get; set; }
    2 references
    public int? second_place_driver_id { get; set; }
    2 references
    public int? third_place_driver_id { get; set; }
}
```

2.9.7 RACESCHEDULES (VERSENNAPTÁR)

```
5 references
public class RaceSchedules
{
    0 references
    public int Id { get; set; }
    1 reference
    public string RaceName { get; set; }
    0 references
    public string TrackName { get; set; }
    2 references
    public DateTime RaceDate { get; set; }
    0 references
    public string CountryCode { get; set; }
    0 references
    public int circuit_id { get; set; }
    0 references
    public int reg_year { get; set; }
}
```


2.9.8 SEASONS (SZABÁLYVÁLTOZÁS SZEZONJAI)

```
public class Seasons
{
    [Key]

    public int reg_change_year { get; set; }

    public string technical_regulations { get; set; }
}
```

2.9.9 SPONSORS (SZPONZOROK)

```
public class Sponsors
{
    [Key]

    public int sponsor_id { get; set; }
    1 reference
    public string sponsor_name { get; set; }
    0 references
    public string industry { get; set; }
}
```

2.9.10 TEAM_SPONSORS (CSAPAT SZPONZORJAI)

```
3 references
public class Team_sponsors
{
    [Key]
    1 reference
    public int team_id { get; set; }
    1 reference
    public int sponsor_id { get; set; }
    0 references
    public int contract_value { get; set; }
}
```

2.9.11 TEAMS (CSAPATOK)

```
3 references
public class Teams
{
    [Key]
    1 reference
    public int team_id { get; set; }
    2 references
    public string team_name { get; set; }
    0 references
    public string country { get; set; }
    0 references
    public int founded_year { get; set; }
    0 references
    public int championships { get; set; }
    1 reference
    public int engine_id { get; set; }
}
```

2.9.12 USER (FELHASZNÁLÓ)

```
3 references
public class User
{
    2 references
    public int Id { get; set; }
    3 references
    public string Username { get; set; } = string.Empty;
    4 references
    public string Email { get; set; } = string.Empty;
    2 references
    public string PasswordHash { get; set; } = string.Empty;
    3 references
    public string Role { get; set; } = "User";
}
```

2.9.13 USERFAVORITES (FELHASZNÁLÓ KEDVENCE)

```
7 references
public class UserFavorites
{
    [Key]
    0 references
    public int Id { get; set; }
    5 references
    public int UserId { get; set; }
    3 references
    public string ItemType { get; set; }
    5 references
    public int ItemId { get; set; }
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)] //adatbázis által generált érték
    0 references
    public DateTime? CreatedAt { get; set; } = DateTime.Now;
}
```

2.9.14 F1ZONEDbCONTEXT

Az F1ZoneDbContext osztály az adatbázis-kapcsolatot és adatmodelleket kezeli. Ez az osztály az Entity Framework Core DbContext osztály kiterjesztése, amely a szolgáltatások és a vezérlők számára biztosítja az adatbázis-hozzáférést.

```

15 references
public class F1ZoneDbContext : DbContext
{
    0 references
    public F1ZoneDbContext(DbContextOptions options) : base(options){ }

    0 references
    public F1ZoneDbContext(){ }

    2 references
    public DbSet<HomepageSettings> HomepageSettings { get; set; }
    0 references
    public DbSet<Circuits> Circuits { get; set; }
    1 reference
    public DbSet<Driver_contracts> Driver_contracts { get; set; }
    1 reference
    public DbSet<Drivers> Drivers { get; set; }
    1 reference
    public DbSet<Engines> Engines { get; set; }
    0 references
    public DbSet<Races> Races { get; set; }
    0 references
    public DbSet<Seasons> Seasons { get; set; }
    1 reference
    public DbSet<Sponsors> Sponsors { get; set; }
    1 reference
    public DbSet<Team_sponsors> Team_sponsors { get; set; }
    2 references
    public DbSet<Teams> Teams { get; set; }
    3 references
    public DbSet<User> Users { get; set; }
    1 reference
    public DbSet<RaceSchedules> RaceSchedules { get; set; }
    2 references
    public DbSet<UserFavorites> UserFavorites { get; set; }
}
    
```

2.9.14.1 KÖRNYEZETSPECIFIKÁCIÓS KONFIGURÁCIÓ

Az adatbáziskapcsolat automatikusan a megfelelő konfigurációt választja:

Fejlesztői környezet

- Helyi SQL szerveret használ
- Adatbázis: f1_history
- Kapcsolati sztring:

Data Source=DESKTOP-G8TRHSO\\SQLSERVERBESZE;

Database=f1_history;

User=sa;

Password=Lacus2005;

TrustServerCertificate=true;

Éles Környezet

- **Még nem ismert**

2.10 CONTROLLERS (VEZÉRLŐK)

A projekt egyik kiemelkedő architekturális megoldása a GenericController (generikus vezérlő) alkalmazása. Ennek segítségével a szabványos CRUD (létrehozás, olvasás, módosítás, törlés) műveleteket egyetlen, újrahasznosítható alaposztályban valósítottuk meg a különböző entitásokhoz (pl. pilóták, csapatok), elkerülve ezzel a kódduplikációt. Ezek a vezérlők a szolgáltatásokkal (Services) együttműködve dolgozzák fel a kéréseket, és strukturált (JSON) formátumban adják vissza az adatokat a Blazor kliens számára.

2.10.1 GENERICCONTROLLER

A GenericController egységesen kezeli az alapvető CRUD műveleteket. Ennek köszönhetően a konkrét entitásokhoz tartozó vezérlők kódja minimálisra csökkent, letisztult maradt és csak a speciális, egyedi végpontokat kellett külön megírni bennük.

```
namespace FlZone.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    31 references
    public class GenericController<T> : ControllerBase where T : class
    {
        private readonly IGenericFlZoneService<T> _service;

        15 references
        public GenericController(IGenericFlZoneService<T> service)
        {
            _service = service;
        }

        //összes lekerdezes
        [HttpGet]
        0 references
        public async Task<ActionResult<List<T>>> GetAll()
        {
            return await _service.GetAll();
        }

        //id alapjan
        [HttpGet("{id:int}")]
        0 references
        public async Task<ActionResult<T>> GetById(int id)
        {
            var entity = await _service.GetById(id);
            return Ok(entity);
        }

        //uj létrehozasa
        [HttpPost]
        0 references
        public async Task<ActionResult<T>> Create([FromBody] T entity)
        {
            var created = await _service.Add(entity);
            return Ok(created);
        }

        //frissites
        [HttpPut]
        0 references
        public async Task<ActionResult> Update([FromBody] T entity)
        {
            await _service.Update(entity);
            return NoContent();
        }

        //torles
        [HttpDelete]
        0 references
        public async Task<ActionResult> Delete(T entity)
        {
            await _service.Delete(entity);
            return NoContent();
        }
    }
}
```

Végpontok:

- **GET /api/[controller]** – Visszaadja az entitásnak az elemeit.
- **GET /api/[controller]/{id}** – Visszaad egy konkrét entitást az azonosítója alapján.
- **POST /api/[controller]** – Új entitást hoz létre.
- **PUT /api/[controller]/{id}** – Módosít egy meglévő entitást.
- **DELETE /api/[controller]/{id}** – Töröl egy entitást.

Az alábbi vezérlők a `GenericController`-ből öröklődnek, így automatikusan megvalósítják a fentebb említett CRUD műveleteket anélkül, hogy külön kódot igényelnének.

- `CircuitsController`
- `Driver_contractsController`
- `DriversController`
- `EnginesController`
- `FavoritesController`
- `PastRacesController`
- `Race_resultsController`
- `RaceSchedulesController`
- `RacesController`
- `SeasonsController`
- `SponsorsController`
- `Team_sponsorsController`
- `TeamsController`

2.10.2 AUTHCONTROLLER

Az `AuthController` a felhasználók hitelesítéséért és jogosultságkezeléséért felel, amely biztonságos jelszótíkosítással (BCrypt) kezeli az új felhasználók regisztrációját, és JWT (JSON Web Token) generálásával biztosítja a sikeres bejelentkezést, valamint a szerepkörök (Admin/User) kiosztását.

```

namespace F1Zone.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    1 reference
    public class AuthController : ControllerBase
    {
        private readonly F1ZoneDbContext _context;
        private readonly IConfiguration _configuration;

        0 references
        public AuthController(F1ZoneDbContext context, IConfiguration configuration)
        {
            _context = context;
            _configuration = configuration;
        }

        [HttpPost("register")]
        0 references
        public async Task<IActionResult> Register([FromBody] RegisterDto model)
        {
            //letezik e az email
            if (await _context.Users.AnyAsync(u => u.Email == model.Email))
            {
                return BadRequest("Ez az email cím már foglalt!");
            }

            //Jelszó titkosítása

            string passwordHash = BCrypt.Net.BCrypt.HashPassword(model.Password);

            // Új felhasználó objektum létrehozása
            var user = new User
            {
                Email = model.Email,
                Username = model.Username,
                PasswordHash = passwordHash,
                Role = model.Role
            };

            // Mentés az adatbázisba
            _context.Users.Add(user);
            await _context.SaveChangesAsync();

            return Ok("Sikeres regisztráció!");
        }
    }
}

```

```

[HttpPost("login")]
0 references
public async Task<IActionResult> Login([FromBody] LoginDto model)
{
    //Megkeressük a felhasználót az email alapján
    var user = await _context.Users.FirstOrDefaultAsync(u => u.Email == model.Email);

    //Ha nincs ilyen email, vagy a jelszó NEM egyezik a tárolt hash-sel
    if (user == null || !BCrypt.Net.BCrypt.Verify(model.Password, user.PasswordHash))
    {
        return Unauthorized("Hibás email vagy jelszó!");
    }

    var token = CreateToken(user);

    return Ok(new { Id = user.Id, Token = token, Username = user.Username, Role = user.Role });
}

//createtoken metodus létrehozása
1 reference
private string CreateToken(User user)
{
    var claims = new List<Claim>
    {
        new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
        new Claim(ClaimTypes.Name, user.Username),
        new Claim(ClaimTypes.Email, user.Email),
        new Claim(ClaimTypes.Role, user.Role)
    };

    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(
        _configuration.GetSection("AppSettings:Secret").Value!));

    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha512Signature);

    var tokenDescriptor = new SecurityTokenDescriptor
    {
        Subject = new ClaimsIdentity(claims),
        Expires = DateTime.Now.AddDays(1),
        SigningCredentials = creds
    };

    var tokenHandler = new JwtSecurityTokenHandler();
    var token = tokenHandler.CreateToken(tokenDescriptor);

    return tokenHandler.WriteToken(token);
}
}

```

Végpontok:

- POST /api/Auth/register – új felhasználó létrehozása az adatbázisban
- POST /api/Auth/login – felhasználói bejelentkezés

2.10.3 CIRCUITSCONTROLLER

A CircuitsController felelős az adatbázisban található pályáknak az adatainak a megjelenítéséért.


```
namespace F1Zone.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    1 reference
    public class CircuitsController : GenericController<Circuits>
    {
        0 references
        public CircuitsController(IGenericF1ZoneService<Circuits> service) : base(service) { }
    }
}
```

2.10.4 DRIVER_CONTRACTS_CONTROLLER

A Driver_contractsController felelős a pilóták szerződéseinek részletével kapcsolatos információk kezeléséért.

```
namespace F1Zone.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    1 reference
    public class Driver_contractsController : GenericController<Driver_contracts>
    {
        0 references
        public Driver_contractsController(IGenericF1ZoneService<Driver_contracts> service) : base(service) { }
    }
}
```

2.10.5 DRIVERS_CONTROLLER

A DriversController felelős a pilótákkal kapcsolatos műveletek végrehajtásáért. Átfogó lekérdezésekkel és egyedi szűréssel egészít ki kliens és adminisztrációs felület igényeinek megfelelően.

```
namespace F1Zone.API.Controllers
{
    [Route("api/drivers")]
    [ApiController]
    1 reference
    public class DriversController : GenericController<Drivers>
    {
        private readonly IGenericF1ZoneService<Drivers> _service;

        private readonly F1ZoneDbContext _context;

        0 references
        public DriversController(IGenericF1ZoneService<Drivers> service, F1ZoneDbContext context) : base(service)
        {
            _service = service;
            _context = context;
        }
    }
}
```

```
[HttpGet("{name}")]
0 references
public async Task<ActionResult<DriverDto>> GetDriverByName(string name)
{
    var searchName = name.Replace("-", " ");

    //Összekötjük a pilótát a szerződéssel és a motorral
    var driverData = await _context.Drivers
        .Where(d => d.driver_name == searchName)
        .Select(d => new DriverDto
        {
            driver_id = d.driver_id,
            driver_name = d.driver_name,
            wins = d.wins,
            podiums = d.podiums,
            fastest_laps = d.fastest_laps,
            championships = d.championships,
            points = d.points,
            biography = d.biography,
            nationality = d.nationality,
            teamname = d.teamname,

            // JOIN a driver_contracts táblához
            salary_estimate = _context.Driver_contracts
                .Where(c => c.driver_id == d.driver_id)
                .Select(c => c.salary_estimate)
                .FirstOrDefault(),

            team_sponsors = _context.Team_sponsors
                .Where(ts => ts.team_id == _context.Teams
                    .Where(t => t.team_name == d.teamname)
                    .Select(t => t.team_id)
                    .FirstOrDefault())
                .Join(_context.Sponsors,
                    ts => ts.sponsor_id,
                    s => s.sponsor_id,
                    (ts, s) => s.sponsor_name)
                .ToList(),

            // JOIN a teams és engines táblákhoz
            manufacturer = (from t in _context.Teams
                            join e in _context.Engines on t.engine_id equals e.engine_id
                            where t.team_name == d.teamname
                            select e.manufacturer).FirstOrDefault()
        })
        .FirstOrDefaultAsync();

    if (driverData == null) return NotFound();

    return Ok(driverData);
}
```

```
[HttpGet("admin-list")]
D references
public async Task<ActionResult<IEnumerable<DriverDto>>> GetDriversForAdmin()
{
    var drivers = await _service.GetAll();

    var result = drivers.Select(d => new DriverDto
    {
        driver_id = d.driver_id,
        driver_name = d.driver_name,
        wins = d.wins,
        championships = d.championships,
        points = d.points,
        nationality = d.nationality,
        debut_year = d.debut_year,
        podiums = d.podiums,
        fastest_laps = d.fastest_laps,
        biography = d.biography,
        teamname = d.teamname,
        teamcolor = d.teamcolor,
        racing_number = d.racing_number
    });

    return Ok(result);
}

[HttpPut("{id}")]
D references
public async Task<ActionResult> UpdateDriver(int id, [FromBody] DriverDto driverDto)
{
    var driver = await _service.GetById(id);
    if (driver == null) return NotFound("A pilóta nem található.");

    driver.driver_name = driverDto.driver_name;
    driver.wins = driverDto.wins;
    driver.championships = driverDto.championships;
    driver.points = driverDto.points;
    driver.nationality = driverDto.nationality;
    driver.debut_year = driverDto.debut_year;
    driver.podiums = driverDto.podiums;
    driver.fastest_laps = driverDto.fastest_laps;
    driver.biography = driverDto.biography;

    try
    {
        await _service.Update(driver);
        return NoContent();
    }
    catch (Exception ex)
    {
        Console.WriteLine($"HIBA: {ex.Message}");
        return BadRequest("Hiba a mentés során.");
    }
}

[HttpGet("team/{teamName}")]
D references
public async Task<ActionResult<IEnumerable<DriverDto>>> GetDriversByTeam(string teamName)
{
    var allDrivers = await _service.GetAll();

    var teamDrivers = allDrivers
        .Where(d => d.teamname != null && d.teamname.Equals(teamName, StringComparison.OrdinalIgnoreCase))
        .Select(d => new DriverDto
        {
            driver_id = d.driver_id,
            driver_name = d.driver_name,
            wins = d.wins,
            championships = d.championships,
            points = d.points,
            nationality = d.nationality,
            biography = d.biography,
            teamname = d.teamname
        });

    return Ok(teamDrivers);
}
```

Egyedi végpontok:

- **GET /api/drivers/{name}** – részletes profilt ad vissza egy adott pilótáról a neve alapján
- **GET /api/drivers/admin-list** – adminisztrációs felület számára biztosít egy minden pilótát tartalmazó listát
- **PUT /api/drivers/{id}** – meglévő pilóta adatainak a módosítását teszi lehetővé
- **GET /api/drivers/team/{teamName}** – csapatnév alapján szűri és listázza a pilótákat

2.10.6 ENGINESCONTROLLER

Az EnginesController felelős a motor szállítókkal kapcsolatos műveletek kezeléséért a rendszerben.

```
namespace FlZone.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class EnginesController : GenericController<Engines>
    {
        public EnginesController(IGenericFlZoneService<Engines> service) : base(service) { }
    }
}
```

2.10.7 FAVORITESCONTROLLER

A FavoritesController a GenericController-ből örököltetve kezeli a felhasználók által elmentett kedvenc pilótákat, mely egy összetett törlési logikával egészül ki a pontos adatbázis kezelés érdekében.

```
namespace F1Zone.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class FavoritesController : GenericController<UserFavorites>
    {
        private readonly F1ZoneDbContext _context;

        public FavoritesController(IGenericF1ZoneService<UserFavorites> service, F1ZoneDbContext context)
            : base(service)
        {
            _context = context;
        }

        // EGYEDI TÖRLÉS: userId és itemId alapján
        [HttpDelete("{userId}/{driverId}")]
        public async Task<ActionResult> DeleteFavorite(int userId, int driverId)
        {
            var fav = await _context.UserFavorites
                .FirstOrDefaultAsync(f => f.UserId == userId && f.ItemId == driverId);

            if (fav == null) return NotFound();

            _context.UserFavorites.Remove(fav);
            await _context.SaveChangesAsync();
            return NoContent();
        }
    }
}
```

Egyedi végpont:

- **DELETE /api/favorites/{userId}/{driverId}** - Lehetővé teszi egy adott felhasználó listájából egy konkrét kedvenc pilóta eltávolítását.

2.10.8 HOMEPAGESettingsController

A HomepageSettingsController a weboldal kezdőlapjának dinamikus tartalmáért felelős. Mivel ez a vezérlő egyedi, konfigurációs jellegű adatokat kezel.

```
namespace F1Zone.API.Controllers
{
    [Route("api/homepagesettings")]
    [ApiController]
    1 reference
    public class HomepageSettingsController : ControllerBase
    {
        private readonly F1ZoneDbContext _context;

        0 references
        public HomepageSettingsController(F1ZoneDbContext context)
        {
            _context = context;
        }

        [HttpGet("{id}")]
        0 references
        public async Task<ActionResult<HomepageSettings>> GetById(int id)
        {
            var settings = await _context.HomepageSettings.FindAsync(id);

            if (settings == null)
                return NotFound();

            return Ok(settings);
        }

        [HttpPut("{id}")]
        0 references
        public async Task<IActionResult> Update(int id, [FromBody] HomepageSettings model)
        {
            if (id != model.Id)
                return BadRequest("Az ID nem egyezik.");

            var existing = await _context.HomepageSettings.FindAsync(id);

            if (existing == null)
                return NotFound("A homepage beállítás nem található.");

            existing.FeaturedCircuitId = model.FeaturedCircuitId;
            existing.FeaturedDriverId = model.FeaturedDriverId;

            await _context.SaveChangesAsync();

            return NoContent();
        }
    }
}
```

Egyedi végpontok:

- **GET /api/homepagesettings/{id}** - visszaadja a kezdőlap aktuális beállításait a megadott azonosító alapján.
- **PUT /api/homepagesettings/{id}** - Lehetővé teszi a kezdőlap tartalmának frissítését. A végpont ellenőrzi az azonosítók egyezését, majd felülírja az azonosítót.

2.10.9 PASTRACESCONTROLLER

A PastRacesController a már lezajlott versenyekkel kapcsolatos műveletek kezeléséért felel.

```
namespace F1Zone.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    1 reference
    public class PastRacesController : GenericController<Races>
    {
        0 references
        public PastRacesController(IGenericF1ZoneService<Races> service) : base(service) { }
    }
}
```

2.10.10 RACESCHEDULESCONTROLLER

A RaceSchedulesController a Formula–1-es versenynaptár adatait és az egyes nagydíjak időpontjait kezeli. Az alapvető adatműveleteken felül ez a vezérlő egy egyedi, időalapú lekérdezéssel is kiegészült.

```
namespace F1Zone.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    1 reference
    public class RaceSchedulesController : GenericController<RaceSchedules>
    {
        private readonly F1ZoneDbContext _context;

        0 references
        public RaceSchedulesController(
            IGenericF1ZoneService<RaceSchedules> service,
            F1ZoneDbContext context
        ) : base(service)
        {
            _context = context;
        }

        [HttpGet("next-race")]
        0 references
        public async Task<ActionResult<RaceSchedules>> GetNextRace()
        {
            var nextRace = await _context.RaceSchedules
                .Where(r => r.RaceDate > DateTime.Now)
                .OrderBy(r => r.RaceDate)
                .FirstOrDefaultAsync();

            if (nextRace == null)
                return NotFound("Nincs közelgő futam.");

            return Ok(nextRace);
        }
    }
}
```

Egyedi végpont:

- **GET /api/RaceSchedules/next-race** - Visszaadja a versenynaptár soron következő futamának adatait.

2.10.11 SEASONSCONTROLLER

A SeasonsController felelős a szabályváltozás elemekkel kapcsolatos műveletek kezeléséért.

```
namespace F1Zone.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    1 reference
    public class SeasonsController : GenericController<Seasons>
    {
        0 references
        public SeasonsController(IGenericF1ZoneService<Seasons> service) : base(service) { }
    }
}
```

2.10.12 SPONSORSCONTROLLER

A SponsorsController felelős a szponzorokkal kapcsolatos műveletek végrehajtásáért.

```
namespace F1Zone.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    1 reference
    public class SponsorsController : GenericController<Sponsors>
    {
        0 references
        public SponsorsController(IGenericF1ZoneService<Sponsors> service) : base(service) { }
    }
}
```

2.10.13 TEAM_SPONSORSCONTROLLER

A Team_sponsorsController látja el a csapat szponzorokkal kapcsolatos műveleteket a rendszerben.

```
namespace F1Zone.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    1 reference
    public class Team_sponsorsController : GenericController<Team_sponsors>
    {
        0 references
        public Team_sponsorsController(IGenericF1ZoneService<Team_sponsors> service) : base(service) { }
    }
}
```


2.10.14 TEAMSCONTROLLER

A TeamsController látja el a csapatokkal kapcsolatos feladatokat a rendszerben.

```
namespace F1Zone.API.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    1 reference
    public class TeamsController : GenericController<Teams>
    {
        0 references
        public TeamsController(IGenericF1ZoneService<Teams> service) : base(service) { }
    }
}
```

2.11 SERVICES (SZOLGÁLTATÁSOK)

2.11.1 GENERICF1ZONEERVICE

A GenericF1ZoneService egy általános célú szolgáltatásosztály. Feladata, hogy közvetítő réteget biztosítson a vezérlők (Controllers) és az adatbázis között. Az osztály aszinkron módon kezeli az összes alapvető adatbázis-műveletet (CRUD) bármilyen típusú entitásra így radikálisan csökkentve a kódDuplikációt.

```
namespace F1Zone.API.SERVICE
{
    2 references
    public class GenericF1ZoneService<T> : IGenericF1ZoneService<T> where T : class
    {
        private readonly F1ZoneDbContext _context;
        private readonly DbSet<T> _set;

        0 references
        public GenericF1ZoneService(F1ZoneDbContext context)
        {
            _context = context;
            _set = _context.Set<T>();
        }

        4 references
        public async Task<List<T>> GetAll()
        {
            return await _set.ToListAsync();
        }

        3 references
        public async Task<T> GetById(int id)
        {
            return await _set.FindAsync(id);
        }

        2 references
        public async Task<T> Add(T entity)
        {
            _set.Add(entity);
            await _context.SaveChangesAsync();
            return entity;
        }

        2 references
        public async Task Delete(T entity)
        {
            _set.Remove(entity);
            await _context.SaveChangesAsync();
        }

        3 references
        public async Task Update(T entity)
        {
            _set.Update(entity);
            await _context.SaveChangesAsync();
        }
    }
}
```

A szolgáltatás alapvető metódusai:

- **GetAll()** - Aszinkron módon lekérdezi és egy listába rendezve visszaadja az adott entitástípushoz tartozó összes rekordot az adatbázis táblájából.
- **GetById(int id)** - Elsődleges kulcs alapján hatékonyan megkeres és visszaad egy konkrét rekordot.
- **Add(T entity)** - Új entitást (rekordot) ad hozzá a megfelelő adatbázis-táblához, majd azonnal elmenti a változásokat.
- **Update(T entity)** - Felülírja és frissíti egy már meglévő adatbázis-rekord tulajdonságait a memóriában, majd a módosításokat véglegesíti az SQL szerveren.

- **Delete(T entity)** - Eltávolít egy megadott entitást a kontextusból, majd végrehajtja a tényleges törlést az adatbázisban.

2.11.2 INTERFACE (INTERFÉSZ)

2.11.3 IGENERICF1ZONESERVICE

Az IGenericF1ZoneService egy generikus interfész. Célja, hogy szigorú, egységes és kötelező struktúrát – egy közös szabályrendszert – biztosítson a szolgáltatásréteg (Service Layer) számára. Ennek az interfésznek az alkalmazásával garantálható, hogy a rendszerben lévő összes entitás egységes formában implementálja az alapvető aszinkron adatbázis-műveleteket. Ezt az interfészt a már bemutatott GenericF1ZoneService osztály valósítja meg a gyakorlatban.

```
namespace F1Zone.API.INTERFACE
{
    20 references
    public interface IGenericF1ZoneService<T> where T : class
    {
        4 references
        Task<List<T>> GetAll();
        3 references
        Task<T?> GetById(int id);
        2 references
        Task<T> Add(T entity);
        3 references
        Task Update(T entity);
        2 references
        Task Delete(T entity);
    }
}
```

Az Interface alapvető metódusai:

- **GetAll()** - Előírja egy metódus létezését, amely képes a kért típusú entitások teljes listájának lekérdezésére.
- **GetById(int id)** - Megköveteli egy azonosító alapján történő, egyetlen entitást visszaadó keresőfüggvény meglétét.
- **Add(T entity)** - Definiálja az új rekordok adatbázisba történő felvételéhez szükséges logikát.
- **Update(T entity)** - Előírja a meglévő adatok memóriában és adatbázisban történő biztonságos módosítását.
- **Delete(T entity)** - Szerződésbe foglalja az entitások végleges törlésének képességét.

2.12 F1ZONE.BLAZORAPP/PROGRAM.CS

Ez a fájl a Blazor kliensalkalmazás elsődleges belépési pontja. Mivel a projekt szerveroldali interaktivitást (Blazor Server renderelést) alkalmaz, az alkalmazás logikája a szerveren fut, a felhasználó böngészőjével pedig egy valós idejű, gyors SignalR kapcsolaton keresztül kommunikál. Ebben a fázisban történik a keretrendszer inicializálása. Itt kerülnek beregisztrálásra mindazok az alapvető szolgáltatások – például a backend API-val kommunikáló HTTP kliensek, a hitelesítésért felelős modulok és az állapotkezelők, amelyek a frontend réteg zökkenőmentes és biztonságos működéséhez elengedhetetlenek.

2.12.1 BLAZOR KOMPONENSEK INICIALIZÁLÁSA

```
builder.Services.AddRazorComponents()  
    .AddInteractiveServerComponents();
```

Ez a szakasz regisztrálja a Razor alapú felhasználói felületet, és bekapcsolja a szerveroldali interaktivitást, amely SignalR kapcsolaton keresztül biztosítja a kliens és a szerver közötti valós idejű, gyors kommunikációt.

2.12.2 BÖNGÉSZŐS ADATTÁRÓLÁS

```
builder.Services.AddBlazoredLocalStorage();
```

A rendszer integrálja a LocalStorage csomagot, amely aszinkron hozzáférést biztosít a böngésző helyi tárhelyéhez. Ez a technológia felel a backendtől kapott JWT hitelesítési kulcsok biztonságos, kliensoldali tárolásáért.

2.12.3 HITELESÍTÉS ÉS JOGOSULTSÁGKEZELÉS

```
//Hitelesítési alapok  
builder.Services.AddAuthorizationCore();  
builder.Services.AddCascadingAuthenticationState();  
  
//AuthProvider regisztrálás  
builder.Services.AddScoped<CustomAuthStateProvider>();  
builder.Services.AddScoped<AuthenticationStateProvider>(sp =>  
    sp.GetRequiredService<CustomAuthStateProvider>());
```

Ez a blokk építi fel az alkalmazás biztonsági rétegét. Az *AddCascadingAuthenticationState* gondoskodik arról, hogy a bejelentkezett felhasználó állapota és adatai a teljes komponensfán (globálisan) elérhetőek legyenek. A saját fejlesztésű CustomAuthProvider és AuthService felelnek a tokenek validálásáért és a bejelentkezési folyamat menedzseléséért.

2.12.4 API KOMMUNIKÁCIÓ

```
builder.Services.AddScoped(sp => new HttpClient
{
    BaseAddress = new Uri("https://localhost:7128/")
});
```

Itt történik a HTTP kliens példányosítása és regisztrálása, amelyen keresztül a Blazor felület REST API hívásokat intéz a backend felé. A BaseAddress beállításával a komponensekben elegendő csak a relatív útvonalakat (pl. /api/drivers) megadni.

2.13 F1ZONE.API/PROGRAM.CS

Ez az állomány a szerveroldali (backend) ASP.NET Core Web API projekt hivatalos belépési pontja. Feladata a webes szolgáltatás inicializálása, valamint a bejövő HTTP kérések feldolgozási láncának (middleware pipeline) konfigurálása.

2.13.1 ADATBÁZIS-KAPCSOLAT

```
builder.Services.AddDbContext<F1ZoneDbContext>(o => o.UseSqlServer(builder.Configuration.GetConnectionString("Default")));
```

A backend rendszer adatkezelésének magja. Itt történik az F1ZoneDbContext beregisztrálása és a Microsoft SQL Server adatbázis-kapcsolat konfigurálása. A rendszer az *appsettings.json* fájlból olvassa ki a "Default" nevű kapcsolati karakterláncot (ConnectionString), így a kód biztonságos, nem tartalmaz mereven beégetett jelszavakat vagy szervercímeket.

2.13.2 FÜGGŐSÉG-INJEKTÁLÁS (DEPENDENCY INJECTION)

```
builder.Services.AddScoped(typeof(IGenericF1ZoneService<>), typeof(GenericF1ZoneService<>));
```

Ezen a ponton regisztráljuk be az alkalmazás saját szolgáltatásait a DI konténerbe. A `typeof` operátorok használatával itt adjuk meg, hogy a korábban bemutatott generikus interfészhez a rendszer minden HTTP kérés esetén a `GenericF1ZoneService<T>` megvalósítást rendelje hozzá.

2.13.3 CORS HÁZIREND

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowBlazor",
        policy => policy.WithOrigins("https://localhost:7198")
            .AllowAnyMethod()
            .AllowAnyHeader());
});
```

Biztonsági beállítás, amely engedélyezi, hogy a teljesen külön porton vagy külön szerveren futó Blazor kliensalkalmazás hálózatközi kéréseket intézhessen a backend felé. Az "AllowBlazor" nevű házirend specifikusan a frontend címére korlátozza a hozzáférést, megakadályozva ezzel az illetéktelen külső forrásokból érkező kéréseket.

2.14 F1.RAZOR

Ez az oldal a főoldal. Ha a felhasználó megnyitja az oldalt ezzel találja magát szembe először. Főoldal funkciója A főoldal a rendszer nyitófelülete, amely a felhasználó számára az alkalmazás legfontosabb tartalmait jeleníti meg. Itt található a navigációs sáv, a bemutatkozó hero szekció, a következő futam visszaszámlálója, valamint az adminisztrátor által kiválasztott kiemelt pálya és versenyző. A főoldal célja, hogy gyors áttekintést adjon a Forma–1 világhoz kapcsolódó legfontosabb információkról, és belépési pontot biztosítson az alkalmazás többi funkciójához.

2.14.1 A FŐOLDAL FELÉPÍTÉSE

A főoldal több jól elkülöníthető vizuális és funkcionális egységből épül fel, amelyek együtt biztosítják az alkalmazás átlátható és felhasználóbarát működését.

Az oldal felső részén egy **navigációs sáv** található, amely lehetővé teszi a felhasználó számára az alkalmazás különböző aloldalai közötti gyors váltást. A menüpontok dinamikusan változnak a felhasználó bejelentkezési állapotától és szerepkörétől függően. Bejelentkezett felhasználó esetén megjelenik a felhasználónév és a kijelentkezési lehetőség, míg adminisztrátor esetén további kezelőfelületre mutató link is elérhetővé válik.

```

<nav class="navbar">
  <div class="nav-container">
    <div class="nav-brand">
      <a href="/" class="logo"><span>ZONE</span></a>
    </div>
    <ul class="nav-links @(isMenuOpen ? "active" : "")">
      <li><a href="/" @onclick="CloseMenu">Kezdőlap</a></li>
      <li><a href="/tracks" @onclick="CloseMenu">Pályák</a></li>
      <li><a href="/drivers" @onclick="CloseMenu">Versenyzők</a></li>
      <li><a href="/schedule" @onclick="CloseMenu">Versenynaptár</a></li>
      <li><a href="/news" @onclick="CloseMenu">Hírek</a></li>
      <li><a href="/regulation_changes" @onclick="CloseMenu">Szabályok</a></li>
      <li><a href="/strategy">Stratégia</a></li>

      @* Admin felhasználók számára extra menüpont megjelenítése *@
      @if (isLoggedIn && userRole == "Admin")
      {
        <li><a href="/admin" @onclick="CloseMenu">Adatok szerkesztése</a></li>
      }

      @* Mobilnézetben a felhasználói műveletek külön jelennek meg *@
      <li class="mobile-only">
        @if (isLoggedIn)
        {
          <span class="user-welcome">Üdv, <strong>@userName</strong>!</span>
          <button class="login-btn" @onclick="Logout">KIJELENTKEZÉS</button>
        }
        else
        {
          <button class="login-btn" @onclick="GoToLogin">BEJELENTKEZÉS</button>
        }
      </li>
    </ul>
    <div class="nav-actions">
      @if (isLoggedIn)
      {
        @if (userRole == "Admin")
        {
          <button class="login-btn desktop-only" @onclick="() => NavManager.NavigateTo("/admin/home")">ADMIN</button>
        }
        <span class="user-welcome">Üdv, <strong>@userName</strong>!</span>
        <button class="login-btn desktop-only" @onclick="Logout">KIJELENTKEZÉS</button>
      }
      else
      {
        <button class="login-btn desktop-only" @onclick="GoToLogin">BEJELENTKEZÉS</button>
      }

      @* Hamburger menü mobil eszközökhöz *@
      <button class="burger-btn" @onclick="ToggleMenu">
        @if (isMenuOpen)
        {
          <span>☰</span>
        }
        else
        {
          <span>☷</span>
        }
      </button>
    </div>
  </div>
</nav>

```

A navigációs sáv alatt helyezkedik el a **hero szekció**, amely egy rövid bemutatkozó üzenetet tartalmaz. Ennek célja a felhasználó figyelmének megragadása, valamint az oldal fő témájának – a Forma-1 világának – kiemelése. A szekció tartalmaz egy gombot is, amely a pályák listájához irányítja a felhasználót, ezzel segítve a további navigációt.

```
<section class="hero">
  <h1>Éld át az F1 izgalmát</h1>
  <p>Fedezd fel a világ leglegendásabb versenypályáit, Monaco utcáitól a silverstone-i nagysebességű kanyarokig.</p>
  <a href="/tracks" class="hero-btn">Pályák felfedezése</a>
</section>
```

Ezt követi a **következő futam visszaszámláló szekció**, amely dinamikusan jelenik meg, amennyiben elérhető adat a soron következő versenyről. A blokk megjeleníti a futam nevét, helyszínét, pontos időpontját, valamint egy folyamatosan frissülő visszaszámlálót napokban, órákban, percekben és másodpercekben. Ez az elem kiemelt szerepet játszik a felhasználói élmény növelésében, mivel valós idejű információt biztosít.

```
@if (nextRace != null)
{
  <section class="countdown-section">
    <div class="countdown-container">
      <div class="countdown-header">
        <h2>Következő futam</h2>
        <p>@nextRace.RaceName • @nextRace.TrackName</p>
        <span>@nextRace.RaceDate.ToString("yyyy.MM.dd HH:mm")</span>
      </div>

      <div class="countdown-grid">
        <div class="countdown-item">
          <span>@timeLeft.Days</span>
          <small>Nap</small>
        </div>
        <div class="countdown-item">
          <span>@timeLeft.Hours</span>
          <small>Óra</small>
        </div>
        <div class="countdown-item">
          <span>@timeLeft.Minutes</span>
          <small>Perc</small>
        </div>
        <div class="countdown-item">
          <span>@timeLeft.Seconds</span>
          <small>Mp</small>
        </div>
      </div>
    </div>
  </section>
}

<section class="tracks-section">
  <div class="section-header">
    <h2>Legutóbbi futam</h2>
  </div>
```


Az oldal következő része a **kiemelt pálya megjelenítése**, amely egy információs kártya formájában jelenik meg. Ez tartalmazza a pálya nevét, országát, rövid leírását, valamint technikai adatait, például a pálya hosszát és a kanyarok számát. A tartalom dinamikusan kerül betöltésre az adatbázisból, és adminisztrátor felhasználók számára szerkeszthető.

```
<div class="tracks-grid">
  @* A főoldalra kiemelt pálya megjelenítése.
  Admin számára kattintható, más felhasználóknál csak információs kártya *@
  <article class="track-card @(isLoggedIn && userRole == "Admin" ? "" : "disabled-card")"
    @onclick="NavigateToAdmin"
    style="cursor:@(isLoggedIn && userRole == "Admin" ? "pointer" : "not-allowed");">
    <div class="track-image">
      
    </div>
    <div class="track-content">
      <div class="track-country">@(selectedCircuit?.country ?? "Nincs kiválasztva")</div>
      <h3 class="track-name">@(selectedCircuit?.circuit_name ?? "Nincs kiválasztott pálya")</h3>
      <p>@(selectedCircuit?.circuit_description ?? "Az admin még nem választott ki pályát a főoldalra.")</p>
      <div class="track-stats">
        <div class="stat">
          <div class="stat-value">@(selectedCircuit is not null ? $"{selectedCircuit.length_km} km" : "-")</div>
          <div class="stat-label">Hossz</div>
        </div>
        <div class="stat">
          <div class="stat-value">@(selectedCircuit?.turns.ToString() ?? "-")</div>
          <div class="stat-label">Kanyarok</div>
        </div>
      </div>
    </div>
  </article>
</div>
```

A pálya alatt található a **kiemelt versenyző (Driver of the Day)** megjelenítése, amely hasonló felépítésű kártyában jelenik meg. Ez a blokk bemutatja a kiválasztott versenyző nevét, nemzetiségét, rövid életrajzát, valamint statisztikai adatait, például győzelmeinek, dobogós helyezéseinek és világbajnoki címeinek számát. Ez az elem szintén adminisztrátor által vezérelt tartalom.

```
<div class="tracks-grid">
  @* A főoldalra kiemelt versenyző megjelenítése.
  Admin számára kattintható, más felhasználóknál csak információs kártya *@
  <article class="track-card @(isLoggedIn && userRole == "Admin" ? "" : "disabled-card")"
    @onclick="NavigateToAdmin"
    style="cursor:@(isLoggedIn && userRole == "Admin" ? "pointer" : "not-allowed");">
    <div class="track-content">
      <div class="track-country">@(selectedDriver?.nationality ?? "Nincs kiválasztva")</div>
      <h3 class="track-name">@(selectedDriver?.driver_name ?? "Nincs kiválasztott versenyző")</h3>
      <p>@(selectedDriver?.biography ?? "Az admin még nem választott ki versenyzőt a főoldalra.")</p>
      <div class="track-stats">
        <div class="stat">
          <div class="stat-value">@(selectedDriver?.wins.ToString() ?? "-")</div>
          <div class="stat-label">Győzelem</div>
        </div>
        <div class="stat">
          <div class="stat-value">@(selectedDriver?.podiums.ToString() ?? "-")</div>
          <div class="stat-label">Dobogó</div>
        </div>
        <div class="stat">
          <div class="stat-value">@(selectedDriver?.championships.ToString() ?? "-")</div>
          <div class="stat-label">VB cím</div>
        </div>
      </div>
    </div>
  </article>
</div>
```

Az oldal alsó részén helyezkedik el a **lábléc**, amely egyszerű információt tartalmaz az alkalmazásról. Ez zárja le vizuálisan az oldalt, és egységes megjelenést biztosít.

```
<footer>
  <p>© 2026 F1 Racing Zone.</p>
</footer>
```

2.14.2 MŰKÖDÉS:

Az oldal betöltésekor a komponens két fő adatlekérdezést hajt végre. Az első lekérdezés az adminisztrátor által korábban kiválasztott főoldali elemeket tölti be, vagyis a kiemelt pályát és a kiemelt versenyzőt. A második lekérdezés a következő futam adatait szerzi be, amely alapján a rendszer egy visszaszámlálót jelenít meg. A felhasználó bejelentkezési állapotának ellenőrzése a böngésző helyi tárolójából történik. Amennyiben a felhasználó hitelesített, az oldal megjeleníti a nevét, valamint a szerepkörétől függően további adminisztrációs lehetőségeket is biztosít. Adminisztrátor esetén a főoldalon található kiemelt kártyák kattinthatóvá válnak, és átirányítanak az adminisztrációs felületre, ahol ezek a tartalmak módosíthatók. A következő futam megjelenítéséhez az alkalmazás időzítőt használ, amely másodpercenként frissíti a hátralévő időt. Amikor a futam kezdési időpontja elérkezik, a visszaszámláló nullára áll és az időzítő leáll.

Használt változók és adatszerkezetek:

- **isLoggedIn**: a felhasználó bejelentkezési állapotát tárolja
- **userName**: a bejelentkezett felhasználó neve
- **userRole**: a felhasználó szerepköre
- **selectedCircuit**: a főoldalon megjelenített kiválasztott pálya adatai
- **selectedDriver**: a főoldalon megjelenített kiválasztott versenyző adatai
- **nextRace**: a soron következő futam adatai
- **timeLeft**: a visszaszámlálóhoz szükséges fennmaradó idő
- **countdownTimer**: az idő frissítését végző időzítő

A főoldal által használt API végpontok

- **GET /api/homepagesettings/1** – a főoldali kiemelések azonosítóinak lekérdezése
- **GET /api/circuits** – a pályák adatainak lekérdezése
- **GET /api/drivers/admin-list** – a versenyzők adatainak lekérdezése
- **GET /api/raceschedules/next-race** – a következő futam adatainak lekérdezése

2.15 TRACKS.RAZOR:

A pályák oldal célja, hogy a felhasználó áttekinthető formában megismerhesse a Forma–1 versenynaptárában szereplő helyszíneket és néhány legendásnak mondható pályát is. A tartalom dinamikusan, API-híváson keresztül töltődik be az adatbázisból.

Az oldal a többi felülethez hasonlóan figyelembe veszi a felhasználó bejelentkezési állapotát is. Ennek megfelelően a navigációs sávban eltérő elemek jelennek meg bejelentkezett, illetve adminisztrátor felhasználók számára.

2.15.1 A PÁLYÁK OLDAL FELÉPÍTÉSE:

A pályák oldal több jól elkülöníthető szerkezeti egységből áll. Az oldal tetején egy **navigációs sáv** található, amely biztosítja az alkalmazás különböző részei közötti átjárást. A menü rezponzív kialakítású, ezért asztali és mobil nézetben is megfelelően használható. A navigációs sávban megjelenik a felhasználó neve és a kijelentkezési lehetőség, amennyiben a felhasználó be van jelentkezve. Adminisztrátori jogosultság esetén további adminisztrációs menüpont is látható.

```

<nav class="navbar">
  <div class="nav-container">
    <div class="nav-brand">
      <a href="/" class="logo">F1<span>ZONE</span></a>
    </div>

    <ul class="nav-links @(isMenuOpen ? "active" : "")">
      <li><a href="/" @onclick="CloseMenu">Kezdőlap</a></li>
      <li><a href="/tracks" @onclick="CloseMenu">Pályák</a></li>
      <li><a href="/drivers" @onclick="CloseMenu">Versenyzők</a></li>
      <li><a href="/schedule" @onclick="CloseMenu">Versenynaptár</a></li>
      <li><a href="/news" @onclick="CloseMenu">Hírek</a></li>
      <li><a href="/regulation_changes" @onclick="CloseMenu">Szabályok</a></li>
      <li><a href="/strategy">Stratégia</a></li>

      @* Admin felhasználók számára külön menüpont megjelenítése *@
      <if (isLoggedIn && userRole == "Admin")>
        {
          <li><a href="/admin" @onclick="CloseMenu">Adatok szerkesztése</a></li>
        }
      </if>

      @* Mobilnézetben a felhasználói műveletek külön blokkban jelennek meg *@
      <li class="mobile-only">
        <if (isLoggedIn)>
          {
            <span class="user-welcome">Üdv, <strong>@userName</strong>!</span>
            <button class="login-btn" @onclick="Logout">KIJELENTEZÉS</button>
          }
        <else>
          {
            <button class="login-btn" @onclick="GoToLogin">BEJELENTEZÉS</button>
          }
        </if>
      </li>
    </ul>

    <div class="nav-actions">
      <if (isLoggedIn)>
        {
          <if (userRole == "Admin")>
            {
              <button class="login-btn desktop-only" @onclick="() => NavManager.NavigateTo("/admin/home")">ADMIN</button>
            }
            <span class="user-welcome">Üdv, <strong>@userName</strong>!</span>
            <button class="login-btn desktop-only" @onclick="Logout">KIJELENTEZÉS</button>
          </if>
          <else>
            {
              <button class="login-btn desktop-only" @onclick="GoToLogin">BEJELENTEZÉS</button>
            }
          </else>

          @* Hamburger menü mobil eszközökhöz *@
          <button class="burger-btn" @onclick="ToggleMenu">
            <if (isMenuOpen)>
              {
                <span>&times;</span>
              }
            <else>
              {
                <span>&#9776;</span>
              }
            </if>
          </button>
        }
      </if>
    </div>
  </div>

```

A navigációs sáv alatt helyezkedik el az oldal **fejlécrésze**, amely címet és rövid leírást tartalmaz. Ez a blokk tájékoztatja a felhasználót arról, hogy az oldalon a versenynaptár pályái böngészhetők végig.

```

<div class="section-header" style="margin-top: 4rem; text-align: center;">
  <h2>Versenynaptár Pályái</h2>
  <p>Böngéssz végig a Forma-1 szezon összes legendás helyszínét!</p>
</div>

```

A fő tartalmi részben a rendszer a pályákat **kártyás elrendezésben** jeleníti meg. Minden kártya tartalmazza az adott pálya képét, országát, nevét, rövid leírást, valamint két fontos technikai adatát: a pálya hosszát és a kanyarok számát. Ez a megoldás egyszerre esztétikus és informatív megjelenítést biztosít.

Betöltés közben az oldal egy **visszajelző üzenetet** jelenít meg, amely tájékoztatja a felhasználót arról, hogy a pályaadatok lekérése folyamatban van.

```

if (tracks == null)
{
  <p style="text-align: center;"><em>Pályák betöltése a boxutóból...</em></p>
}
else
{
  #* A pályák kártyás megjelenítése *#
  <div class="tracks-grid">
    @foreach (var track in tracks)
    {
      <article class="track-card">
        <div class="track-image">
          
        </div>
        <div class="track-content">
          <div class="track-country">@track.Country</div>
          <h3 class="track-name">@track.Circuit_name</h3>
          <p>@track.Circuit_description</p>
          <div class="track-stats">
            <div class="stat"><div class="stat-value">@track.Length_km km</div><div class="stat-label">Hossz</div></div>
            <div class="stat"><div class="stat-value">@track.Turns</div><div class="stat-label">Kanyar</div></div>
          </div>
        </div>
      </article>
    }
  </div>
}

```

2.15.2 MŰKÖDÉS:

Az oldal inicializálásakor a komponens API-hívást indít, amelynek célja a pályák adatainak lekérése. Az így kapott adatok egy listába kerülnek, majd ezek alapján dinamikusan felépül a kártyanézet. Amennyiben az adatok még nem érkeztek meg, az oldal ideiglenesen egy betöltési üzenetet jelenít meg.

A felhasználó bejelentkezési állapotának ellenőrzése az első renderelés után történik a böngésző helyi tárolójában található adatok alapján. Ha a felhasználó hitelesített, akkor az oldal betölti a felhasználó nevét és szerepkörét is. Ez alapján dől el, hogy a navigációs sávban milyen felhasználói elemek jelenjenek meg.

A mobilbarát működés érdekében az oldal hamburger menüt is tartalmaz. A menü állapota egy logikai változó segítségével változik, így a navigációs elemek kis képernyőn is megfelelően kezelhetők.

A kijelentkezési folyamat során az alkalmazás törli a helyi tárolóban eltárolt adatokat, majd visszairányítja a felhasználót a főoldalra.

2.15.3 FONTOSABB METÓDUSOK

A pályák oldal működésében több metódus is fontos szerepet játszik:

- **OnInitializedAsync()** – az oldal indulásakor lekéri a pályák adatait az API-ból
- **OnAfterRenderAsync(bool firstRender)** – az első renderelés után ellenőrzi a felhasználó bejelentkezési állapotát
- **ToggleMenu()** – megnyitja vagy bezárja a mobil navigációs menüt
- **CloseMenu()** – bezárja a menüt navigáció után
- **GoToLogin()** – a bejelentkezési oldalra navigál
- **Logout()** – törli a helyi tárolt adatokat és kijelentkezteti a felhasználót

2.15.4 FELHASZNÁLT ADATMODELL

Az oldal a pályák megjelenítéséhez egy Track nevű modellt használ. Ez tartalmazza a pálya azonosítóját, nevét, helyszínét, országát, hosszát, kanyarjainak számát, képeinek elérési útját, valamint rövid leírását. Ezek az adatok együtt elegendőek a pályakártyák teljes megjelenítéséhez.

Használt API végpont:

- **GET** /api/Circuits

2.16 DRIVERS.RAZOR:

A versenyzők oldal célja, hogy a felhasználó áttekinthető és strukturált formában böngészhesse a Forma–1 aktuális pilótáit. Az oldal nem csupán megjeleníti az adatokat, hanem lehetőséget biztosít a felhasználók számára kedvenc versenyzők megjelölésére is, amely egy személyre szabott élményt eredményez, amennyiben be van jelentkezve. Az oldal dinamikusan kommunikál a backend API-val, és a felhasználó állapotától függően eltérő funkcionalitást biztosít.

2.16.1 A VERSENYZŐK OLDAL FELÉPÍTÉSE:

Az oldal tetején egy **navigációs sáv** található, amely megegyezik az alkalmazás többi részén használt megoldással. A menü dinamikusan változik a felhasználó bejelentkezési állapota és szerepköre alapján. Bejelentkezett felhasználók esetén megjelenik a felhasználónév és a kijelentkezési lehetőség, adminisztrátorok számára pedig további kezelőfelület is elérhető.

```
<nav class="navbar">
  <div class="nav-container">
    <div class="nav-brand">
      <a href="/" class="logo"><span>ZONE</span></a>
    </div>

    <ul class="nav-links" @(isMenuOpen ? "active" : "")>
      <li><a href="/" @onclick="CloseMenu">Kezdőlap</a></li>
      <li><a href="/tracks" @onclick="CloseMenu">Pályák</a></li>
      <li><a href="/drivers" @onclick="CloseMenu">Versenyzők</a></li>
      <li><a href="/schedule" @onclick="CloseMenu">Versenynaptár</a></li>
      <li><a href="/news" @onclick="CloseMenu">Hírek</a></li>
      <li><a href="/regulation_changes" @onclick="CloseMenu">Szabályok</a></li>
      <li><a href="/strategy">Stratégia</a></li>

      @* Admin felhasználók számára extra menüpont jelenik meg *@
      @if (isLoggedIn && userRole == "Admin")
      {
        <li><a href="/admin" @onclick="CloseMenu">Adatok szerkesztése</a></li>
      }

      @* Mobilkészítményen külön jelennek meg a felhasználói műveletek *@
      <li class="mobile-only">
        @if (isLoggedIn)
        {
          <span class="user-welcome">Üdv, <strong>@userName</strong>!</span>
          <button class="login-btn" @onclick="Logout">KIJELENTKEZÉS</button>
        }
        else
        {
          <button class="login-btn" @onclick="GoToLogin">BEJELENTKEZÉS</button>
        }
      </li>
    </ul>

    <div class="nav-actions">
      @if (isLoggedIn)
      {
        @if (userRole == "Admin")
        {
          <button class="login-btn desktop-only" @onclick="()" => NavManager.NavigateTo("/admin/home")>ADMIN</button>
        }
        <span class="user-welcome">Üdv, <strong>@userName</strong>!</span>
        <button class="login-btn desktop-only" @onclick="Logout">KIJELENTKEZÉS</button>
      }
      else
      {
        <button class="login-btn desktop-only" @onclick="GoToLogin">BEJELENTKEZÉS</button>
      }

      @* Hamburger menü mobil eszközökhöz *@
      <button class="burger-btn" @onclick="ToggleMenu">
        @if (isMenuOpen)
        {
          <span>5times</span>
        }
        else
        {
          <span>50776</span>
        }
      </button>
    </div>
  </div>
</nav>
```


A navigációs sáv alatt helyezkedik el egy **hero szekció**, amely az oldal címét és rövid leírását tartalmazza. Ez a blokk segíti a felhasználót az oldal funkciójának gyors megértésében.

```
<section class="hero">
  <h1>Piloták</h1>
  <p>Kattints egy pilótára, hogy többet tudj meg róla</p>
</section>
```

A fő tartalmi részben a versenyzők **csapatok szerint csoportosítva** jelennek meg. Minden csapat egy külön szekciót alkot, amely vizuálisan is elkülönül. A csapat neve mellett egy színes jelölés is megjelenik, amely az adott istállóhoz tartozó színvilágot reprezentálja. A csapatokon belül a versenyzők **kártyás elrendezésben** jelennek meg. Egy kártya tartalmazza a pilóta nevét, csapatát, nemzetiségét, valamint rajtszámát vizuális kiemeléssel. A kártyák kattinthatók, és a kiválasztott versenyző részletes adatlapjára navigálnak.

```
<div class="container">

  @* Amíg a pilóták nem töltődnek be, betöltési szöveget jelenítünk meg *@
  @if (groupedDrivers == null)
  {
    <p>Adatok betöltése...</p>
  }
  else
  {
    @* A pilótákat csapatok szerint csoportosítva jelenítjük meg *@
    @foreach (var group in groupedDrivers)
    {
      <div class="team-section">
        <div class="team-header">
          @* 1. A csapat függőleges csíkja *@
          <div class="team-color-bar" style="background: @(group.FirstOrDefault()?.teamcolor ?? "#3671C6");"></div>
          <h2>@group.Key</h2>
        </div>

        <div class="drivers-grid">
          @foreach (var driver in group)
          {
            <div class="driver-card-wrapper" style="position: relative;">

              @* Maga a kattintható kártya *@
              <a class="driver-card" href="/driver/@driver.driver_name.Replace(" ", "-").ToLower()" style="display: block;">
                <span class="driver-number-bg" style="color: @driver.teamcolor; opacity: 0.2;">
                  @(driver.racing_number ?? 0)
                </span>

                <div class="driver-avatar" style="background: @(HexToRgba(driver.teamcolor, 0.1)); border: 1px solid @driver.teamcolor; width: 40px; height: 40px; display: flex; align-items: center; justify-content: center; margin: 5px auto;">
                  @(driver.racing_number ?? 0)
                </div>

                <div class="driver-info">
                  <div class="driver-name">@driver.driver_name</div>
                  <div class="driver-team">@driver.teamname</div>
                  <div class="driver-nationality">@driver.nationality?.ToUpper()</div>
                </div>
              </a>
            </div>
          }
        </div>
      </div>
    }
  }
</div>
```

Bejelentkezett felhasználók esetén minden kártyán megjelenik egy **kedvencekhez adás ikon**, amely lehetővé teszi a pilóta mentését vagy eltávolítását a felhasználó kedvencei közül.


```

@* Bejelentkezett felhasználó esetén lehetőség van
   a pilóta kedvencekhez adására vagy eltávolítására *@
@if (isLoggedIn)
{
    <div class="favorite-container" style="position: absolute; top: 15px; right: 15px; z-index: 100;">
        <button @onclick="() => ToggleFavorite(driver.driver_id)"
            style="background: none; border: none; cursor: pointer; padding: 5px; outline: none;">
            @if (favoriteDriverIds.Contains(driver.driver_id))
            {
                <span style="color: #ff1801; font-size: 1.4rem; filter: drop-shadow(0 0 5px #ff1801);">✖</span>
            }
            else
            {
                <span style="color: #ff1801; font-size: 1.4rem;">♥</span>
            }
        </button>
    </div>
}

```

Az oldal alján egy **lábléc** található, amely az alkalmazás alapinformációit jeleníti meg.

```

<footer>
    <p>© 2026 F1 Racing Zone</p>
</footer>

```

2.16.2 FONTOSABB METÓDUSOK:

- **OnInitializedAsync()** – lekéri a versenyzők listáját és csoportosítja őket csapat szerint
- **OnAfterRenderAsync(bool firstRender)** – ellenőrzi a felhasználó állapotát és betölti a kedvenceket
- **ToggleFavorite(int driverId)** – hozzáadja vagy eltávolítja a versenyzőt a kedvencek közül
- **LoadFavorites()** – lekéri a felhasználó kedvenc versenyzőit az API-ból
- **ToggleMenu()** – mobil menü kezelése
- **CloseMenu()** – menü bezárása
- **GoToLogin()** – navigáció a bejelentkezési oldalra
- **Logout()** – kijelentkezteti a felhasználót

2.16.3 FELHASZNÁLT ADATMODELLEK:

Az oldal a versenyzők megjelenítéséhez egy `DriverDto` modellt használ, amely tartalmazza a pilóta nevét, csapatát, nemzetiségét, rajtszámát és egyéb statisztikai adatait.

A kedvencek kezeléséhez egy `UserFavorites` modell kerül felhasználásra, amely összekapcsolja a felhasználót és a kiválasztott elemet (jelen esetben versenyzőt).

Használt API végpontok:

- **GET /api/drivers/admin-list** – versenyzők adatainak lekérése
- **GET /api/favorites** – felhasználó kedvenceinek lekérése
- **POST /api/favorites** – új kedvenc hozzáadása
- **DELETE /api/favorites/{userId}/{driverId}** – kedvenc eltávolítása

2.17 DRIVERDETAIL.RAZOR

A versenyző statisztika oldal célja, hogy a felhasználó egy kiválasztott pilótáról részletes információkat kapjon. Míg a versenyzők listája csak összefoglaló megjelenítést biztosít, ez az oldal már mélyebb betekintést nyújt az adott versenyző életrajzába, szakmai adataiba és statisztikáiba.

Az oldal dinamikusan, az URL-ben kapott azonosító alapján tölti be a megfelelő pilóta adatait az API-ból. Ennek köszönhetően ugyanaz a komponens több különböző versenyző részletes adatlapjának megjelenítésére is alkalmas.

2.17.1 A VERSENYZŐ RÉSZLETEZŐ OLDAL FELÉPÍTÉSE:

Az oldal tetején egy **navigációs sáv** található, amely a webalkalmazás többi oldalához hasonlóan biztosítja a főbb funkciók elérését. A navigációs elemek a felhasználó bejelentkezési állapotától és jogosultsági szintjétől függően változnak. Bejelentkezett felhasználók esetén megjelenik a felhasználónév és a kijelentkezési lehetőség, adminisztrátori szerepkör esetén pedig külön adminisztrációs opció is látható.


```

<div class="container">
  <div class="detail-section">
    <h2>Életrajz</h2>
    <div class="bio-card">
      @if (driver == null)
      {
        <div class="loading-box">
          <p>Adatok betöltése a boxutcából...</p>
        </div>
      }
      else
      {
        <p>@driver.biography</p>
      }
    </div>
  </div>

```

Az életrajz alatt található egy **technikai adatok blokk**, amely a versenyzőhöz és jelenlegi csapatához kapcsolódó kiegészítő információkat jelenít meg. Itt szerepel például a becsült fizetés, az aktuális csapat neve, a főbb szponzorok listája, valamint a motorpartner. Ez a rész kibővíti a pusztá sportstatisztikát, és üzleti-technikai oldalról is bemutatja a versenyző környezetét.

```

@if (driver != null)
{
  <div class="technical-details">
    <div class="tech-card">
      <div class="tech-icon">🏎️</div>
      <div class="tech-info">
        <span class="tech-label">Becsült fizetés</span>
        <span class="tech-value">@driver.salary_estimate?.ToString("N0") $</span>
      </div>
    </div>

    <div class="tech-card">
      <div class="tech-icon">🏠</div>
      <div class="tech-info">
        <span class="tech-label">Aktuális csapat</span>
        <span class="tech-value">@(driver?.teamname ?? "Nincs adat")</span>
      </div>
    </div>

    <div class="tech-card">
      <div class="tech-icon">👥</div>
      <div class="tech-info">
        <span class="tech-label">Csapat fő szponzorai</span>
        <span class="tech-value">
          @if (driver.team_sponsors != null && driver.team_sponsors.Any())
          {
            @string.Join(", ", driver.team_sponsors)
          }
          else
          {
            <span>Nincs adat</span>
          }
        </span>
      </div>
    </div>

    <div class="tech-card">
      <div class="tech-icon">🏎️</div>
      <div class="tech-info">
        <span class="tech-label">Motorpartner</span>
        <span class="tech-value">@(driver?.manufacturer ?? "Nincs adat")</span>
      </div>
    </div>
  </div>
}

```

A következő egység a **karrier- és szezonstatisztikák megjelenítése**, ahol a legfontosabb mutatók kártyás formában jelennek meg. Ebben a blokkban látható a győzelmek, dobogós helyezések, leggyorsabb körök, világbajnoki címek és pontok száma. A vizuális kialakítás célja, hogy a felhasználó gyorsan áttekinthető formában lássa a pilóta teljesítményének legfontosabb elemeit.

```

<h2>Karrier / Szezon statisztika</h2>
@if (driver != null)
{
    @* A pilóta legfontosabb statisztikáinak kártyás megjelenítése *@
    <div class="stats-row">
        <div class="stat-card">
            <div class="stat-value">@driver.wins > 0 ? "gold" : "">@driver.wins</div>
            <div class="stat-label">Győzelmek</div>
        </div>
        <div class="stat-card">
            <div class="stat-value">@driver.podiums</div>
            <div class="stat-label">Pódiumok</div>
        </div>
        <div class="stat-card">
            <div class="stat-value">@driver.fastest_laps</div>
            <div class="stat-label">Leggyorsabb kör</div>
        </div>
        <div class="stat-card">
            <div class="stat-value">@driver.championships > 0 ? "gold" : "">@driver.championships</div>
            <div class="stat-label">VB címek</div>
        </div>
        <div class="stat-card">
            <div class="stat-value">@driver.points</div>
            <div class="stat-label">Pontok</div>
        </div>
    </div>
}
else
{
    <p class="loading-text">Statisztikák betöltése...</p>
}

@* Navigáció vissza a versenyzők listájához *@
<div class="actions">
    <button class="back-btn" @onclick="BackToDrivers">← Vissza a többi pilótához!</button>
</div>
</div>

```

Az oldal alsó részén egy **visszanavigáló gomb** található, amely a felhasználót visszavezeti a versenyzők listájához. Ezt egészíti ki a **lábléc**, amely az oldal lezárásaként jelenik meg.

```

@* Navigáció vissza a versenyzők listájához *@
<div class="actions">
    <button class="back-btn" @onclick="BackToDrivers">← Vissza a többi pilótához!</button>
</div>
</div>

```

2.17.2 A VERSENYZŐ STATISZTIKA OLDAL MŰKÖDÉSE:

Az oldal működése az URL-ben kapott paraméterre épül. A komponens a megnyitáskor kiolvassa a versenyző azonosítóját az útvonalból, majd ennek alapján API-hívással lekéri a megfelelő pilóta részletes adatait. Ez biztosítja, hogy minden versenyzőhöz ugyanaz a részletező felület használható legyen.

Az oldal inicializálásakor nemcsak a pilóta adatainak lekérése történik meg, hanem a rendszer a felhasználó hitelesítési állapotát is ellenőrzi. A bejelentkezési állapot és a szerepkör meghatározása egyrészt az autentikációs állapotból, másrészt szükség esetén a helyi tárolóból történik. Ennek célja, hogy a felület megfelelően tudja kezelni az adminisztrációs opciókat és a felhasználói navigációt.

Az első renderelés után az oldal külön ellenőrzi, hogy a bejelentkezett felhasználó kedvencei között szerepel-e az aktuális versenyző. Ehhez a rendszer lekéri a felhasználó azonosítóját a helyi tárolóból, majd összeveti azt a kedvencek API-ból lekért adataival. Ez a logika előkészíti a személyre szabott működést, még akkor is, ha a kedvenc állapot megjelenítése a felületen csak részben van kihasználva.

Amennyiben az URL paraméter megváltozik, a komponens újra lekéri a megfelelő pilóta adatait. Ez azért fontos, mert ugyanaz a részletező oldal újratöltés nélkül is képes másik versenyző megjelenítésére.

A kijelentkezési művelet során az alkalmazás törli a helyi tárolóból a felhasználói adatokat, majd visszairányítja a felhasználót a kezdőlapra.

2.17.3 FONTOSABB METÓDUSOK:

- **OnInitializedAsync()** – betölti a kiválasztott pilóta adatait, valamint ellenőrzi a felhasználó hitelesítési állapotát
- **OnAfterRenderAsync(bool firstRender)** – az első renderelés után lekéri a felhasználó azonosítóját és ellenőrzi a kedvenc állapotot
- **OnParametersSetAsync()** – URL paraméterváltozás esetén újratölti a pilóta adatait
- **LoadDriver()** – lekéri az aktuális versenyző részletes adatait az API-ból
- **ToggleMenu()** – mobilmenü állapotát váltja
- **CloseMenu()** – bezárja a mobilmenüt
- **GoToLogin()** – a bejelentkezési oldalra navigál
- **Logout()** – kijelentkezteti a felhasználót
- **BackToDrivers()** – visszavigál a versenyzők listájához

2.17.4 FELHASZNÁLT ADATMODELLEK:

A versenyző részletező oldal központi adatmodellje a **DriverDto**, amely a kiválasztott pilóta részletes adatainak tárolására szolgál. Ez a modell tartalmazza a versenyző nevét, életrajzát, győzelmeit, dobogós helyezéseit, leggyorsabb köreit, világbajnoki címeit és pontjait. A részletező oldalon ezen kívül további mezők is szerepelnek, például a becsült fizetés, az aktuális csapat neve, a főbb szponzorok, valamint a motorpartner. Ez a modell biztosítja az oldal tartalmi magját.

A kedvencek ellenőrzéséhez a rendszer a **UserFavorites** modellt használja, amely a felhasználók és a kedvencként megjelölt elemek közötti kapcsolatot tárolja. Ebben az esetben ez annak meghatározására szolgál, hogy az aktuális versenyző szerepel-e a felhasználó kedvencei között.

Az oldal működéséhez közvetetten a felhasználói autentikációs állapot is kapcsolódik, amely a helyi tároló és az autentikációs szolgáltatás segítségével kerül kezelésre.

Használt API végpontok:

- **GET /api/drivers/{id}** – a kiválasztott versenyző részletes adatainak lekérése
- **GET /api/favorites** – a felhasználó kedvenceinek lekérése

2.18 SCHEDULE.RAZOR

A versenynaptár oldal célja, hogy a felhasználó egy helyen áttekinthesse a teljes Forma–1 szezon futamait. Az oldal időrendben jeleníti meg a versenyeket, és egyértelműen megkülönbözteti a már lezajlott és a még előttünk álló eseményeket.

A felhasználó nemcsak megtekintheti az adatokat, hanem a jövőbeli futamok esetében további részletekhez is navigálhat.

2.18.1 A VERSENYNAPTÁR OLDAL FELÉPÍTÉSE:

Az oldal tetején egy **navigációs sáv** található, amely biztosítja az alkalmazás többi részéhez való hozzáférést. A navigáció dinamikusan alkalmazkodik a felhasználó bejelentkezési állapotához és szerepköréhez.


```

<nav class="navbar">
  <div class="nav-container">
    <div class="nav-brand">
      <a href="/" class="Logo">F1</span>ZONE</span></a>
    </div>

    <ul class="nav-links @(isMenuOpen ? "active" : "")">
      <li><a href="/" @onclick="CloseMenu">Kezdőlap</a></li>
      <li><a href="/tracks" @onclick="CloseMenu">Pályák</a></li>
      <li><a href="/drivers" @onclick="CloseMenu">Versenyzők</a></li>
      <li><a href="/schedule" @onclick="CloseMenu">Versenynaptár</a></li>
      <li><a href="/news" @onclick="CloseMenu">Hírek</a></li>
      <li><a href="/regulation_changes" @onclick="CloseMenu">Szabályok</a></li>
      <li><a href="/strategy">Stratégia</a></li>

      @* Admin felhasználók számára külön menüpont megjelenítése *@
      @if (isLoggedIn && userRole == "Admin")
      {
        <li><a href="/admin" @onclick="CloseMenu">Adatok szerkesztése</a></li>
      }

      @* Mobilnőzetben a felhasználói műveletek külön blokkban jelennek meg *@
      <li class="mobile-only">
        @if (isLoggedIn)
        {
          <span class="user-welcome">Üdv, <strong>@userName</strong>!</span>
          <button class="login-btn" @onclick="Logout">KIJELENTÉZÉS</button>
        }
        else
        {
          <button class="login-btn" @onclick="GoToLogin">BEJELENTÉZÉS</button>
        }
      </li>
    </ul>

    <div class="nav-actions">
      @if (isLoggedIn)
      {
        @if (userRole == "Admin")
        {
          <button class="login-btn desktop-only" @onclick="() => NavManager.NavigateTo("/admin/home")">ADMIN</button>
        }
        <span class="user-welcome">Üdv, <strong>@userName</strong>!</span>
        <button class="login-btn desktop-only" @onclick="Logout">KIJELENTÉZÉS</button>
      }
      else
      {
        <button class="login-btn desktop-only" @onclick="GoToLogin">BEJELENTÉZÉS</button>
      }

      @* Hamburger menü mobil eszközökhöz *@
      <button class="burger-btn" @onclick="ToggleMenu">
        @if (isMenuOpen)
        {
          <span>✖</span>
        }
        else
        {
          <span>☰</span>
        }
      </button>
    </div>
  </div>
</nav>

```

fő tartalmi rész egy **versenynaptár lista**, amely kártyás formában jeleníti meg az egyes futamokat.

Minden futam egy külön **race-card** elemként jelenik meg, amely az alábbi részekből áll:

- futam sorszáma (ROUND jelölés)
- ország zászlaja
- verseny dátuma
- futam neve
- pálya neve
- státusz vagy műveleti gomb

A kártyák vizuálisan is elkülönülnek attól függően, hogy a futam már lezajlott-e. A múltbeli futamok külön stílust kapnak, míg a jövőbeli eseményekhez aktív interakció tartozik.

```
<div class="schedule-container">
  @* A teljes versenynaptár listázása kártyás formában *@
  @foreach (var race in races)
  {
    <div class="race-card @(race.RaceDate < DateTime.Now ? "past-race" : "")">
      <div class="round-number">ROUND @(races.IndexOf(race) + 1)</div>
      <div class="card-left">
        
        <div class="race-info">
          <span class="race-date">@race.RaceDate.ToString("MM dd").ToUpper()</span>
          <h3 class="race-name">@race.RaceName</h3>
          <span class="track-name">@race.TrackName</span>
        </div>
      </div>
      <div class="card-right">
        @if (race.RaceDate < DateTime.Now)
        {
          <span class="status-badge finished">LEZAJLOTT</span>
        }
        else
        {
          <button class="details-btn" onclick="() => NavigateToDetails(race.Id)">Részletek <i class="fl-icon-arrow"></i></button>
        }
      </div>
    </div>
  }
</div>
@code {
```

2.18.2 A VERSENYNAPTÁR OLDAL MŰKÖDÉSE:

Az oldal betöltésekor a rendszer automatikusan lekéri a teljes versenynaptárat az API-ból. Az adatok egy listában kerülnek tárolásra, majd egy ciklus segítségével jelennek meg a felhasználói felületen.

A rendszer minden futam esetében összehasonlítja a futam dátumát az aktuális idővel. Ennek alapján kétféle állapot különböztethető meg:

- **lezajlott futam** – a rendszer egy státusz jelzést jelenít meg („LEZAJLOTT”), és a kártya inaktívvá válik
- **közelgő futam** – a rendszer egy „Részletek” gombot jelenít meg, amely a futam részletes oldalára navigál

Ez a logika biztosítja, hogy a felhasználó mindig releváns interakciót kapjon, és ne tudjon értelmetlen műveleteket végrehajtani múltbeli eseményeken.

Az oldal a renderelés után külön ellenőrzi a felhasználó bejelentkezési állapotát a helyi tároló alapján. Ennek eredménye határozza meg, hogy milyen navigációs elemek jelennek meg.

A „Részletek” gomb egy dinamikus útvonalra navigál, amely tartalmazza a kiválasztott futam azonosítóját, így lehetővé teszi a részletes adatok betöltését egy másik oldalon.

2.18.3 FONTOSABB METÓDUSOK:

- **OnInitializedAsync()** – lekéri a versenynaptár adatait az API-ból
- **OnAfterRenderAsync(bool firstRender)** – kezeli a bejelentkezési állapot kiolvasását
- **NavigateToDetails(int raceId)** – a kiválasztott futam részletező oldalára navigál
- **ToggleMenu()** – mobilmenü állapotának váltása
- **CloseMenu()** – mobilmenü bezárása
- **GoToLogin()** – navigáció a bejelentkezési oldalra
- **Logout()** – kijelentkezteti a felhasználót

2.18.4 FELHASZNÁLT ADATMODELLEK

A versenynaptár oldal működésének alapját a `RaceDto` adatmodell biztosítja. Ez a modell tartalmazza a futamok megjelenítéséhez szükséges legfontosabb adatokat.

A modell mezői:

- **Id** – a futam egyedi azonosítója
- **RaceName** – a futam neve
- **TrackName** – a verseny helyszínéül szolgáló pálya neve
- **RaceDate** – a futam időpontja
- **CountryCode** – az országkód, amely a zászló megjelenítéséhez szükséges

Ez az adatmodell kifejezetten a listaoldal számára lett optimalizálva, vagyis csak azokat az adatokat tartalmazza, amelyek a gyors és áttekinthető megjelenítéshez szükségesek.

Használt API végpontok:

- **GET /api/raceschedules** – a teljes versenynaptár lekérése

2.19 NEWS.RAZOR

A hírek oldal célja, hogy a felhasználó naprakész Forma–1 híreket olvashasson az alkalmazáson belül. Az oldal külső hírszolgáltató API-ból tölti be a kapcsolódó cikkeket, majd ezeket kártyás formában jeleníti meg. Az oldal nem csupán egyszerű hírolvasó felületként működik, hanem a bejelentkezett felhasználók számára személyre szabott szűrést is biztosít a kedvenc versenyzőik alapján.

A rendszer így képes összekapcsolni a külső tartalmat a belső felhasználói preferenciákkal, ami jelentősen növeli a felhasználói élményt.

2.19.1 A HÍREK OLDAL FELÉPÍTÉSE:

Az oldal tetején egy **navigációs sáv** található, amely az alkalmazás többi oldalához hasonló módon biztosítja a főbb funkciók elérését. A navigációs elemek a felhasználó bejelentkezési állapota és szerepköre alapján dinamikusan változnak. Bejelentkezett felhasználók számára megjelenik a felhasználónév és a kijelentkezési lehetőség, adminisztrátorok számára pedig további adminisztrációs opció is látható.

```

<nav class="navbar">
  <div class="nav-container">
    <div class="nav-brand">
      <a href="/" class="Logo">F1</span> ZONE</span></a>
    </div>

    <ul class="nav-links @if(isMenuOpen ? "active" : "")">
      <li><a href="/" @onclick="CloseMenu">Kezdőlap</a></li>
      <li><a href="/tracks" @onclick="CloseMenu">Pályák</a></li>
      <li><a href="/drivers" @onclick="CloseMenu">Versenyzők</a></li>
      <li><a href="/schedule" @onclick="CloseMenu">Versenynaptár</a></li>
      <li><a href="/news" @onclick="CloseMenu">Hírek</a></li>
      <li><a href="/regulation_changes" @onclick="CloseMenu">Szabályok</a></li>
      <li><a href="/strategy">Stratégia</a></li>

      @* Admin felhasználók számára külön menüpont megjelenítése *@
      @if (isLoggedIn && userRole == "Admin")
      {
        <li><a href="/admin" @onclick="CloseMenu">Adatok szerkesztése</a></li>
      }

      @* Mobilkészletben a felhasználói műveletek külön blokkban jelennek meg *@
      <li class="mobile-only">
        @if (isLoggedIn)
        {
          <span class="user-welcome">Üdv, <strong>@userName</strong>!</span>
          <button class="login-btn" @onclick="Logout">KIJELENTEZÉS</button>
        }
        else
        {
          <button class="login-btn" @onclick="GoToLogin">BEJELENTEZÉS</button>
        }
      </li>
    </ul>

    <div class="nav-actions">
      @if (isLoggedIn)
      {
        @if (userRole == "Admin")
        {
          <button class="login-btn desktop-only" @onclick="() => Nav.NavigateTo("/admin/home")">ADMIN</button>
        }
        <span class="user-welcome">Üdv, <strong>@userName</strong>!</span>
        <button class="login-btn desktop-only" @onclick="Logout">KIJELENTEZÉS</button>
      }
      else
      {
        <button class="login-btn desktop-only" @onclick="GoToLogin">BEJELENTEZÉS</button>
      }

      @* Hamburger menü mobil eszközökhöz *@
      <button class="burger-btn" @onclick="ToggleMenu">
        @if (isMenuOpen)
        {
          <span>6times</span>
        }
        else
        {
          <span>689776</span>
        }
      </button>
    </div>
  </div>
</div>

```

A navigációs sáv alatt helyezkedik el a **híroldal fejléce**, amely címet és rövid leírást tartalmaz. Ez a rész egyértelműen jelzi az oldal funkcióját, vagyis a legfrissebb Forma–1 hírek megjelenítését. A fejléc alatt jelenik meg a **kedvencek szerinti szűrőkapcsoló**, amely kizárólag akkor látható, ha a felhasználó be van jelentkezve, és rendelkezik legalább egy kedvenc versenyzővel. A gomb segítségével a felhasználó válthat a teljes hírlista és a kedvenceihez kapcsolódó hírek között.

```
<header class="news-header">
  <h1>Legfrissebb Hírek</h1>
  <p>Hivatalos F1 hírek magyar nyelven</p>

  @* Bejelentkezett felhasználó esetén, ha vannak kedvenc pilóták,
  megjelenítjük a kedvencek szerinti szűrő kapcsolót *@
  @if (isLoggedIn && favoriteDriverNames.Any())
  {
    <div class="filter-container mt-3">
      <button class="filter-toggle-btn @(showOnlyFavorites ? "active" : "")" @onclick="() => showOnlyFavorites = !showOnlyFavorites">
        <i class="fas fa-heart"></i> @(showOnlyFavorites ? "Minden hír mutatása" : "Csak a kedvenceim hírei")
      </button>
    </div>
  }
</header>
```

A fő tartalmi részben a rendszer többféle állapotot kezel. Betöltés közben loader szekció jelenik meg, hiba esetén hibaüzenet, üres találati lista esetén pedig üres állapotot jelző blokk látható. Amennyiben vannak megjeleníthető hírek, azok egy kártyás hírrácsban jelennek meg.

```
<div class="container">
  @* Hiba esetén visszajelző üzenet megjelenítése *@
  @if (!string.IsNullOrEmpty(errorMessage))
  {
    <div class="alert alert-danger">@errorMessage</div>
  }

  @* Betöltés közben tájékoztató szöveg jelenik meg *@
  @if (articles == null)
  {
    <div class="loader-container">
      <p>Hírek keresése a pályán...</p>
    </div>
  }

  @* Ha nincs megjeleníthető hír, külön üres állapotot mutatunk *@
  else if (DisplayedArticles.Count == 0)
  {
    <div class="empty-news">
      <p>@(showOnlyFavorites ? "Egyik kedvenc pilótáról sem találtunk friss hírt." : "A lista üres, nem találtunk híreket.")</p>
    </div>
  }
  else
  {
```

Minden egyes hírkártya tartalmazza a cikk képét, forrását, címét, rövid leírását és publikálási dátumát. Ha az adott hír kapcsolódik valamelyik kedvenc versenyzőhöz, akkor a rendszer vizuálisan kiemeli azt, és egy külön jelöléssel is ellátja.

```

{
    /** A szűrés utáni hírek listájának kártyás megjelenítése */
    <div class="news-grid">
        @foreach (var article in DisplayedArticles)
        {
            bool isFavArticle = favoriteDriverNames.Any(name =>
                (article.Title?.Contains(name, StringComparison.OrdinalIgnoreCase) ?? false) ||
                (article.Description?.Contains(name, StringComparison.OrdinalIgnoreCase) ?? false));

            <div class="news-card @(isFavArticle ? "favorite-highlight" : "")" @onclick="@(() => OpenArticle(article.Url))">
                @if (!string.IsNullOrEmpty(article.UrlToImage))
                {
                    <div class="card-image">
                        
                        @if (isFavArticle)
                        {
                            <span class="fav-badge">KEDVENC</span>
                        }
                    </div>
                }
                <div class="card-content">
                    <span class="source-tag">@article.Source?.Name</span>
                    <h3>@article.Title</h3>
                    <p>@article.Description</p>
                    <div class="card-footer">
                        <span>@article.PublishedAt.ToString("yyyy.MM.dd. HH:mm")</span>
                    </div>
                </div>
            </div>
        }
    </div>
}

```

2.19.2 A HÍREK OLDAL MŰKÖDÉSE:

Az oldal inicializálásakor a komponens külső API-hívást indít, amelynek célja a Forma-1 témájú hírek lekérése. A lekérdezés során a rendszer magyar nyelvű, publikálási idő szerint rendezett híreket kér le. A kapott válaszból csak a használható cikkek kerülnek eltárolásra, az érvénytelen vagy eltávolított elemek kiszűrésre kerülnek.

A felhasználó bejelentkezési állapotának ellenőrzése az első renderelés után történik meg a helyi tároló alapján. Bejelentkezett felhasználó esetén a rendszer lekéri a felhasználó azonosítóját, nevét és szerepkörét, majd ezen információk alapján további személyre szabott működést biztosít.

A személyre szabás alapja a felhasználó kedvenc versenyzőinek listája. Az alkalmazás először lekéri a kedvenceket az adatbázisból, majd ezekhez hozzárendeli a megfelelő versenyzőneveket. Ezt követően a hírek listája szűrhetővé válik a kedvenc pilóták nevei alapján.

A szűrés működése úgy valósul meg, hogy a rendszer megvizsgálja a hírek címét és leírását, és ellenőrzi, hogy azok tartalmazzák-e valamelyik kedvenc versenyző nevét vagy annak egy értelmezhető részletét. Ez a megoldás rugalmasabb találati rendszert biztosít, mint a teljes név szerinti, szigorú egyezés.

Amennyiben a felhasználó bekapcsolja a szűrőt, az oldal csak azokat a cikkeket jeleníti meg, amelyek a kedvenc versenyzőkhöz köthetők. Ha a szűrő ki van kapcsolva, az összes elérhető hír látható marad.

A hírkártyák kattinthatók, így a felhasználó a kiválasztott cikk eredeti forrására navigálhat.

2.19.3 FONTOSABB METÓDUSOK:

- **OnInitializedAsync()** – az oldal indulásakor lekéri a híreket a külső API-ból
- **FetchNews()** – végrehajtja a hírek lekérését és eltárolását
- **OnAfterRenderAsync(bool firstRender)** – betölti a felhasználói állapotot és a kedvenc versenyzők neveit
- **LoadFavoriteNames()** – lekéri a felhasználó kedvenc pilótáit és azok neveit
- **OpenArticle(string url)** – megnyitja a kiválasztott hírcikket
- **ToggleMenu()** – mobil menü állapotának kezelése
- **CloseMenu()** – bezárja a mobil menüt
- **GoToLogin()** – a bejelentkezési oldalra navigál
- **Logout()** – kijelentkezteti a felhasználót

2.19.4 FELHASZNÁLT ADATMODELLEK:

A hírek oldal működéséhez több adatmodell kapcsolódik, mivel az oldal külső API-válaszokat és belső felhasználói adatokat egyaránt kezel.

A külső hírszolgáltató válaszát a `NewsResponse` modell reprezentálja. Ez tartalmazza a válasz állapotát, valamint a lekért hírek listáját. Ez a modell teszi lehetővé, hogy a rendszer strukturált formában dolgozza fel a kapott adatokat.

Az egyes hírcikkek adatait a `NewsArticle` modell tárolja. Ez tartalmazza a hír forrását, címét, leírását, URL-jét, a kép elérési útját, valamint a publikálási dátumot. Ez a modell alkotja a hírkártyák megjelenítésének alapját.

A hírforrás megjelenítéséhez a Source modell szolgál, amely jelen esetben a forrás nevét tárolja.

A felhasználó kedvenceinek összekapcsolásához a rendszer a UserFavorites modellt használja, amely megadja, hogy az adott felhasználó mely versenyzőket jelölte meg kedvencként. Ezzel párhuzamosan a Drivers modell segítségével a rendszer a kedvenc azonosítókából tényleges versenyzőneveket állít elő, amelyek a hírek szűréséhez szükségesek.

Használt API végpontok és külső szolgáltatások

- **GET <https://newsapi.org/v2/everything>** – Forma–1 témájú hírek lekérése külső hírszolgáltatótól
- **GET [/api/favorites](#)** – a felhasználó kedvenceinek lekérése
- **GET [/api/drivers](#)** – a versenyzők adatainak lekérése a kedvenc nevek feloldásához

2.20 REGULATIONCHANGES.RAZOR

A szabályváltozások oldal célja, hogy a felhasználó áttekinthető formában megismerhesse a Forma–1 történetében bekövetkezett jelentősebb technikai szabálmódosításokat. Az oldal idővonal jellegű megjelenítést alkalmaz, amely lehetővé teszi, hogy a változások kronológiai sorrendben, könnyen követhető módon jelenjenek meg.

A felület informatív szerepet tölt be, mivel a sportág technikai fejlődésének fontos állomásait gyűjti össze és mutatja be.

2.20.1 A SZABÁLYVÁLTOZÁSOK OLDAL FELÉPÍTÉSE:

Az oldal felső részén egy **navigációs sáv** található, amely az alkalmazás többi részéhez hasonlóan biztosítja a főbb oldalak elérését. A navigációs elemek itt is a felhasználó bejelentkezési állapotához és szerepköréhez igazodnak. Bejelentkezett felhasználók esetén megjelenik a felhasználónév és a kijelentkezési lehetőség, adminisztrátorok esetén pedig további kezelőfelületi opciók is elérhetők.

```

<nav class="navbar">
  <div class="nav-container">
    <div class="nav-brand">
      <a href="/" class="logo">F1<span>ZONE</span></a>
    </div>
    <ul class="nav-links @(isMenuOpen ? "active" : "")">
      <li><a href="/" @onclick="CloseMenu">Kezdőlap</a></li>
      <li><a href="/tracks" @onclick="CloseMenu">Pályák</a></li>
      <li><a href="/drivers" @onclick="CloseMenu">Versenyzők</a></li>
      <li><a href="/schedule" @onclick="CloseMenu">Versenynaptár</a></li>
      <li><a href="/news" @onclick="CloseMenu">Hírek</a></li>
      <li><a href="/regulation_changes" @onclick="CloseMenu">Szabályok</a></li>
      <li><a href="/strategy">Stratégia</a></li>

      @* Admin felhasználók számára külön menüpont megjelenítése *@
      @if (isLoggedIn && userRole == "Admin")
      {
        <li><a href="/admin" @onclick="CloseMenu">Adatok szerkesztése</a></li>
      }

      @* Mobilnézetben a felhasználói műveletek külön blokkban jelennek meg *@
      <li class="mobile-only">
        @if (isLoggedIn)
        {
          <span class="user-welcome">Üdv, <strong>@userName</strong>!</span>
          <button class="login-btn" @onclick="Logout">KIJELENTKEZÉS</button>
        }
        else
        {
          <button class="login-btn" @onclick="GoToLogin">BEJELENTKEZÉS</button>
        }
      </li>
    </ul>
    <div class="nav-actions">
      @if (isLoggedIn)
      {
        @if (userRole == "Admin")
        {
          <button class="login-btn desktop-only" @onclick="() => NavManager.NavigateTo("/admin/home")">ADMIN</button>
        }
        <span class="user-welcome">Üdv, <strong>@userName</strong>!</span>
        <button class="login-btn desktop-only" @onclick="Logout">KIJELENTKEZÉS</button>
      }
      else
      {
        <button class="login-btn desktop-only" @onclick="GoToLogin">BEJELENTKEZÉS</button>
      }
    }

    @* Hasburger menü mobil eszközökhöz *@
    <button class="burger-btn" @onclick="ToggleMenu">
      @if (isMenuOpen)
      {
        <span>×</span>
      }
      else
      {
        <span>☰</span>
      }
    </button>
  </div>
</nav>

```

A navigációs sáv alatt egy **fejlécblokk** helyezkedik el, amely az oldal címét és rövid leírását tartalmazza. Ez a rész tájékoztatja a felhasználót arról, hogy az oldalon a Forma–1 technikai szabályváltozásai böngészhetők végig.

```
<div class="timeline-container">
  <div class="section-header">
    <h2>Technikai Szabályváltozások</h2>
    <p>Az oldalon az első hivatalos szezonról a mai napig a nagyobb, jelentősegteljesebb szabályváltozásokat gyűjtöttük össze ide.</p>
  </div>
```

A fő tartalmi rész egy **idővonalas megjelenítést** alkalmaz. Ebben minden szabályváltozás külön elemként jelenik meg, amely két fő részből áll: az adott év megjelenítéséből, illetve a hozzá tartozó technikai szabályleírásból. Ez a szerkezet biztosítja, hogy a változások történeti sorrendben, logikusan elkülönítve jelenjenek meg. Amíg az adatok nem töltődnek be, az oldal egy **betöltési állapotot jelző szöveget** jelenít meg.

```
@* Betöltés közben tájékoztató szöveg jelenik meg *@
@if (regulations == null)
{
  <p class="loading">Adatok betöltése a boxutcából...</p>
}
else
{
  @* A szabályváltozások idővonalas megjelenítése évszám szerint *@
  <div class="timeline">
    @foreach (var reg in regulations)
    {
      <div class="timeline-item">
        <div class="timeline-year">@reg.reg_change_year</div>
        <div class="timeline-content">
          <p>@reg.Technical_regulations</p>
        </div>
      </div>
    }
  </div>
}
</div>
```

2.20.2 A SZABÁLYVÁLTOZÁSOK OLDAL MŰKÖDÉSE:

Az oldal inicializálásakor a komponens API-hívást indít, amelynek célja a technikai szabályváltozások adatainak lekérése. A kapott elemek egy listába kerülnek, majd ezek alapján dinamikusan felépül az idővonal.

A rendszer az első renderelést követően külön ellenőrzi a felhasználó bejelentkezési állapotát a böngésző helyi tárolójának segítségével. Amennyiben a felhasználó hitelesített, az oldal betölti a nevét és szerepkörét is. Ez elsősorban a navigációs sáv megfelelő működéséhez szükséges, mivel ettől függ, hogy milyen menüpontok jelennek meg.

A tartalom megjelenítése teljes mértékben adatvezérelt, vagyis az idővonal elemei nem statikusan szerepelnek az oldalon, hanem a backendből érkező adatok alapján jönnek létre. Ez lehetővé teszi, hogy az oldal tartalma a háttérrendszer módosításával frissíthető legyen anélkül, hogy a frontend kódon változtatni kellene.

Hiba esetén az alkalmazás jelenleg a konzolra írja ki a problémát, így a fejlesztő számára követhető, ha az adatlekérés során valamilyen hiba lép fel.

2.20.3 FONTOSABB METÓDUSOK

- **OnInitializedAsync()** – lekéri a szabályváltozások adatait az API-ból
- **OnAfterRenderAsync(bool firstRender)** – beolvassa a felhasználó bejelentkezési állapotát és adatait
- **ToggleMenu()** – mobilmenü állapotának kezelése
- **CloseMenu()** – a mobilmenü bezárása
- **GoToLogin()** – navigáció a bejelentkezési oldalra
- **Logout()** – kijelentkezteti a felhasználót és törli a helyi tárolt adatokat

2.20.4 FELHASZNÁLT ADATMODELLEK

A szabályváltozások oldal központi adatmodellje a `RegulationDto`, amely a technikai szabálmódosítások megjelenítéséhez szükséges adatokat tartalmazza. A modell egyik legfontosabb mezője a változás évét tároló azonosító, amely az idővonal kronológiai szerkezetét biztosítja. Emellett a modell tartalmazza a technikai szabályváltozás szöveges leírását is, amely az adott évhez kapcsolódó módosítás lényegét mutatja be.

Ez a modell teszi lehetővé, hogy az idővonal ne statikus tartalomként jelenjen meg, hanem adatbázis-alapú, dinamikusan frissíthető információs blokkokból épüljön fel.

Használt API végpontok

- **GET /api/Seasons** – a technikai szabályváltozások adatainak lekérése

2.21 STRATEGY.RAZOR

A stratégia oldal célja, hogy a felhasználó különböző gumistratégiákat szimulálhasson egy kiválasztott Forma–1 pályán. Az oldal egy interaktív számítási felületet biztosít, ahol a felhasználó megadhatja az egyes etapokhoz tartozó gumikeverékeket, a boxkiállások köreit, valamint opcionálisan egy kétkiállásos stratégiát is modellezhet.

A rendszer a megadott paraméterek alapján kiszámolja a köridőket, az összesített versenyidőt, az átlagos köridőt, majd az eredményeket grafikus és táblázatos formában jeleníti meg. Az oldal ezzel nem csupán statikus adatokat mutat, hanem aktív stratégiai elemzést tesz lehetővé.

2.21.1 A STRATÉGIA OLDAL FELÉPÍTÉSE

A stratégia oldal felső részén egy **navigációs sáv** található, amely az alkalmazás többi részéhez hasonló módon biztosítja az aloldalak közötti átjárást. A navigációs elemek itt is a felhasználó bejelentkezési állapotához és szerepköréhez igazodnak. Mivel ez az oldal csak hitelesített felhasználók számára érhető el, a tartalom megjelenítése is ehhez kötött.

```

<nav class="navbar">
  <div class="nav-container">
    <div class="nav-brand">
      <a href="/" class="logo">F1<span>ZONE</span></a>
    </div>

    <ul class="nav-links @(isMenuOpen ? "active" : "")">
      <li><a href="/" @onclick="CloseMenu">Kezdőlap</a></li>
      <li><a href="/tracks" @onclick="CloseMenu">Pályák</a></li>
      <li><a href="/drivers" @onclick="CloseMenu">Versenyzők</a></li>
      <li><a href="/schedule" @onclick="CloseMenu">Versenynaptár</a></li>
      <li><a href="/News" @onclick="CloseMenu">Hírek</a></li>
      <li><a href="/regulation_changes" @onclick="CloseMenu">Szabályok</a></li>
      <li><a href="/strategy">Stratégia</a></li>

      @* Admin felhasználók számára külön menüpont megjelenítése *@
      @if (isLoggedIn && userRole == "Admin")
      {
        <li><a href="/admin" @onclick="CloseMenu">Adatok szerkesztése</a></li>
      }

      @* Mobilnézetben a felhasználói műveletek külön blokkban jelennek meg *@
      <li class="mobile-only">
        @if (isLoggedIn)
        {
          <span class="user-welcome">Üdv, <strong>@userName</strong>!</span>
          <button class="login-btn" @onclick="Logout">KIJELENTKEZÉS</button>
        }
        else
        {
          <button class="login-btn" @onclick="GoToLogin">BEJELENTKEZÉS</button>
        }
      </li>
    </ul>

    <div class="nav-actions">
      @if (isLoggedIn)
      {
        @if (userRole == "Admin")
        {
          <button class="login-btn desktop-only" @onclick='() => NavManager.NavigateTo("/admin/home")'>ADMIN</button>
        }
        <span class="user-welcome desktop-only">Üdv, <strong>@userName</strong>!</span>
        <button class="login-btn desktop-only" @onclick="Logout">KIJELENTKEZÉS</button>
      }
      else
      {
        <button class="login-btn desktop-only" @onclick="GoToLogin">BEJELENTKEZÉS</button>
      }

      @* Hamburger menü mobil eszközökhöz *@
      <button class="burger-btn" @onclick="ToggleMenu">
        <span>@(isMenuOpen ? "x" : "☰")</span>
      </button>
    </div>
  </div>
</nav>

```

A navigációs sáv alatt helyezkedik el az oldal fő címe, majd ez alatt egy háromoszlopos elrendezésű szimulációs felület.

Az első oszlop a beállítások panel, ahol a felhasználó kiválaszthatja a pályát, az első és második etap gumikeverékét, valamint a boxkiállítás körét. Ugyanitt érhető el az opcionális kétkiállásos stratégia bekapcsolása is, amely további paramétereket jelenít meg, például a harmadik etap gumiját és a második kerékcseré körét.


```

@* 2. OSZLOP: VIZUÁLIS ELEMZÉS *@
<div class="analysis-card" style="background: #111111; border: 1px solid #252525; border-radius: 12px; box-shadow: 0 10px 20px rgba(0,0,0,0.4); padding: 1.5rem;">
  <h3 style="margin-top: 0; text-align: center; color: #fff; font-weight: 300; text-transform: uppercase; letter-spacing: 1px;">Teljesítmény Görbe</h3>

  <div style="display: flex; justify-content: center; gap: 3rem; margin-bottom: 1.5rem;">
    <div style="text-align: center;">
      <div style="color: #888; font-size: 0.7rem; text-transform: uppercase;">Versenyidő</div>
      <div style="color: #ff1801; font-size: 1.2rem; font-weight: bold;">@FormatTime(totalRaceTime)</div>
    </div>

    <div style="text-align: center;">
      <div style="color: #888; font-size: 0.7rem; text-transform: uppercase;">Pálya</div>
      <div style="color: #fff; font-size: 1.2rem;">@(allCircuits.FirstOrDefault(c => c.circuit_id.ToString() == selectedTrackId)?.circuit_name ?? "-")</div> @*itt modositas volt*@
    </div>

    <div style="text-align: center;">
      <div style="color: #888; font-size: 0.7rem; text-transform: uppercase;">Átlagidő</div>
      <div style="color: #fff; font-size: 1.2rem;">@(totalRaceTime > 0 ? $"{Math.Round(totalRaceTime / lapTimes.Count, 3)} mp" : "-")</div>
    </div>
  </div>

  <div id="chartParent" style="height: 380px; width: 100%; background: #111; padding: 15px; border-radius: 12px; border: 1px solid #333; position: relative;">
    <canvas id="strategyChart"></canvas>
  </div>
</div>

@* 3. OSZLOP: ADATOK *@
<div class="data-card" style="background: #111111; border: 1px solid #252525; border-radius: 12px; height: 650px; display: flex; flex-direction: column; padding: 1.5rem;">
  <h3 style="margin-top: 0; margin-bottom: 1rem; text-align: center; font-weight: 300; text-transform: uppercase; letter-spacing: 1px;">Telemetria</h3>
  <div class="table-container" style="flex-grow: 1; overflow-y: auto; padding-right: 5px;">
    <table style="width: 100%; text-align: left; border-collapse: collapse; font-size: 0.85rem;">
      <thead style="position: sticky; top: 0; background: #1a1a1a; z-index: 10;">
        <tr style="border-bottom: 2px solid #ff1801; color: #666;">
          <th style="padding: 10px 5px;">ID</th>
          <th style="padding: 10px 5px;">ID</th>
          <th style="padding: 10px 5px;">ÁLLAPOT</th>
        </tr>
      </thead>
      <tbody>
        @foreach (var lap in lapTimes)
        {
          double deg = lap.Time - tyreSpecs[lap.TyreType].baseTime;
          double limit = lap.TyreType == "Soft" ? 2.0 : lap.TyreType == "Hard" ? 3.5 : 2.5;
          bool isDead = deg > limit;

          <tr style="border-bottom: 1px solid #2a2a2a; @if(isDead ? "background: rgba(255,24,1,0.08);" : "")">
            <td style="padding: 10px 5px;">
              <span style="color: #666;">@lap.Number</span>
              <span style="color: @lap.TyreColorRgba; margin-left: 5px;">@lap.TyreType[0]</span>
            </td>
            <td style="padding: 10px 5px;">@if(isDead ? "color: #ff1801; font-weight: bold;" : "color: #eee;")>
              @lap.Time.ToString("F3")
            </td>
            <td style="padding: 10px 5px; color: #ff1801; font-size: 0.7rem; font-style: italic; font-weight: bold;">
              @if(isDead ? "⚠️ Sono, we are cooked!" : "")
            </td>
          </tr>
        }
      </tbody>
    </table>
  </div>
</div>

```

2.21.2 A STRATÉGIA OLDAL MŰKÖDÉSE:

Az oldal első renderelése után a rendszer ellenőrzi, hogy a felhasználó hitelesített állapotban van-e. Ha nem, akkor automatikusan átirányítja a bejelentkezési oldalra. Bejelentkezett felhasználó esetén az oldal betölti a felhasználó nevét és szerepkörét, majd lekéri az elérhető pályák listáját az API-ból.

A pálya kiválasztása után a rendszer automatikusan újraszámolja a teljes versenytávhoz szükséges körök számát a pálya hosszának figyelembevételével. Ez azért szükséges, mert a szimuláció alapja a Forma–1-ben használt hozzávetőleges versenytáv, amely körülbelül 305 kilométer.

A stratégiaszámítás a felhasználó által megadott paraméterek alapján történik. A rendszer meghatározza a pálya karakterisztikájából adódó szorzót, figyelembe veszi a gumitípus alap köridőit, valamint a körönkénti kopási mértékeket. Ezután etapokra bontva kiszámolja az összes köridőt, és ezekből előállítja a teljes versenyidőt.

A számítás során a rendszer külön kezeli az egykiállásos és a kétkiállásos stratégiát. Egykiállásos stratégia esetén a futam két etapból áll, míg kétkiállásos stratégia esetén három etap kerül kiszámításra. Mindkét esetben a boxkiállásokhoz egy fix időbüntetés kerül hozzáadásra.

A szimuláció befejezését követően a felhasználói felület frissül, majd a rendszer JavaScript segítségével kirajzolja a grafikonon a köridők alakulását. Ezzel párhuzamosan a telemetria táblázat is megjeleníti az összes szimulált kört.

Az oldal működésének fontos eleme, hogy a felhasználó minden paraméter módosítása után újraszámolhatja a teljes stratégiát, így különböző taktikai variációk könnyen összehasonlíthatók.

2.21.3 FONTOSABB METÓDUSOK

- **OnAfterRenderAsync(bool firstRender)** – ellenőrzi a hitelesítést, betölti a felhasználói adatokat és a pályalistát
- **UpdateTotalLaps()** – újraszámolja a teljes körszámot a kiválasztott pálya hossza alapján
- **CalculateStrategy()** – végrehajtja a teljes stratégiaszámítást, majd frissíti a grafikus megjelenítést
- **AddStint(int startLap, int endLap, string tyreType, double trackMult)** – kiszámolja egy adott etap köridőit
- **GetTyreColor()** – visszaadja az aktuális gumikeverékhez tartozó színt
- **GetColorRgba(string tyreName)** – a gumitípushoz tartozó RGBA színt adja vissza
- **FormatTime(double seconds)** – formázza az összesített időt megjelenítésre
- **ToggleMenu()** – kezeli a mobilmenü nyitását és zárását

- **CloseMenu()** – bezárja a mobilmenüt
- **GoToLogin()** – a bejelentkezési oldalra navigál
- **Logout()** – kijelentkezteti a felhasználót

2.21.4 FELHASZNÁLT ADATMODELLEK

A stratégia oldal működéséhez több belső adatstruktúra is szükséges, mivel az oldal egyszerre kezel pálya adatokat, felhasználói inputokat és szimulált köreredményeket.

A pályák adatainak tárolásához a **Circuits** modell kerül felhasználásra. Ez tartalmazza a pálya azonosítóját, nevét, országát, hosszát, a kanyarok számát, valamint egyéb leíró adatokat. A stratégia számítása szempontjából különösen fontos a pálya hossza és a kanyarok száma, mivel ezek hatással vannak a versenytáv meghatározására és a kopási szorzóra.

A szimuláció eredményeit a LapData modell tárolja. Ez a modell minden egyes körre külön eltárolja a kör sorszámát, a használt gumitípus nevét, a kiszámított köridőt, valamint a gumitípushoz tartozó vizuális színt. Ez a struktúra teszi lehetővé, hogy az eredmények egyszerre jelenjenek meg táblázatos és grafikus formában.

A gumik viselkedését egy belső szótár reprezentálja, amely minden gumikeverékhez hozzárendeli az alap köridőt és a körönkénti teljesítményromlást. Ez az adatstruktúra nem külön DTO, hanem a szimuláció matematikai modelljének része.

Használt API végpontok

- **GET /api/circuits** – a pályák adatainak lekérése a szimulációhoz

2.22 ADDDRIVER.RAZOR

Az új pilóta felvétele oldal célja, hogy az adminisztrátor új versenyzőt rögzíthessen a rendszer adatbázisában.

Ez az oldal az adminisztrációs felület részeként működik, és egy strukturált űrlapot biztosít a pilóták alapadatainak, statisztikai mutatóinak és megjelenítéshez szükséges jellemzőinek felvételéhez.

Az oldal lehetővé teszi, hogy az adminisztrátor programkód módosítása nélkül bővítse a rendszerben tárolt versenyzők listáját. Ez különösen fontos olyan adatvezérelt alkalmazások esetében, ahol a tartalom frissítése a kezelőfelületen keresztül történik.

2.22.1 AZ ÚJ PILÓTA FELVÉTELE OLDAL FELÉPÍTÉSE:

Az űrlap a következő főbb adatcsoportokat tartalmazza:

- a pilóta alapadatai, például név és nemzetiség
- statisztikai adatok, például győzelmek, világbajnoki címek, pontszám, dobogós helyezések és leggyorsabb körök száma
- karrierhez kapcsolódó információk, például bemutatkozás éve
- csapathoz tartozó adatok, például csapatnév és csapatszín
- megjelenítéshez szükséges adatok, például rajtszám és életrajzi leírás

```

<div class="f1-body login-page">
  <div class="login-card admin-card">
    <h3>Új pilóta felvétele</h3>

    <EditForm Model="@newDriver" OnValidSubmit="SaveDriver">
      <div class="form-group">
        <label>Név</label>
        <input type="text" @bind-value="newDriver.driver_name" required />
      </div>
      <div class="form-group">
        <label>Győzelmek</label>
        <input type="number" @bind-value="newDriver.wins" />
      </div>
      <div class="form-group">
        <label>VB címek</label>
        <input type="number" @bind-value="newDriver.championships" />
      </div>
      <div class="form-group">
        <label>Pontszám</label>
        <input type="number" step="0.5" @bind-value="newDriver.points" />
      </div>
      <div class="form-group">
        <label>Nemzetiség (pl. HUN, GER)</label>
        <input type="text" @bind-value="newDriver.nationality" />
      </div>
      <div class="form-group">
        <label>Bemutakozás éve</label>
        <input type="number" @bind-value="newDriver.debut_year" />
      </div>
      <div class="form-group">
        <label>Dobogós helyezések</label>
        <input type="number" @bind-value="newDriver.podiums" />
      </div>
      <div class="form-group">
        <label>Leggyorsabb körök</label>
        <input type="number" @bind-value="newDriver.fastest_laps" />
      </div>
      <div class="form-group">
        <label>Csapatnév (pl. Audi, Cadillac, Ferrari):</label>
        <input type="text" @bind="newDriver.teamname" class="form-control" />
      </div>
      <div class="form-group">
        <label>Csapatszín (Hex kód, pl. #FF0000):</label>
        <input type="color" @bind="newDriver.teamcolor" class="form-control color-picker" />
      </div>
    </EditForm>
  </div>
</div>

```

Az oldal alján két műveleti gomb található. Az egyik a pilóta adatainak mentésére szolgál, a másik a művelet megszakítását és az Admin főoldalra történő visszavigázást biztosítja.

Amennyiben a mentési folyamat során visszajelzés szükséges, az oldal egy **státuszüzenetet** is megjelenít, amely tájékoztatja a felhasználót a művelet eredményéről.

```

<button type="submit" class="hero-btn">PILOTA MENTÉSE</button>
<button type="button" class="logout-btn" @onclick='() => NavManager.NavigateTo("/admin")'>MÉGSE</button>
</EditForm>

@if (!string.IsNullOrEmpty(statusMessage))
{
  <p class="status-msg">@statusMessage</p>
}

```

2.22.2 AZ ÚJ PILÓTA FELVÉTELE OLDAL MŰKÖDÉSE

Az oldal működésének központi eleme egy EditForm, amely a megadott adatokat egy Drivers típusú objektumba köti. Az adminisztrátor által bevitt értékek automatikusan ebbe a modellbe kerülnek, így az űrlap elküldésekor a teljes új pilótaobjektum rendelkezésre áll.

A mentési folyamat az űrlap sikeres elküldése után indul el. Ekkor a rendszer POST kérést küld az API felé, amely tartalmazza az újonnan létrehozott versenyző összes megadott adatát. Ha a backend sikeresen feldolgozza a kérést, az oldal sikeres mentési üzenetet jelenít meg, majd rövid várakozás után visszairányítja az adminisztrátort az adminisztrációs felület főoldalára.

Sikertelen mentés esetén az oldal hibaüzenetet jelenít meg, így az adminisztrátor azonnal visszajelzést kap arról, hogy a művelet nem fejeződött be sikeresen.

Az oldal működése tehát egyszerre valósít meg adatbevitelt, backend kommunikációt, állapotkezelést és felhasználói visszajelzést.

2.22.3 FONTOSABB METÓDUSOK

- **SaveDriver()** – az űrlapban megadott adatokat elküldi az API-nak, majd a válasz alapján visszajelzést ad
- **NavManager.NavigateTo("/admin")** – megszakítás vagy sikeres mentés után visszairányít az admin oldalra

2.22.4 FELHASZNÁLT ADATMODELLEK

Az oldal központi adatmodellje a Drivers osztály, amely az új versenyző teljes adathalmazát reprezentálja. Ez a modell tartalmazza a pilóta nevét, statisztikai mutatóit, nemzetiségét, bemutatkozási évét, csapatnevét, csapatszínét, rajtszámát, valamint az életrajzi leírását.

Ez a modell egyszerre szolgál adatbeviteli célokat és backend kommunikációs célokat, vagyis ugyanaz a struktúra jelenik meg az űrlapmezők mögött, mint amelyet az API a mentés során feldolgoz.

Az oldal egy további egyszerű állapotváltozót is használ, a *statusMessage* szöveget, amely a mentési folyamat visszajelzését tárolja.

Használt API végpontok

- **POST /api/drivers** – új versenyző létrehozása és mentése az adatbázisba

2.23 ADMINDASHBOARD.RAZOR

Az adminisztrációs főoldal célja, hogy az adminisztrátor számára központi kezelőfelületet biztosítson a versenyzők adatainak karbantartásához. Az oldal lehetőséget ad új pilóta felvételére, meglévő pilóta adatainak szerkesztésére, valamint a rendszerben szereplő versenyzők törlésére.

Ez a modul a projekt adminisztrációs részének alapvető eleme, mivel ezen keresztül történik a versenyzőkhöz kapcsolódó adatok kezelése. Az oldal működése jogosultsághoz kötött, így kizárólag admin szerepkörű felhasználók férhetnek hozzá.

2.23.1 AZ ADMINISZTRÁCIÓS FŐOLDAL FELÉPÍTÉSE:

Az oldal tetején egy **visszanavigáló gomb** található, amely lehetővé teszi az adminisztrátor számára a főoldalra történő gyors visszatérést.

```
<div class="fl-body admin-container">
  <button class="back-btn" @onclick='() => NavManager.NavigateTo("/")'>
    <i class="bi bi-arrow-left"></i> VISSZA A FŐOLDALRA
  </button>
</div>
```

Ezt követi egy **fejlécszekció**, amely az adminisztrációs panel címét és rövid leírását tartalmazza. Ez a rész egyértelműen jelzi, hogy a felület a versenyzők adatainak kezelésére szolgál. A fejléc alatt helyezkedik el egy **műveleti sáv**, amely két fontos elemet tartalmaz: egy címblokkot, valamint egy új pilóta hozzáadására szolgáló gombot. Ez a gomb az új pilóta felvétele oldalra navigál, ahol az adminisztrátor új rekordot rögzíthet a rendszerben.

```
<div class="section-header">
  <h2>Adminisztrációs Panel</h2>
  <p>Pilóták statisztikáinak és adatainak kezelése</p>
</div>

<div class="admin-header">
  <h3>Pilóták statisztikáinak kezelése</h3>
  <button class="hero-btn add-btn" @onclick='() => NavManager.NavigateTo("/admin/add-driver")'>
    + ÚJ PILÓTA HOZZÁADÁSA
  </button>
</div>
```

A műveleti oszlop két gombot tartalmaz:

- **Szerkesztés** – a kiválasztott pilóta szerkesztőoldalára navigál
- **Törlés** – megerősítés után eltávolítja a pilótát a rendszerből

Amennyiben az adatok még nem töltődtek be, az oldal egy betöltési állapotot jelző szöveget jelenít meg.

```
@foreach (var driver in drivers)
{
  <tr>
    <td>@driver.driver_name</td>
    <td>@driver.wins</td>
    <td>@driver.championships</td>
    <td>
      <button class="edit-btn" @onclick="() => OpenEditModal(driver)">Szerkesztés</button>
      <button class="logout-btn" style="background-color: #dc3545; margin-left: 10px;"
        @onclick="() => ConfirmDelete(driver)">
        Törlés
      </button>
    </td>
  </tr>
}
</tbody>
</table>
```

2.23.2 AZ ADMINISZTRÁCIÓS FŐOLDAL MŰKÖDÉSE:

Az oldal első renderelése után a rendszer ellenőrzi, hogy a helyi tárolóban szereplő felhasználó rendelkezik-e adminisztrátori jogosultsággal. Ha a felhasználó nem admin, akkor az oldal automatikusan visszairányítja őt a főoldalra. Ez biztosítja, hogy az adminisztrációs funkciókhoz illetéktelen felhasználók ne férjenek hozzá.

Amennyiben a jogosultságellenőrzés sikeres, az oldal API-hívást indít, amely lekéri a pilóták listáját. Az így kapott adatok alapján épül fel a táblázatos megjelenítés.

A szerkesztési művelet kiválasztásakor a rendszer a megfelelő szerkesztőoldalra navigál, ahol a kiválasztott pilóta részletes adatainak módosítása lehetséges.

A törlési művelet végrehajtása előtt az alkalmazás megerősítést kér a felhasználótól. Ha az adminisztrátor jóváhagyja a műveletet, a rendszer DELETE kérést küld a backend felé, amely a kijelölt pilóta rekordját törli az adatbázisból. Sikeres művelet esetén az oldal azonnal frissíti a lokális listát, így a törölt elem eltűnik a táblázatból. Hiba esetén figyelmeztető üzenet jelenik meg.

A felület tehát egyszerre lát el jogosultságkezelési, adatlekérési és adatkarbantartási feladatokat.

2.23.3 FONTOSABB METÓDUSOK:

- **OnAfterRenderAsync(bool firstRender)** – ellenőrzi az admin jogosultságot, majd betölti a pilóták listáját
- **OpenEditModal(DriverDto driver)** – a kiválasztott pilóta szerkesztőoldalára navigál
- **ConfirmDelete(DriverDto selectedDriver)** – megerősítés után törli a kiválasztott pilótát, majd frissíti a listát

2.23.4 FELHASZNÁLT ADATMODELLEK:

Az adminisztrációs főoldal a pilóták adatainak megjelenítéséhez a DriverDto modellt használja. Ez a modell tartalmazza a versenyző azonosítóját, nevét, győzelmeinek számát, világbajnoki címeit, pontjait, nemzetiségét, bemutatkozási évét, dobogós helyezéseit, leggyorsabb köreit, valamint életrajzi adatait. Az admin főoldalon ezek közül elsősorban a név, a győzelmek és a világbajnoki címek jelennek meg.

Az oldal egy statusMessage változót is használ, amely a törlési műveletekhez kapcsolódó visszajelzések kezelésére szolgál. Ez lehetővé teszi, hogy a rendszer rövid státuszüzenettel jelezze a művelet eredményét.

Használt API végpontok:

- **GET /api/drivers/admin-list** – a pilóták adminisztrációs listájának lekérése
- **DELETE /api/drivers** – a kiválasztott pilóta törlése

2.24 ADMINHOME.RAZOR

A főoldali beállítások admin oldala arra szolgál, hogy az adminisztrátor kiválaszthassa, mely pálya és mely versenyző jelenjen meg a nyilvános kezdőlapon kiemelt tartalomként. Ez a megoldás lehetővé teszi, hogy a főoldal tartalma programkód módosítása nélkül, közvetlenül a kezelőfelületen keresztül változtatható legyen.

Az oldal az adminisztrációs modul részeként működik, és fontos szerepet játszik abban, hogy a kezdőlap dinamikusan és szerkeszthetően jelenítse meg a kiemelt elemeket.

2.24.1 A FŐOLDALI BEÁLLÍTÁSOK OLDAL FELÉPÍTÉSE:

Az oldal felső részén egy **admin navigációs sáv** található, amely egyszerű, célzott vezérlést biztosít. A bal oldalon az admin panel megnevezése látható, míg a jobb oldalon két műveleti gomb helyezkedik el: az egyik a főoldalra történő visszanavigálást teszi lehetővé, a másik pedig a kijelentkezést biztosítja.

```
@* Admin navigációs sáv *@  
<nav class="admin-navbar">  
  <div class="admin-navbar-left">  
    <h2>Admin Panel</h2>  
  </div>  
  
  <div class="admin-navbar-right">  
    <button class="nav-btn" @onclick="GoHome">Főoldal</button>  
    <button class="nav-btn logout-btn" @onclick="Logout">Kijelentkezés</button>  
  </div>  
</nav>
```

A navigációs sáv alatt jelenik meg az oldal **fejlécszekciója**, amely címet és rövid leírást tartalmaz. Ez a blokk egyértelműen meghatározza az oldal funkcióját, vagyis azt, hogy az adminisztrátor itt tudja kiválasztani a kezdőlapon megjelenő kiemelt pályát és versenyzőt.

```
<div class="admin-container">  
  <h1>Homepage beállítások</h1>  
  <p class="admin-subtitle">  
    Itt tudod kiválasztani, hogy melyik pálya és melyik versenyző jelenjen meg a főoldalon.  
  </p>  
</div>
```

A központi tartalom két különálló **admin kártyából** épül fel. Az első blokk a főoldalon megjelenő pálya kiválasztására szolgál, ahol egy legördülő listából választható ki a kívánt pálya. A második blokk a **“Driver of the Day”** elem kiválasztását teszi lehetővé, szintén legördülő lista segítségével.

```

@* A főoldalon megjelenő legutóbbi futam pályájának kiválasztása *@
<div class="admin-card">
  <h2>Legutóbbi futam</h2>
  <label for="trackSelect">Válassz pályát:</label>
  <select id="trackSelect" @bind="selectedCircuitId">
    <option value="0">-- Válassz pályát --</option>
    @foreach (var circuit in circuits)
    {
      <option value="@circuit.circuit_id">@circuit.circuit_name</option>
    }
  </select>
</div>

@* A főoldalon megjelenő "Driver of the Day" kiválasztása *@
<div class="admin-card">
  <h2>Driver of the Day</h2>
  <label for="driverSelect">Válassz versenyzőt:</label>
  <select id="driverSelect" @bind="selectedDriverId">
    <option value="0">-- Válassz versenyzőt --</option>
    @foreach (var driver in drivers)
    {
      <option value="@driver.driver_id">@driver.driver_name</option>
    }
  </select>
</div>

```

A két kiválasztó mező alatt található a **mentés gomb**, amely a módosítások adatbázisba mentését indítja el. Az oldal alján szükség esetén egy **visszajelző üzenet** is megjelenik, amely tájékoztatja a felhasználót a mentés sikerességéről vagy az esetleges hibáról.

```

@* Visszajelző üzenet sikeres mentés vagy hiba esetén *@
@if (!string.IsNullOrEmpty(message))
{
  <p class="message">@message</p>
}
</div>

```

2.24.2 A FŐOLDALI BEÁLLÍTÁSOK OLDAL MŰKÖDÉSE:

Az oldal inicializálásakor a rendszer több adatlekérdezést hajt végre. Először betölti a versenyzők listáját, majd a pályák listáját, végül lekéri az aktuálisan mentett főoldali beállításokat. Ezek alapján a rendszer automatikusan előre kijelöli a korábban elmentett pályát és versenyzőt a legördülő listákban.

Az első renderelést követően az alkalmazás ellenőrzi, hogy a felhasználó hitelesített állapotban van-e. Amennyiben nem, a rendszer visszairányítja a felhasználót a főoldalra. Ez biztosítja, hogy a beállítások módosítása csak jogosultsággal rendelkező felhasználó számára legyen elérhető.

A mentési folyamat során a rendszer ellenőrzi, hogy mindkét kiválasztás megtörtént-e. Ha az adminisztrátor nem választott ki pályát vagy versenyzőt, a felület hibaüzenetet jelenít meg. Sikeres kitöltés esetén a rendszer összeállít egy beállítási objektumot, majd ezt PUT kérés formájában elküldi a backend API felé.

Sikeres mentés esetén a felhasználó azonnali visszajelzést kap arról, hogy a módosítás bekerült az adatbázisba. Sikertelen mentés esetén a rendszer megjeleníti a szerver válaszát vagy a kivétel szövegét, így a hiba oka követhető marad.

Ez az oldal tehát egyszerre valósít meg adatbetöltést, jogosultságellenőrzést, állapotkezelést és konfigurációs mentést.

2.24.3 FONTOSABB METÓDUSOK:

- **OnInitializedAsync()** – betölti a versenyzők, pályák és az aktuális főoldali beállítások adatait
- **OnAfterRenderAsync(bool firstRender)** – ellenőrzi a felhasználó hitelesítési állapotát
- **GoHome()** – visszavigáz a főoldalra
- **Logout()** – kijelentkezteti a felhasználót
- **SaveHomepageSettings()** – elmenti a kiválasztott pályát és versenyzőt az adatbázisba

2.24.4 FELHASZNÁLT ADATMODELLEK:

Az oldal működéséhez három fő adatmodell kapcsolódik.

A HomepageSettings modell a főoldalon megjelenő kiemelt tartalmak mentésére szolgál. Ez tartalmazza a rekord azonosítóját, a kiválasztott pálya azonosítóját és a kiválasztott versenyző azonosítóját. Ez a modell biztosítja, hogy a rendszer egyértelműen tudja, mely elemeket kell a kezdőlapra megjeleníteni.

A DriverDto modell a versenyzők legördülő listában történő megjelenítéséhez szükséges adatokat tartalmazza. Bár a modell több mezőt is tárol, ezen az oldalon elsősorban a versenyző azonosítója és neve kerül felhasználásra.

A Circuits modell a pályák adatainak kezelésére szolgál. Ebből a modellből az oldal elsősorban a pálya azonosítóját és nevét használja a kiválasztó lista feltöltésére, de a modell további mezőket is tartalmaz a pályák részletesebb reprezentációjához.

Használt API végpontok:

- **GET /api/drivers/admin-list** – a versenyzők listájának lekérése
- **GET /api/circuits** – a pályák listájának lekérése
- **GET /api/homepagesettings/1** – az aktuális főoldali beállítások lekérése
- **PUT /api/homepagesettings/1** – a kiválasztott főoldali beállítások mentése

2.25 EDITDRIVER.RAZOR

A pilóta szerkesztő oldal az adminisztrációs modul része, amely lehetővé teszi egy meglévő versenyző adatainak módosítását. Az oldal célja, hogy az adminisztrátor közvetlenül frissíthesse a pilóták statisztikáit és egyéb adatait anélkül, hogy az adatbázist manuálisan kellene módosítani.

Ez az oldal egy paraméterezett útvonalon érhető el, ahol az URL tartalmazza a szerkesztendő pilóta azonosítóját.

2.25.1 AZ OLDAL FELÉPÍTÉSE:

Az oldal egy egyszerű, célorientált szerkesztő felületből áll, amely egy központi kártyában jelenik meg. A felső rész egy címblokk, amely dinamikusan jeleníti meg a szerkesztett pilóta nevét, így az adminisztrátor egyértelmű visszajelzést kap arról, hogy melyik rekordot módosítja.

```
<div class="f1-body login-page">
  <div class="login-card admin-card">
    <h3>@driver.driver_name szerkesztése</h3>
```

A cím alatt egy **státusz visszajelző blokk** található, amely a mentési folyamat állapotát jeleníti meg. Ez lehet információs, sikeres vagy hibát jelző üzenet.

```
@if (!string.IsNullOrEmpty(statusMessage))
{
  <p class="status-msg @statusClass">@statusMessage</p>
}
```

A fő tartalmat egy **EditForm** komponens alkotja, amely a pilóta adatainak módosítására szolgál. A form különböző input mezőket tartalmaz, amelyek a pilóta statisztikai és alapadatait fedik le, például győzelmek száma, világbajnoki címek, pontszám, nemzetiség, debütálási év, dobogós helyezések, leggyorsabb körök és életrajz.

```
<EditForm Model="@driver" OnValidSubmit="UpdateDriver">
  <div class="form-group">
    <label>Győzelmek száma</label>
    <input type="number" @bind-value="driver.wins" />
  </div>

  <div class="form-group">
    <label>VB címek</label>
    <input type="number" @bind-value="driver.championships" />
  </div>

  <div class="form-group">
    <label>Pontszám</label>
    <input type="number" step="0.5" @bind-value="driver.points" />
  </div>

  <div class="form-group">
    <label>Nemzetiség</label>
    <input type="text" @bind-value="driver.nationality" />
  </div>

  <div class="form-group">
    <label>Bemutatók év</label>
    <input type="number" @bind-value="driver.debut_year" />
  </div>

  <div class="form-group">
    <label>Dobogók</label>
    <input type="number" @bind-value="driver.podiums" />
  </div>

  <div class="form-group">
    <label>Leggyorsabb körök</label>
    <input type="number" @bind-value="driver.fastest_laps" />
  </div>

  <div class="form-group">
    <label>Életrajz</label>
    <textarea @bind="driver.biography" rows="3" class="form-control"></textarea>
  </div>

  <button type="submit" class="hero-btn">ADATOK FRISSÍTÉSE</button>
  <button type="button" class="logout-btn" @onclick='() => NavManager.NavigateTo("/admin")'>MÉGSE</button>
</EditForm>
```

A form alján két műveleti gomb található. Az egyik az adatok mentését indítja el, míg a másik megszakítja a műveletet és visszavigáz az admin főoldalra.

```
</div>
<button type="submit" class="hero-btn">ADATOK FRISSÍTÉSE</button>
<button type="button" class="logout-btn" @onClick='() => NavManager.NavigateTo("/admin")'>KÉGSE</button>
```

2.25.2 AZ OLDAL MŰKÖDÉSE:

Az oldal inicializálásakor a rendszer lekéri az összes pilóta adatát az API-ból, majd az URL-ben kapott azonosító alapján kiválasztja a szerkesztendő pilótát. Ez a kiválasztott objektum kerül a form modelljébe, így a mezők automatikusan kitöltődnek a meglévő adatokkal.

A felhasználó módosíthatja az adatokat, majd a mentés gombra kattintva elindíthatja a frissítési folyamatot. A rendszer ekkor egy PUT kérést küld az API felé, amely tartalmazza a módosított pilóta adatokat.

A mentés során az oldal visszajelzést ad a felhasználónak. Sikeres mentés esetén pozitív üzenet jelenik meg, majd rövid késleltetés után a rendszer visszairányítja a felhasználót az admin listázó oldalra. Hiba esetén a felület hibajelzést mutat, így az adminisztrátor tudomást szerez a problémáról.

2.25.3 FONTOSABB METÓDUSOK:

- **OnInitializedAsync()** – betölti az összes pilótát, majd kiválasztja a szerkesztendő
- **UpdateDriver()** – elküldi a módosított adatokat az API felé és kezeli a visszajelzést

2.25.4 FELHASZNÁLT ADATMODELL:

A komponens a DriverDto modellt használja, amely a pilóták adatainak átvitelére szolgál a frontend és a backend között. A modell tartalmazza a pilóta azonosítóját, nevét, statisztikáit és egyéb releváns adatait, amelyek szerkeszthetők ezen az oldalon.

Használt API végpontok:

- **GET /api/drivers/admin-list** – az összes pilóta adatainak lekérése
- **PUT /api/drivers/{id}** – a kiválasztott pilóta adatainak frissítése

2.26 LOGIN.RAZOR

A bejelentkezési oldal célja, hogy a felhasználó hitelesített módon hozzáférhessen az alkalmazás azon funkcióihoz, amelyek csak regisztrált vagy jogosultsággal rendelkező felhasználók számára érhetők el. Az oldal egy egyszerű, központi űrlapot biztosít az email cím és a jelszó megadására, majd a megadott adatokat a backend hitelesítési végpontja felé továbbítja.

Sikeres bejelentkezés esetén a rendszer eltárolja a felhasználó legfontosabb adatait a böngésző helyi tárolójában, így a későbbi oldalak is képesek felismerni a hitelesített állapotot.

2.26.1 A BEJELENTKEZÉSI OLDAL FELÉPÍTÉSE:

Az oldal egy központi elhelyezésű **bejelentkezési kártyából** áll, amely jól elkülönül a háttértől, így a felhasználó figyelmét közvetlenül az azonosítási folyamatra irányítja.

A kártya felső részén egy **fejlécblokk** található, amely tartalmazza az alkalmazás logóját, az oldal címét és egy rövid leírást. Ez a rész egyértelművé teszi az oldal funkcióját, és kontextust ad a felhasználónak.


```
<div class="f1-body login-page">
  <div class="login-card">
    <div class="login-header">
      <a href="/" class="logo">F1</span>ZONE</span></a>
      <h2>Bejelentkezés</h2>
      <p>Lépj be a profilodba a teljes élményért!</p>
    </div>
```

A központi tartalmi egység egy **bejelentkezési űrlap**, amely két alapvető mezőt tartalmaz:

- email cím
- jelszó

```
@* Bejelentkezési űrlap, amely sikeres validáció után meghívja a login kezelő metódust *@
<EditForm Model="@loginModel" OnValidSubmit="HandleLogin">
  <div class="form-group">
    <label>Email cím</label>
    <input type="email" @bind-value="loginModel.Email" placeholder="pelda@email.com" required />
  </div>

  <div class="form-group">
    <label>Jelszó</label>
    <input type="password" @bind-value="loginModel.Password" placeholder="*****" required />
  </div>
```

Az űrlap alatt szükség esetén egy **hibaüzenet blokk** jelenik meg, amely tájékoztatja a felhasználót a sikertelen hitelesítésről.

```
@* Sikertelen bejelentkezés esetén hibaüzenet megjelenítése *@
@if (!string.IsNullOrEmpty(errorMessage))
{
  <div class="error-msg">@errorMessage</div>
}

<button type="submit" class="hero-btn">BEJELENTKEZÉS</button>
```

A kártya alsó részén egy **navigációs hivatkozás** található, amely a regisztrációs oldalra irányítja azokat a felhasználókat, akik még nem rendelkeznek fiókkal.

```
<div class="login-footer">
  <p>Nincs még fiókod? <a href="/register">Regisztráció</a></p>
</div>
```

2.26.2 A BEJELENTKEZÉSI OLDAL MŰKÖDÉSE:

Az oldal működésének központi eleme az `EditForm`, amely a megadott adatokat a `LoginDto` modellhez köti. A felhasználó által megadott email cím és jelszó automatikusan ebbe az objektumba kerül, így az űrlap elküldésekor a teljes bejelentkezési adatcsomag rendelkezésre áll.

Az űrlap sikeres elküldése után a rendszer POST kérést küld a hitelesítési API felé. A backend ellenőrzi a megadott hitelesítő adatokat, majd siker esetén visszaadja a felhasználóhoz kapcsolódó adatokat, például azonosítót, felhasználónevet, jogosultsági szintet és tokent.

A frontend ezeket az adatokat eltárolja a böngésző helyi tárolójában. Ez biztosítja, hogy az alkalmazás későbbi oldalai is hozzáférjenek a felhasználó hitelesítési állapotához, valamint a szerepköréhez. Különösen fontos a `userRole` és az `isAuthenticated` értékek eltárolása, mivel ezek alapján dől el, hogy a rendszer mely funkciókat teszi elérhetővé.

Sikertelen bejelentkezés esetén az oldal nem navigál tovább, hanem hibaüzenetet jelenít meg a felhasználónak. Ez azonnali visszajelzést biztosít, és lehetővé teszi az adatok újbóli megadását.

Sikeres hitelesítés után a rendszer visszairányítja a felhasználót a főoldalra, immár hitelesített állapotban.

2.26.3 FONTOSABB METÓDUSOK:

- **HandleLogin()** – elküldi a bejelentkezési adatokat az API felé, majd kezeli a választ
- **NavManager.NavigateTo("/", forceLoad: true)** – sikeres hitelesítés után visszairányít a főoldalra

2.26.4 FELHASZNÁLT ADATMODELLEK:

Az oldal a `LoginDto` modellt használja a bejelentkezési űrlap adatainak tárolására. Ez a modell tartalmazza a felhasználó által megadott email címet és jelszót, vagyis a hitelesítéshez szükséges minimális adatokat.

A szerverről érkező válasz feldolgozásához a rendszer `UserDto` objektumot használ. Ez a modell tartalmazza a sikeres bejelentkezéshez kapcsolódó adatokat, például a felhasználó azonosítóját, nevét, szerepkörét és a hitelesítési tokent. Ezek az információk később a felhasználói állapotkezelés alapját adják.

Az oldal ezen kívül egy `errorMessage` nevű állapotváltozót használ, amely a sikertelen hitelesítési kísérletekhez tartozó hibaüzenetet tárolja.

Használt API végpont:

- **POST /api/auth/login** – a felhasználó hitelesítése és a bejelentkezési adatok ellenőrzése

2.27 REGISTER.RAZOR

A regisztrációs oldal célja, hogy új felhasználók számára lehetőséget biztosítson saját fiók létrehozására. Az oldal egy egyszerű és áttekinthető űrlapot kínál, amelyen keresztül a felhasználó megadhatja a regisztrációhoz szükséges alapadatait. A megadott információk a backend hitelesítési végpontjához kerülnek továbbításra, amely létrehozza az új felhasználói rekordot a rendszerben.

A regisztrációs oldal fontos szerepet tölt be az alkalmazásban, mivel ezen keresztül válik lehetővé a későbbi bejelentkezés, valamint a személyre szabott és jogosultság alapú funkciók használata.

2.27.1 A REGISZTRÁCIÓS OLDAL FELÉPÍTÉSE

Az oldal felépítése hasonló a bejelentkezési oldaléhoz, így a két felület vizuálisan és funkcionálisan is egységes felhasználói élményt nyújt. A központi részen egy **regisztrációs kártya** található, amely a teljes űrlapot és a hozzá tartozó kiegészítő információkat tartalmazza. A kártya felső részén egy **fejlécblokk** helyezkedik el, amely megjeleníti az alkalmazás logóját, az oldal címét és egy rövid leírást. Ez a rész egyértelműen jelzi, hogy a felület új fiók létrehozására szolgál.

```
<div class="f1-body login-page">
  <div class="login-card">
    <div class="login-header">
      <a href="/" class="logo">F1<span>ZONE</span></a>
      <h2>Regisztrálj, hogy több funkcióhoz férj hozzá!</h2>
      <p>Hozd létre a profilod és kövesd az F1 világát!</p>
    </div>
  </div>
</div>
```

A fejléc alatt található a **regisztrációs űrlap**, amely három alapvető mezőt tartalmaz:

- felhasználónév
- email cím
- Jelszó

```
<EditForm Model="@registerModel" OnValidSubmit="HandleRegister">
  <div class="form-group">
    <label>Felhasználónév</label>
    <input type="text" @bind-value="registerModel.Username" placeholder="Schumi91" required />
  </div>

  <div class="form-group">
    <label>Email cím</label>
    <input type="email" @bind-value="registerModel.Email" placeholder="pelda@email.com" required />
  </div>

  <div class="form-group">
    <label>Jelszó</label>
    <input type="password" @bind-value="registerModel.Password" placeholder="*****" required />
  </div>
</EditForm>
```

Az űrlap alatt szükség esetén egy **hibaüzeneti blokk** jelenik meg, amely tájékoztatja a felhasználót, ha a regisztráció nem sikerült.

```
@if (!string.IsNullOrEmpty(errorMessage))
{
  <div class="error-msg">@errorMessage</div>
}
else
{
  <div class="success-msg">Regisztráció sikeres!</div>
}
```

A kártya alsó részén egy **bejelentkezési hivatkozás** található, amely lehetővé teszi, hogy a már regisztrált felhasználók gyorsan átváltsanak a belépési felületre.

```
<div class="login-footer">
  <p>Van már fiókod? <a href="/login">Bejelentkezés</a></p>
</div>
```

2.27.2 A REGISZTRÁCIÓS OLDAL MŰKÖDÉSE

Az oldal működésének központi eleme egy EditForm, amely a felhasználó által megadott adatokat a RegisterDto modellhez köti. Az inputmezőkön keresztül megadott értékek automatikusan a modell megfelelő mezőibe kerülnek.

Az űrlap elküldése után a rendszer POST kérést küld a backend hitelesítési végpontjához. A kérés tartalmazza a felhasználónév, email cím és jelszó adatait, amelyek alapján a backend megpróbálja létrehozni az új felhasználót.

Sikeres regisztráció esetén a rendszer nem jelentkezeti be automatikusan a felhasználót, hanem átirányítja a bejelentkezési oldalra. Ezzel világosan elkülönül a regisztráció és a hitelesítés folyamata.

Sikertelen regisztráció esetén az oldal hibaüzenetet jelenít meg. Ez jellemzően olyan helyzetekben fordulhat elő, amikor az email cím már szerepel a rendszerben, vagy a backend nem tudta végrehajtani a műveletet.

Az oldal így egyszerre valósítja meg az adatbevitelt, a backend kommunikációt és a felhasználói visszajelzést.

2.27.3 FONTOSABB METÓDUSOK

- **HandleRegister()** – elküldi a regisztrációs adatokat az API felé, majd kezeli a választ
- **NavManager.NavigateTo("/login")** – sikeres regisztráció után átirányít a bejelentkezési oldalra

2.27.4 FELHASZNÁLT ADATMODELLEK

Az oldal központi adatmodellje a *RegisterDto*, amely a regisztrációs űrlap által bekért adatokat tárolja. Ez a modell tartalmazza a felhasználónevét, email címét és jelszavát, vagyis az új felhasználói fiók létrehozásához szükséges minimális információkat.

Az oldal ezen kívül egy *errorMessage* állapotváltozót is használ, amely a sikertelen regisztrációhoz kapcsolódó hibaüzenetet tárolja és jeleníti meg a felhasználó számára.

Használt API végpont

- **POST /api/auth/register** – új felhasználói fiók létrehozása

2.28 RACEDETAILS.RAZOR

A verseny részletező oldal célja, hogy a felhasználó számára egy adott futamhoz kapcsolódó részletes információkat jelenítsen meg. Az oldal dinamikusan működik, és az URL-ben kapott azonosító alapján tölti be a megfelelő verseny adatait az API-ból.

Ez az oldal a versenynaptár oldal kiegészítéseként működik, lehetővé téve a felhasználók számára, hogy egy adott futamról részletesebb információkat kapjanak.

2.28.1 AZ OLDAL FELÉPÍTÉSE

Az oldal egy egyszerű, jól áttekinthető struktúrát követ, amely a navigációt és az információk megjelenítését helyezi előtérbe. A felső részen egy **visszanavigáló gomb** található, amely lehetővé teszi, hogy a felhasználó gyorsan visszatérjen a versenynaptár oldalra.

```
<div class="nav-actions">
  <button class="back-btn" @onclick="GoBack">
    <span class="arrow">←</span> Vissza a versenynaptárhoz
  </button>
</div>
```

Az oldal fő tartalma egy központi konténerben helyezkedik el, amely a kiválasztott futam adatait jeleníti meg. Amíg az adatok betöltése folyamatban van, egy **betöltési állapot** jelenik meg, amely vizuális visszajelzést ad a felhasználónak.

```
<div class="details-container">
  @if (details == null)
  {
    <div class="loading-box">
      <div class="spinner"></div>
      <p>Adatok betöltése a paddockból...</p>
    </div>
  }
</div>
```

Sikeres adatlekérés után az oldal az alábbi információkat jeleníti meg:

- a verseny neve
- a pálya neve
- a verseny időpontja
- a futam aktuális státusza (lezajlott vagy közelgő)

Az adatok kártyás elrendezésben jelennek meg, amely jól elkülöníti a fő és kiegészítő információkat.

```
<h1 class="race-title">@details.RaceName - Részletek</h1>

<div class="details-grid">
  <div class="info-card main-info">
    <h3><i class="bi bi-flag"></i> Pálya: @details.TrackName</h3>
    <p class="date-text">
      <i class="bi bi-calendar3"></i>
      Időpont: @details.RaceDate.ToString("yyyy. MMMM dd. HH:mm")
    </p>
  </div>

  <div class="info-card extra-info">
    <p class="status">Státusz: @(details.RaceDate < DateTime.Now ? "Lezajlott" : "Közelgő")</p>
  </div>
</div>

<div class="nav-actions">
  <button class="back-btn" @onclick="GoBack">
    <span class="arrow">←</span> Vissza a versenynaptárhoz
  </button>
</div>
```

2.28.2 AZ OLDAL MŰKÖDÉSE

Az oldal egy paraméterezett útvonalon érhető el, ahol az URL tartalmazza a kiválasztott futam azonosítóját. Ez az azonosító kerül be a komponens Id paraméterébe.

Az oldal inicializálásakor a rendszer egy GET kérést küld az API felé, amely a megadott azonosító alapján visszaadja a futam adatait. Az így kapott adatokat a details változó tárolja, amelyhez a felület közvetlenül kötődik.

Amíg az API válasz nem érkezik meg, a felhasználó egy betöltési animációt lát. Ezután a rendszer automatikusan frissíti a felületet a lekért adatokkal.

Az oldal dinamikusan meghatározza a futam státuszát is a jelenlegi idő és a verseny időpontjának összehasonlításával. Ez alapján jelenik meg, hogy a futam már lezajlott vagy még várható.

A visszavigáló gomb lehetővé teszi a felhasználó számára, hogy egyetlen kattintással visszatérjen a versenynaptár oldalra.

2.28.3 FONTOSABB METÓDUSOK

- **OnInitializedAsync()** – lekéri a kiválasztott futam adatait az API-ból
- **GoBack()** – visszavigál a versenynaptár oldalra

2.28.4 FELHASZNÁLT ADATMODELL

Az oldal a `RaceDto` modellt használja, amely a futamok legfontosabb adatait tartalmazza. Ez a modell magában foglalja a futam azonosítóját, nevét, a pálya nevét, a verseny időpontját és az országcódot.

A modell célja, hogy egységes adatstruktúrát biztosítson a frontend és a backend közötti kommunikáció során.

Használt API végpont:

- **GET /api/raceschedules/{id}** – egy adott futam részletes adatainak lekérése

2.29 RACERESULT.RAZOR

A versenyeredmény oldal célja, hogy a felhasználó egy adott futam hivatalos eredményeit áttekinthető, vizuálisan kiemelt formában megismerhesse. Az oldal egy kiválasztott versenyhez kapcsolódóan megjeleníti a dobogós helyezetteket, a leggyorsabb kör adatait, valamint a teljes versenyidőt.

Ez az oldal dinamikusan működik, és az URL-ben kapott futamazonosító alapján tölti be a megfelelő adatokat a backendből. Az eredményoldal így a versenynaptár és a múltbeli futamok adatait kiegészítő részletes információs nézetként működik.

2.29.1 A VERSENYEREDMÉNY OLDAL FELÉPÍTÉSE

Az oldal központi eleme egy **eredménykártya**, amely a kiválasztott futamhoz kapcsolódó legfontosabb adatokat jeleníti meg. A felső részben egy **címblöck** található, amely a futam nevét és az „eredmények” megjelölést tartalmazza. Ez azonnal jelzi a felhasználó számára, hogy egy konkrét verseny végső eredményeit látja.

A fő tartalmi rész két nagyobb blokkra bontható.

Az első a **dobogós szekció**, amely a verseny első három helyezettjét jeleníti meg külön kiemeléssel. A dobogós helyek vizuális jelölése segíti a gyors értelmezést, és hangsúlyt ad a győztes, második és harmadik helyezett megkülönböztetésének.

```
<div class="podium-section">
  <div class="podium-row first-place">
    <div class="position-badge gold">1</div>
    <div class="driver-info">
      <span class="driver-name">@(Gyoztes != null ? Gyoztes.driver_name : "Töltés...")</span>
    </div>
  </div>

  <div class="podium-row">
    <div class="position-badge silver">2</div>
    <div class="driver-info">
      <span class="driver-name">@(Masodik != null ? Masodik.driver_name : "Nincs adat")</span>
    </div>
  </div>

  <div class="podium-row">
    <div class="position-badge bronze">3</div>
    <div class="driver-info">
      <span class="driver-name">@(Harmadik != null ? Harmadik.driver_name : "Nincs adat")</span>
    </div>
  </div>
</div>
```

A második a **statisztikai szekció**, amely két fontos eredményt mutat meg:

- a leggyorsabb kör időeredményét és a hozzá tartozó pilótát
- a teljes hivatalos versenyidőt

```

<div class="stats-section">
  <div class="stat-box">
    <span class="stat-label">LEGGYORSABB KÖR</span>
    <span class="stat-value highlight">@Eredmeny.fastest_lap_time.ToString(@"mm\:ss\.fff")</span>
    <span class="stat-driver">@(Leggyorsabb != null ? Leggyorsabb.driver_name : "Nincs adat")</span>
  </div>

  <div class="stat-box">
    <span class="stat-label">TELJES VERSENYIDŐ</span>
    <span class="stat-value">@Eredmeny.race_time.ToString(@"hh\:mm\:ss\.fff")</span>
    <span class="stat-driver">Hivatalos időeredmény</span>
  </div>
</div>

```

Amennyiben az adatok még nem érhetők el, az oldal egy **betöltési állapotot** jelenít meg.

```

@if (Eredmeny == null)
{
  <div class="loading-spinner">Töltés...</div>
}

```

2.29.2 A VERSENYEREDMÉNY OLDAL MŰKÖDÉSE

Az oldal működése az URL-ben kapott `Raceld` paraméterre épül. Az oldal inicializálásakor a rendszer először lekéri az adott futam eredményadatait a múltbeli versenyek végpontjáról. Ez az alapadat határozza meg, hogy mely pilóták és statisztikai információk jelenjenek meg az oldalon.

A futam eredményének lekérése után a rendszer további API hívásokkal tölti be a dobogós helyezettek adatait. A győztes lekérése kötelezően megtörténik, míg a második és harmadik helyezett csak akkor kerül lekérésre, ha az adatbázisban ezek az azonosítók rendelkezésre állnak.

Ezt követően az oldal külön lekéri a futam nevét a versenynaptárhoz kapcsolódó végpontról, hogy az eredménykártya fejlécében a pontos versenynév jelenhessen meg. Végül egy újabb API hívás történik a leggyorsabb körhöz tartozó pilóta adatainak meghatározására.

Az oldal tehát több egymásra épülő adatlekérésből állítja össze a végső megjelenítést. Ez azt jelenti, hogy a felület nem egyetlen összetett objektumból dolgozik, hanem több, egymással kapcsolatban álló rekordból állítja elő a futam teljes eredményképét.

Az első renderelést követően a rendszer külön ellenőrzi a felhasználó bejelentkezési állapotát is a helyi tároló alapján. Ez elsősorban a navigációs sáv helyes működéséhez szükséges.

2.29.3 FONTOSABB METÓDUSOK

- **OnInitializedAsync()** – lekéri a futam eredményadatait, a dobogós pilótákat, a futam nevét és a leggyorsabb körhöz tartozó versenyzőt
- **OnAfterRenderAsync(bool firstRender)** – betölti a felhasználó hitelesítési állapotát és adatait
- **GoToLogin()** – a bejelentkezési oldalra navigál
- **ToggleMenu()** – kezeli a mobil menü állapotát
- **CloseMenu()** – bezárja a mobil menüt
- **Logout()** – kijelentkezteti a felhasználót és törli a helyi tároló adatait

2.29.4 FELHASZNÁLT ADATMODELLEK

Az oldal központi adatmodellje a Races modell, amely a múltbeli futam eredményeit tartalmazza. Ez foglalja magában többek között a győztes, a második és harmadik helyezett azonosítóját, a leggyorsabb körhöz tartozó pilóta azonosítóját, a leggyorsabb kör idejét, valamint a teljes versenyidőt.

A dobogós helyezettek és a leggyorsabb kört futó versenyző adatainak megjelenítéséhez a Drivers modell kerül felhasználásra. Ez a modell biztosítja a pilóták nevének és egyéb jellemzőinek megjelenítését.

A futam nevének megjelenítéséhez a RaceSchedules modell tartozó adatai is szükségesek, mivel ebből kerül kinyerésre a verseny hivatalos megnevezése.

Az oldal emellett több külön változóban tárolja az egyes szereplőkhöz tartozó objektumokat, például a győztest, a második és harmadik helyezettet, illetve a leggyorsabb kör pilótáját.

Használt API végpontok

- **GET /api/pastraces/{RaceId}** – az adott futam eredményadatainak lekérése
- **GET /api/drivers/{id}** – egy adott pilóta adatainak lekérése
- **GET /api/raceschedules/{RaceId}** – a futam nevének és alapadatainak lekérése

2.30 KÉPERNYŐTERVEK

Bejelentkezés, Regisztráció

The diagram shows a wireframe for a login and registration interface. It consists of a dark gray rectangular area with a white border. Inside, there are several white rectangular boxes with black text, arranged vertically. The boxes are labeled: 'oldalnev' (top), 'bejelentkezés' (second), 'emailcim' (third), 'jelszo' (fourth), 'bejelentkezés gomb' (fifth), and 'regisztráció' (bottom right).

oldalnev

szoveg?

felhasznalonev

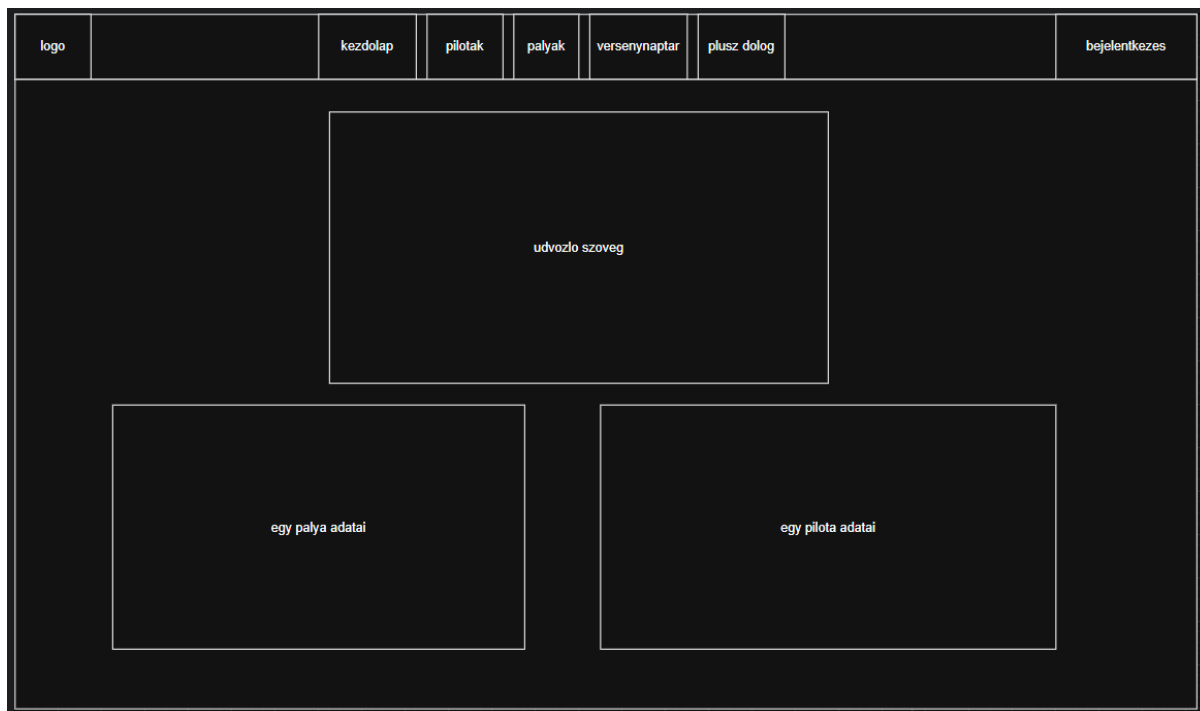
emailcim

jelszo

regisztrációs gomb

bejelentkezés

Főoldal



Pilóták oldal



Piloták adatai oldal

logo		kezdolap	pilotak	palyak	versenynaptar	plusz dolog		bejelentkezés
------	--	----------	---------	--------	---------------	-------------	--	---------------

pilota nev

szoveg a pilotarol

statisztikai a pilotanak

Admin oldal

The diagram illustrates the layout of the Admin page. It features a dark background with several white-bordered buttons and a table.

- At the top left is a button labeled "vissza a fooldalra gomb".
- At the top right is a button labeled "admin oldal".
- Below the "admin oldal" button is a button labeled "uj pilota gomb".
- In the center is a table with 6 rows and 3 columns. Each row contains a button labeled "pilota", a button labeled "szerkesztes", and a button labeled "torles".

2.31 BEJELENTKEZÉSI ADATOK

Email	Szerepkör	Jelszó
admin@f1.hu	admin	jelszo123
pista@f1.hu	user	jelszo123

2.32 FEJLESZTÉSI LEHETŐSÉGEK

Az F1Zone platform jelenlegi verziója szilárd architektúrát és stabil működést biztosít a Formula–1-es adatok kezeléséhez és megjelenítéséhez.

A választott technológiai megoldások (Blazor Web App és .NET Core Web API) rendkívül jól skálázhatóak, így számos területen nyílik lehetőség a további bővítésre. Az alábbiakban néhány olyan irányvonalat foglalmaztunk meg, amelyek a jövőben tovább növelhetik a felhasználói élményt és az oldal funkcionalitását:

1. Közösségi és interaktív funkciók bővítése

- **Tippjáték és rajongói interakciók:** Egy "F1 Guess" jellegű tippjáték modul beépítése, ahol a regisztrált felhasználók megtippelhetik az időmérő vagy a futam dobogósait, és ezért pontokat gyűjthetnek egy globális ranglistán.

2. Értesítési rendszer bevezetése

- **Automatikus emlékeztetők:** E-mailes vagy böngészős (Push) értesítések küldése a kedvencnek jelölt pilóták híreiről, illetve a közelgő versenyhétvégék eseményeinek (szabadedzések, időmérő) kezdete előtt.

3. Fejlett vizualizáció és adatelemzés

- **Interaktív grafikonok:** Dinamikus diagramok (pl. Chart.js komponensek) integrálása, amelyek vizuálisan is megmutatják a csapatok pontversenyének alakulását az évad során, vagy lehetővé teszik két pilóta történelmi statisztikáinak közvetlen összehasonlítását.

4. Adatok exportálása és riportkészítés

- **Statisztikák letöltése:** Olyan funkció kialakítása, amely lehetővé teszi a felhasználók (vagy csak az adminisztrátorok) számára, hogy a pilóták történelmi statisztikáit, a csapatok eredményeit vagy éppen az aktuális versenynaptárat egyetlen kattintással PDF vagy CSV (Excel) formátumban exportálják. Ez jelentősen növelné a platform hasznosíthatóságát a statisztikák iránt érdeklődő rajongók számára.

2.33 TESZTELÉSI TERV

A tesztelési terv célja az F1Zone platform funkcióinak szigorú ellenőrzése, az esetleges szoftverhibák (bugok) feltárása és javítása, valamint a felhasználói élmény és a rendszerbiztonság optimalizálása a végleges élesítés előtt.

2.33.1 TESZTELÉSI CÉLOK

- Az oldal alapvető üzleti logikájának és funkcióinak (CRUD műveletek) hibátlan működésének ellenőrzése.
- A felhasználói élmény (UX) és a reszponzivitás értékelése különböző felbontású eszközökön.
- A hitelesítési, jogosultságkezelési és adatvédelmi szempontok felülvizsgálata (JWT tokenek és jelszótitkosítás tesztelése).

2.33.2 TESZTELÉSI TÍPUSOK

2.33.2.1 FUNKCIONÁLIS TESZTEK

- **Pilóták és csapatok keresése, listázása:** Ellenőrizni, hogy a szűrési funkciók (pl. csapatnév alapján) megfelelően működnek-e, és a lekérdezett statisztikák relevánsak-e.
- **Kezdőlap dinamikus tartalmai:** Biztosítani, hogy a visszaszámláló a pontos szerveridőhöz igazodik, illetve a kiemelt pályák és pilóták adatai hibamentesen betöltődnek.
- **Felhasználói regisztráció és bejelentkezés:** Tesztelni a regisztrációs folyamatot, a jelszavak BCrypt hashelését, és a sikeres bejelentkezést.
- **Adminisztrációs funkciók:** Ellenőrizni, hogy a megfelelő jogosultsággal (Admin) rendelkező felhasználók képesek-e új entitásokat (pl. szabályváltozásokat, futamokat) hozzáadni, meglévőket szerkeszteni vagy törölni.

2.33.2.2 EGYSÉGTESZTEK

- **Backend logika:** A *GenericF1ZoneService* és a származtatott szolgáltatások adatbázis-műveleteinek tesztelése (mentés, frissítés, törlés) az Entity Framework Core kontextuson keresztül.
- **Érvényesítések :** Ellenőrizni, hogy a DTO modellek és az űrlapok (formok) megfelelően kezelik-e és utasítják-e el a hibás vagy hiányzó adatokat.

2.33.2.3 INTEGRÁCIÓS TESZTEK

- **Frontend és Backend összekapcsolása:** Biztosítani, hogy a Blazor kliensalkalmazás és a .NET Core Web API közötti REST kommunikáció (HttpClient hívások) gyors és zökkenőmentes.
- **Jogosultságkezelés (Authorization):** Tesztelni a végpontok védelmét; ellenőrizni, hogy a bejelentkezés nélküli felhasználók hozzáférése megtagadásra kerül-e a dedikált adminisztrációs útvonalakon és API végpontokon.

2.33.3 TESZTELÉSI KÖRNYEZET

- **Böngészők:** Google Chrome, Mozilla Firefox, Microsoft Edge, Safari.
- **Eszközök:** Asztali számítógépek (különböző monitorfelbontásokkal), okostelefonok.
- **Operációs rendszerek:** Windows, iOS, Android.

2.33.4 TESZTESETEK

2.33.4.1 REGISZTRÁCIÓ, BEJELENTKEZÉS

Teszteset	Tesztelési cél	Lépések	Elvárt eredmény
1.	Sikeres regisztráció	1. Regisztrációs oldal megnyitása 2. Töltse ki az összes mezőt 3. Küldje el a regisztrációját	Automatikus visszaérkezés a bejelentkezés oldalra.
2.	Hibás email-cím kezelése	Hibás email-cím megadása, pl: <code>teszt2&.hu</code>	Hibaüzenet: „Kérjük, írjon egy „@” karaktert az e-mail címbe. A(z) „teszt2&.hu” címből hiányzik a „@” jel.”
3.	Duplikált regisztráció	Egy már regisztrált e-mail címmel történő regisztrálás.	Hibaüzenet: „A regisztráció nem sikerült. Lehet, hogy már létezik ez az email?”
4.	Sikeres bejelentkezés	Bejelentkezés egy helyes email és jelszó kombinációval.	Átírányítás a főoldalra.
5.	Hibás bejelentkezés	Bejelentkezés egy hibás email és jelszó kombinációval.	Hibaüzenet: „Hibás email vagy jelszó!”

2.33.4.2 PILÓTÁK LISTÁZÁSA

Teszteset	Tesztelési cél	Lépések	Elvárt eredmény
1.	Pilóták megjelenítése	A Versenyzők menüpont megnyitása a navigációs sávból.	Megjelenik az összes pilóta kártyája ki található az adatbázisban.
2.	További információ egy pilótáról.	Pilóta kártyájára történő kattintás.	Sikeres átírányítás a választott pilóta adatlapjára.

2.33.4.3 KEDVENCEK FUNKCIÓ

Teszteset	Tesztelési cél	Lépések	Elvárt eredmény
1.	Pilóta hozzáadása a kedvencekhez.	1. Sikeres bejelentkezés. 2. A Versenyzők oldal megnyitása. 3. A versenyző kártyáján lévő szürke szív ikon kattintása.	A kattintott szív szürkéről pirosra vált.
2.	Pilóta törlése a kedvencekből.	1. Sikeres bejelentkezés. 2. A Versenyzők oldal megnyitása. 3. A versenyző kártyáján lévő piros szív ikon kattintása.	A szív piros színről szürke színre változik át.
3.	Kedvenc pilóta vagy pilóták szűrése a hírek oldalon.	1. Sikeres bejelentkezés. 2. A Versenyzők oldal megnyitása. 3. A versenyző kártyáján lévő szürke szív ikon kattintása. 4. A hírek oldal megnyitása a navigációs sávból. 5. Az újonnan megjelent „Csak a kedvencem hírei” gomb kattintása	Ha található a pilótáról hír akkor csak is azok a hírek listázása az oldalon.

2.33.4.4 ADMINISZTRÁCIÓS FELÜLET

Teszteset	Tesztelési cél	Lépések	Elvárt eredmény
1.	Admin felület megnyitása	1. Jelentkezzen be egy admin jogosultságú fiókkal. 2. Kattintson a navigációs sávban megjelent „adatok szerkesztése” gombra.	Megnyílik a pilóták szerkesztésére és hozzáadására alkalmas oldal.
2.	Pilóta adatainak szerkesztése.	1. Az „Adatok szerkesztése” oldal megnyitása. 2. Szerkesztés gomb megnyomása egy pilótánál. 3. Tetszőleges adat átírása. 4. Mentés gomb megnyomása.	2 másodperc után visszairányít a pilóták szerkesztéséhez.
3.	Kezdőlap beállítások frissítése.	1. A fő oldalon kattintás a pályára vagy a pilótára. 2. Pálya és a pilóta kiválasztása a legördülő listából. 3. Mentés gomb megnyomása.	A főoldalon frissültek a pálya és a pilóta adatai.

3 FELHASZNÁLÓI DOKUMENTÁCIÓ

3.1 A PROGRAM CÉLJÁNAK ÉS LÉNYEGESEBB FUNKCIÓINAK ÖSSZEFOGLALÁSA

Az F1Zone egy információs weboldal, amely a Formula–1 világa iránt érdeklődők számára készült, különös tekintettel azokra, akik most ismerkednek a sporttal, és szeretnének mélyebb betekintést nyerni annak működésébe és történetébe.

Az oldalon a felhasználók megismerhetik az aktuális szezon versenynaptárát, beleértve a futamok helyszíneit és időpontjait, valamint részletes információkat találhatnak az aktív versenyzőkről és azok statisztikáiról. Emellett áttekintést nyújtunk a legfontosabb szabályváltozásokról évről évre, valamint folyamatosan frissülő hírekkel segítjük a naprakész tájékozódást.

3.2 SZÜKSÉGES HARDVERESZKÖZÖK ÉS SZOFTVEREK FELSOROLÁSA

Hardverkövetelmények

- Asztali számítógép vagy laptop, mobilkészülék
- Internetkapcsolat

Szoftverkövetelmények

- Bármilyen böngésző
- Operációs rendszer amin meglehet nyitni egy böngészőt

3.3 TELEPÍTÉS ÉS INDÍTÁS LÉPÉSEINEK ISMERTETÉSE

Az F1Zone egy webes alkalmazás, nem szükséges bármit telepíteni. Megfelelő internetkapcsolat és egy böngésző szükséges.

Használatba vétel lépései:

- Nyissa meg a böngészőt
- Címsorba írja be a weboldal címét: <https://www.f1zone.hu>

3.4 A PROGRAM RÉSZLETES BEMUTATÁSA

3.4.1 LÁTOGATÓI (VENDEG) FUNKCIÓK

3.4.1.1 KEZDŐLAP ÉS A DINAMIKUS VISSZASZÁMLÁLÓ MŰKÖDÉSE

- A főoldalon található egy visszaszámláló, ami a következő verseny magyar idő szerinti kezdetéig számol vissza másodpercre pontosan.



- Lejjebb görgetve az oldalon megtalálható két információs doboz. Az egyikben a legutóbbi futam van feltüntetve adataival együtt, a másikon pedig annak a futamnak a megválasztott „Driver of the day” pilótája.

Legutóbbi futam

JAPÁN
Suzuka International Racing Course
Egyedülálló „nyolcas” alakú vonalvezetés. A versenyzők kedvence a híres 130R kanyar és az ikonikus „S” kanyarok miatt.

5,807 km
HOSSZ

18
KANYAROK

Driver of the Day

OLASZ
Kimi Antonelli
Olasz pilóta, született 2006-ban. A Mercedes juniorprogram neveltje, aki rendkívül fiatalon dominálta az utánpótlás bajnokságokat. 2025-ben kapott állandó ülést a Mercedesnél Lewis Hamilton távozása után. A sportág egyik legnagyobb ígéretként tartják számon, akitől a jövő világbajnoki címét várják el.

1
GYŐZELEM

1
DOBOGÓ

0
VB CÍM

3.4.1.2 VERSENYZŐK ÉS PÁLYÁK LISTÁZÁSA

A platform lehetőséget biztosít az aktuális Formula–1-es mezőny, valamint a versenynaptárban szereplő helyszínek és néhány régebbi, klasszikus pálya gyors és átlátható áttekintésére. Mindkét funkció a felső navigációs sávból érhető el, és bejelentkezés nélkül (vendégként) is szabadon böngészhető.

- A kezdőlapra vagy bármely aloldalon kattintson a felső menüsorban található „**Versenyzők**” menüpontra.
- A betöltődő oldalon egy rácsos elrendezésű listát lát, ahol minden pilóta egy különálló, információs kártyán jelenik meg.
- A kártyákon az alapvető információk (a pilóta neve, rajtszáma és az általa képviselt csapat) azonnal leolvashatók.

Piloták
Kattints egy pilotára, hogy többet tudj meg róla

Ferrari

- 44**
Lewis Hamilton
FERRARI
BRIT
- 16**
Charles Leclerc
FERRARI
MONACÓI

Williams

- 55**
Carlos Sainz
WILLIAMS
SPANYOL
- 23**
Alexander Albon
WILLIAMS
THAI

- Hasonló módon, a felső navigációs sáv „**Pályák**” menüpontjára kattintva érhető el a versenyhelyszínek listája.
- A felület itt is kártyás elrendezést használ, képekkel és alapvető adatokkal (pl. pálya neve, hossza, kanyarok száma) mutatja be az egyes nagydíjak helyszíneit.

Versenynaptár Pályái

Böngéssz végig a Forma-1 szezon összes legendás helyszínét!



MAGYARORSZÁG

Hungaroring

A „monacói pálya falak nélkül”. Szűk, kanyargós, nehéz rajta előzni, de a stratégia és a gumikezelés kulcsfontosságú.

4,381 km **14**

HOSSZ KANYAR



MONACO

Monaco

A naptár ékköve. Egy utcai pálya, ahol a legkisebb hiba is a falban végződik. Itt az időmérő az egész hétvége legfontosabb eseménye.

3,337 km **19**

HOSSZ KANYAR



BAHRAIN

Bahrain International Circuit

Sivatagi pálya, ahol a szél és a homok gyakran bezavar. Kiváló előzési pontokkal rendelkezik, és villanyfényben rendezik.

5,412 km **15**

HOSSZ KANYAR

3.4.1.3 RÉSZLETES PILÓTA ADATLAP MEGTEKINTÉSE

Amennyiben rákattint egy konkrét pilóta kártyájára, a rendszer átirányítja őt az adott pilóta adatlapjára. Ezek az oldalak letisztult, sötét tónusú dizájnnal jelennek meg.

- Az oldal tetején nagy betűkkel megjelenik a pilóta neve, alatta pedig egy szöveges áttekintés (Életrajz) olvasható a versenyző karrierjéről és jelentőségéről.
- Egy különálló, ikonokkal ellátott rácsos elrendezésben található a pilóta aktuális szerződésével kapcsolatos információk. Itt olvasható le a becsült fizetése, az aktuális csapata, a csapat fő szponzorai, valamint a motorpartner.
- Az oldal alsóbb részén kapott helyet a legfontosabb statisztikákat összefoglaló sáv. Itt külön, kiemelt kártyákon jelennek meg a pilóta győzelmei, pódiumos helyezései, leggyorsabb körei, szerzett pontjai és a világbajnoki címei. A kiemelkedő eredményeket (pl. győzelmek és VB címek megléte esetén) a rendszer automatikusan aransárga színnel hangsúlyozza.
- Az oldal legalján egy dedikált „Vissza a többi pilótához!” gomb segíti a felhasználót a gyors visszatérésben a fő listához.

Lewis Hamilton

ÉLETRAJZ

Hétszeres világbajnok, a sportág statisztikailag legsikeresebb pilótája, aki 2025-től a Ferrari versenyzője. Karrierje során minden fontos rekordot megdöntött, és ma már nemcsak pilótaként, hanem globális ikonként is meghatározó alakja a Formula 1-nek. Tapasztalata és sebessége még mindig elit.

 BECSÜLT FIZETÉS 50 000 000 \$	 AKTUÁLIS CSAPAT Ferrari	 CSAPAT FŐ SZPONSZORAI Shell, HP	 MOTORPARTNER Ferrari
---	---	---	--

KARRIER / SZEZON STATISZTIKA

105 GYŐZELMEK	201 PÓDIUMOK	67 LEGGYORSABB KÖR	7 VB CÍMEK	4820,5 PONTOK
-------------------------	------------------------	------------------------------	----------------------	-------------------------

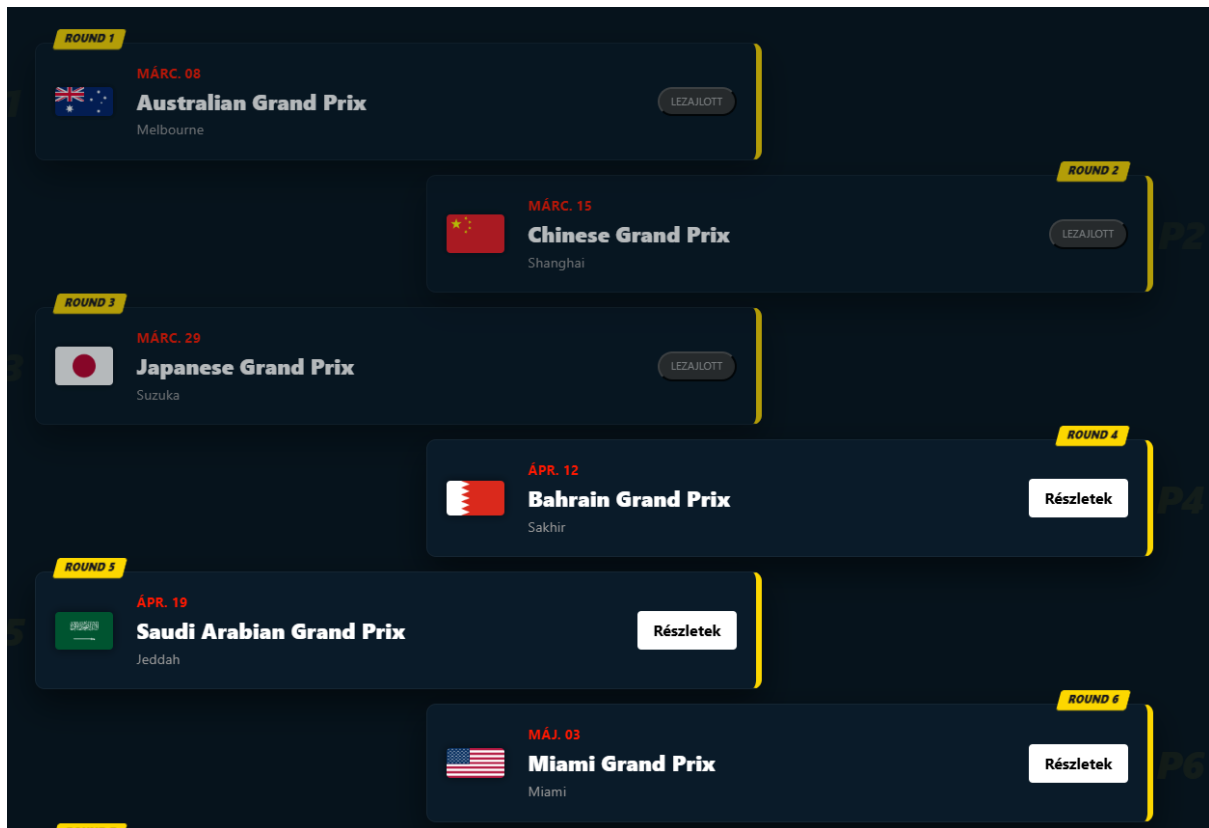
[← Vissza a többi pilótához!](#)

3.4.1.4 VERSENNAPTÁR, HÍREK ÉS SZABÁLYOK BÖNGÉSZÉSE

A platform nemcsak adatbázisként, hanem naprakész információs portálként is szolgál a Formula–1 rajongói számára. A felső navigációs sávból három további fontos menüpont érhető el, amelyek a szezon aktuális történéseit és háttérinformációit mutatják be. Ezek a funkciók szintén regisztráció nélkül, minden látogató számára nyitottak.

Versenynaptár:

- A felső menüsorban a „**Versenynaptár**” gombra kattintva egy időrendbe állított listát vagy kártyás nézetet kap az aktuális futamairól.
- Az egyes nagydíjaknál feltüntetésre kerül a pontos dátum, a kezdés időpontja, valamint a helyszín.
- A felület egyértelműen jelzi a futamok státuszát (pl. „Közelgő” vagy „Befejezett”), így könnyen átlátják, hol tart éppen a bajnokság.



Hírek:

- A „**Hírek**” menüpontot kiválasztva egy hírfolyam jelenik meg, ahol a cikkek időrendben, a legfrissebbtől visszafelé haladva listázódnak.
- Minden hírkártya tartalmaz egy figyelemfelkeltő címet, egy rövid összefoglalót, valamint a publikálás dátumát.
- A felhasználó a kártyára kattintva nyithatja meg a teljes cikket, hogy részleteiben is tájékozódjon az eseményekről.

LEGFRISSEBB HÍREK

Hivatalos F1 hírek magyar nyelven



ORIGO.HU

Max Verstappen távozik a Red Bulltól? Reagált a csapatfőnök

Friss hírek Magyarországról és a nagyvilágból, sport, technika, szórakozás, tudomány, kultúra. Az ORIGO a legfrissebb hazai és nemzetközi hírekkel szolgál olvasói számára.

2026.04.05. 13:43



ORIGO.HU

Vicces ajándékot kapott Charles Leclerc a monacói hercegtől

Friss hírek Magyarországról és a nagyvilágból, sport, technika, szórakozás, tudomány, kultúra. Az ORIGO a legfrissebb hazai és nemzetközi hírekkel szolgál olvasói számára.

2026.04.04. 13:53



PORTFOLIO.HU

Sport és ESG: Hogyan (ne) zöldítsük a profi sportot

Az ESG – környezetvédelem, társadalmi felelősségvállalás és vállalatirányítás – ma már a profi sport világában is kulcstémává vált. Szinte nincs olyan nagy sportszervezet, amelynek ne lenne fenntarthatósági stratégiája vagy nettózérő-célkitűzése, és a szponzo...

2026.04.04. 13:00

Szabályok (Szabályváltozások)

- A „**Szabályok**” (vagy Szabályváltozások) menüpont alatt a látogatók megismerkedhetnek a sportág aktuális előírásaival.
- Ez a felület az új szurkolók számára kiemelten hasznos lehet, vagy a nagy szabályváltozások megértéséhez.
- A tartalom jól strukturált, bekezdésekre és pontokra bontott formában jelenik meg, így a hosszú, hivatalos dokumentumok helyett egy könnyen olvasható, felhasználóbarát összefoglalót nyújt az aerodinamikai, motorikus és versenyszervezési szabályokról.

Technikai Szabályváltozások

Az oldalon az első hivatalos szezontól a mai napig a nagyobb, jelentősegteljesebb szabályváltozásokat gyűjtöttük össze ide.

- 1950**
A Forma-1 első hivatalos szezonja. 1.5 literes kompresszoros vagy 4.5 literes szívómotorok. A biztonsági előírások ekkor még minimálisak voltak, a versenyzők bőrszakban és szemüvegben vezettek. Az autók motorja elöl helyezkedett el, és a technológia nagy része a háború előtti konstrukciókon alapult.
- 1966**
A "Return to power" éra kezdete: a motorok hengerűrtartalmát 3.0 literre növelték, hatalmas ugrás a teljesítményben. Ez a korszak kényszerítette ki a mérnökökből a tapadás javítását, ami a szárnyak megjelenéséhez vezetett. A motorok ereje megduplázódott a korábbi évekhez képest, elérve az 500 lóerőt. Megjelentek az első középmotoros autók, amelyek forradalmasították az irányíthatóságot.
- 1978**

3.4.2 FIÓKKEZELÉS ÉS HITELESÍTÉS

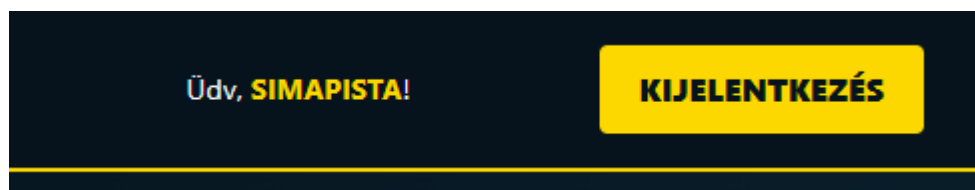
3.4.2.1 REGISZTRÁCIÓ ÚJ SZURKOLÓKÉNT

Az oldal személyre szabott funkcióinak eléréséhez a látogatóknak felhasználói fiókot kell létrehozniuk.

- A felső navigációs sávban található „**BEJELENTKEZÉS**” gombra kattintva a hitelesítési oldalra jut a felhasználó.
- Itt a bejelentkezési űrlap alatt található „*Nincs még fiókod? Regisztráció*” linkre kattintva érhető el a regisztrációs felület.
- A sikeres regisztrációhoz meg kell adni egy érvényes e-mail címet, egy tetszőleges felhasználónevet, valamint egy biztonságos jelszót. A rendszer a háttérben ellenőrzi az adatok helyességét.

3.4.2.2 SIKERES REGISZTRÁCIÓ ÉS BEJELENTKEZÉS

- A regisztrációt követően, vagy már meglévő fiók esetén a Bejelentkezés oldalon az e-mail cím és a jelszó megadásával lehet belépni a rendszerbe.
- Sikeres hitelesítés esetén a rendszer a böngésző memóriájában tárolja a bejelentkezési állapotot.
- A felhasználói élmény azonnal megváltozik: a felső navigációs sávban megjelenik a felhasználó neve („Üdv, **FELHASZNÁLÓNÉV!**”), a „Bejelentkezés” gombot pedig felváltja a „Kijelentkezés” opció.



3.4.3 REGISZTRÁLT FELHASZNÁLÓI FUNKCIÓK

3.4.3.1 PILÓTÁK HOZZÁADÁSA A KEDVENCEKHEZ, TÖRLÉS A KEDVENCEKBŐL

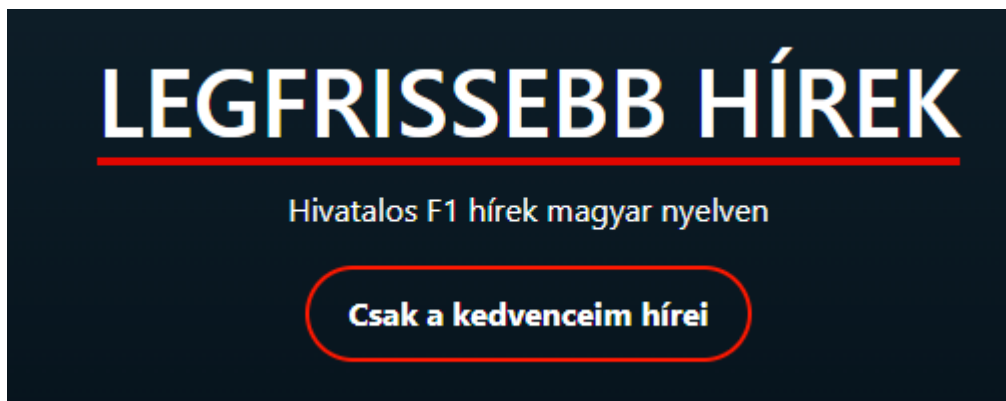
A bejelentkezett felhasználók személyre szabhatják az élményt kedvenc versenyzőik kiválasztásával.

- Egy pilóta kártyájának a sarkánál lévő szív ikonra kattintva pirossá változik a szív, innen lehet tudni, hogy hozzá van adva a kedvencekhez.
- Ha már nem kívánja kiemelten követni az adott pilótát, az ismételt kattintással törölheti őt a kedvencek listájáról.



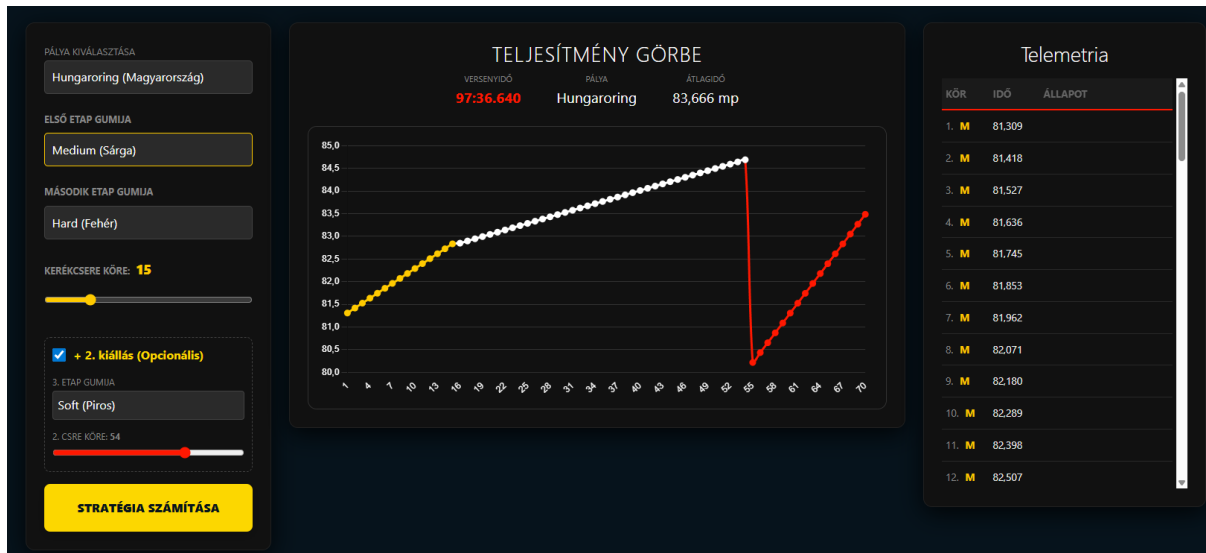
3.4.3.2 HÍREK SZŰRÉSÉ KEDVENC ALAPJÁN

- A „Hírek” menüpont alatt a bejelentkezett felhasználók számára elérhetővé válik egy szűrési opció, amellyel aktiválható a „Csak a kedvenceim hírei” nézet.
- Bekapcsolásakor a hírfolyam dinamikusan frissül, és elrejt azokat a cikkeket, amelyek nem kedvelt pilótákhoz kapcsolódnak, így biztosítva a releváns információfogyasztást.



3.4.3.3 HOZZÁFÉRÉS A STRATÉGIA MENÜPONTHOZ ÉS DIAGRAMOKHOZ

- A „Stratégia” menüpontra kattintva a rendszer ellenőrzi a jogosultságot.
- Bejelentkezett felhasználó esetén betöltődik az interaktív stratégiai oldal, ahol a diagramokon elemezhetők a köridők, a boksziállások és a használt gumikeverékek.



3.4.3.4 KIJELENTKEZÉS A FIÓKBÓL

- A munkamenet biztonságos lezárásához a felső menüsorban (mobilon a lenyíló menüben) található „KIJELENTKEZÉS” gombra kattinthat.
- A rendszer ekkor törli a helyi hitelesítési adatokat, és automatikusan visszairányítja a felhasználót a kezdőlapra, visszaállítva a vendég nézetet.

3.4.4 ADMINISZTRÁTOR FUNKCIÓK

3.4.4.1 ADMINISZTRÁTOR BEJELENTKEZÉS ÉS A REJTETT MENÜ ELÉRÉSE

A rendszer jogosultsági szinteket kezel. Ha egy adminisztrátori jogosultsággal rendelkező fiók (admin) jelentkezik be, a felület kibővül.

- A bejelentkezést követően a felső navigációs sávban megjelenik egy új, a normál felhasználók elől elrejtett menüpont: „Adatok szerkesztése”.
- Erre kattintva az adminisztrátor átkerül a vezérlőpultra.

3.4.4.2 ADATOK SZERKESZTÉSE FELÜLET BEMUTATÁSA

- Az oldalon egy átlátható lista vagy táblázat fogadja az admint a rendszerben lévő pilótákról.
- Minden sor mellett található a módosítást (Szerkesztés) lehetővé tevő vezérlőgombok és a törlés gombok is megtalálhatóak mellettük.

3.4.4.3 PILÓTÁK ADATAINAK MÓDOSÍTÁSA

Mivel a Formula-1-es adatok versenyről versenyre változnak, az adminisztrátor könnyedén frissítheti ezeket:

- A kiválasztott pilóta melletti Szerkesztés gombra kattintva betöltődik az adatlapja egy szerkeszthető űrlap formájában.
- Itt módosíthatók a kulcsfontosságú statisztikák, például a szerzett pontok, győzelmek, pódiumok vagy a leggyorsabb körök száma.
- A „Mentés” gombra kattintva az adatbázis frissül, és a változás azonnal megjelenik a publikus, látogatói oldalakon is.

The screenshot shows a form for editing driver data. The fields and their values are:

VB címké	7
Pontszám	4820,5
Nemzetiség	Brit
Bemutakozás éve	2007
Dobogók	201
Leggyorsabb körök	67

Below the form is a section titled "Életrajz" (Biography) with the text: "Hétszeres világbajnok, a sportág statisztikailag legsikeresebb pilótája, sebessége még mindig elit."

At the bottom of the form are two buttons: "ADATOK FRISSÍTÉSE" (Update Data) and "MÉGSE" (Cancel).

3.4.4.4 KEZDŐLAP BEÁLLÍTÁSOK (KIEMELT PILÓTA ÉS PÁLYA CSERÉJE)

A kezdőlap tartalmának frissítése is az admin felületről történik.

- A vezérlőpulton egy külön szekció szolgál a Kezdőlap tartalmának beállítására.
- Az adminisztrátor legördülő listákból választhatja ki, hogy melyik pilóta és melyik pálya (pl. a soron következő nagydíj) jelenjen meg kiemelten a főoldalon.
- A módosítás mentése után a kezdőlap azonnal az új kiemelt adatokkal töltődik be a látogatók számára.

HOMEPAGE BEÁLLÍTÁSOK

Itt tudod kiválasztani, hogy melyik pálya és melyik versenyző jelenjen meg a főoldalon.

Legutóbbi futam

VÁLASSZ PÁLYÁT:

Red Bull Ring

Driver of the Day

VÁLASSZ VERSENYZŐT:

Esteban Ocon

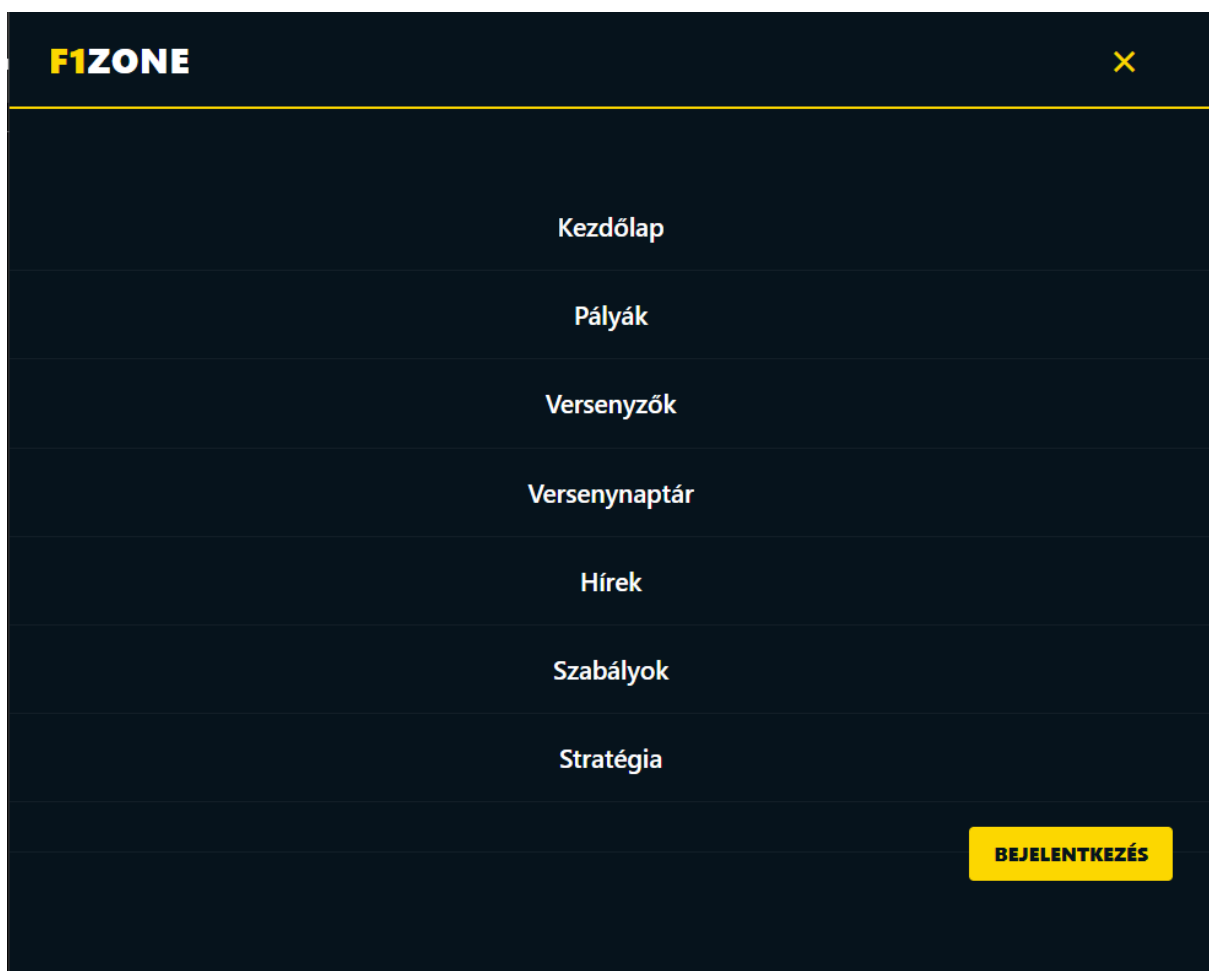
MENTÉS

3.4.5 AZ OLDALT SEGÍTŐ TÁJÉKOZÓDÁSI OPCIK

3.4.5.1 RESZPONZÍV NAVIGÁCIÓS SÁV ÉS A MOBIL HAMBURGER MENÜ

A webalkalmazás modern, reszponzív dizájnnal rendelkezik, így minden kijelzőméreten tökéletesen használható.

- Asztali nézetben a menüpontok vízszintesen, a fejlécben helyezkednek el.
- Ha a felhasználó kisebb képernyőn (pl. tableten vagy okostelefonon) nyitja meg az oldalt, a hagyományos menü eltűnik, és helyét egy sárga "Hamburger" ikon veszi át.
- Az ikonra kattintva egy elegáns, lenyíló mobilmenü jelenik meg, amely kényelmes, érintőképernyőre optimalizált formában kínálja fel az összes menüpontot, beleértve a felhasználói fiók kezelését is.



3.4.5.2 MUNKAMENET LEJÁRATÁNAK KEZELÉSE

A biztonság érdekében a rendszer figyeli a bejelentkezett felhasználók munkamenetét.

- Ha a bejelentkezéshez használt azonosító érvényessége lejár, vagy az adatok törölődnek a böngészőből, a rendszer biztonsági okokból automatikusan kijelentkezteti a felhasználót.
- Ilyenkor a védett oldalak (pl. Stratégia) elérése megszűnik, és a felhasználónak újra be kell jelentkeznie.

3.4.5.3 VISSZAJELZÉSEK ÉS HIBAÜZENETEK (PL. JOGOSULTSÁG HIÁNYA)

- Ha egy nem regisztrált (vendég) látogató olyan linket próbál megnyitni, amihez nincs joga (pl. a .../strategy URL-t direktbe beírva), a rendszer nem omlik össze.
- Helyette a rendszer automatikusan átirányítja a vendéget a bejelentkezés, illetve regisztrációhoz.

4 ÖSSZEFOGLALÁS

4.1 KÖSZÖNETNYILVÁNÍTÁS

Szeretnénk köszönetet nyilvánítani tanárainknak Méhes József tanárúrnak, Szalainé Török Edit tanárnőnek, Csukárdi Rajmund tanárúrnak és Szabó Rózsa tanárnőnek, nélkülük nem jöhetett volna létre ez projekt.

Szeretnénk hálásan megköszönni **Méhes József** tanár úrnak a munkáját, akinek vezetésével megismerhettük és elsajátíthattuk a teljes REST API backend használatát és számos fortélyát. Folyamatos, építő jellegű és részletes visszajelzései nemcsak a projektünk színvonalát emelték, hanem a rendszerszintű, mérnöki látásmódunkat is nagymértékben formálták.

Köszönjük **Szalainé Török Edit** tanárnőnek, hogy fáradhatatlanul segített elmélyíteni a tanórákon leadott anyagot, és megosztotta velünk széleskörű tudását. Külön köszönjük a kiemelkedő és áldozatos vizsgafelkészítő munkáját, amellyel felkészített minket az akadályok magabiztos és sikeres leküzdésére.

Külön hálával tartozunk **Csukárdi Rajmund** tanár úrnak, akinek végtelen türelme és higgadtsága rengeteget segített abban, hogy átlássuk, hogyan is épül fel és áll össze egy ekkora volumenű projekt – még annak ellenére is, hogy a megvalósítás során végül egy másik, általa nem oktatott módszert alkalmaztunk. Ez a szakmai rugalmasság és támogatás sokat jelentett számunkra.

Végül, de nem utolsósorban nagyra értékeljük **Szabó Rózsa** tanárnő munkáját és odafigyelését, aki még a tanulmányaink végén, a legnagyobb hajrában sem feledkezett meg rólunk, és folyamatos támogatásával, valamint iránymutatásával segített minket a célba érésben.

4.2 ELKÉSZÜLT PROJEKTÜNK ÉRTÉKELÉSE

Rendkívül büszkék vagyunk az F1Zone platform fejlesztése során elért eredményeinkre. Sikerült egy stabil, jól skálázható és komplex rendszert felépítenünk, amely nemcsak egy egyszerű statisztikai adatbázis, hanem egy interaktív Formula–1-es stratégiai portál is.

A projekt során rengeteget fejlődünk technológiai téren: mélyebb betekintést nyertünk a Blazor komponensalapú architektúrájába, a JSInterop grafikonos megjelenítéseibe, valamint a REST API-k Generic Controller alapú, professzionális háttérrendszerének felépítésébe.

Bár a fejlesztés és különösen a szigorú, mockoláson alapuló automatizált tesztelés (bUnit, xUnit) komoly kihívások elé állított minket, ezek a feladatok óriási mértékben járultak hozzá a rendszerszemléletű gondolkodásunkhoz.

Természetesen a projekt nem tekinthető maradéktalanul befejezettnek: a felhasználói felület (UI) bizonyos interaktív részeinek finomítása, valamint a komplexebb telemetriai adatok jövőbeli integrálása további izgalmas fejlődési lehetőségeket tartogat.

Összességében azonban egy olyan magas színvonalú, hibatűrő vizsgamunkát hoztunk létre, amely szilárd alapot biztosít a jövőbeli továbbfejlesztésekhez.

5 IRODALOMJEGYZÉK

Webes technológiák és Keretrendszerek (Blazor & .NET)

- Microsoft Corporation. (2024). *ASP.NET Core Blazor bevezetés és dokumentáció*. Microsoft Learn.
 - Elérhető: <https://learn.microsoft.com/hu-hu/aspnet/core/blazor>
- Microsoft Corporation. (2024). *JavaScript-együttműködés (JSInterop) az ASP.NET Core Blazorban*. Microsoft Learn.
 - Elérhető: <https://learn.microsoft.com/hu-hu/aspnet/core/blazor/javascript-interoperability/>

Szakmai videóanyagok és Kurzusok (YouTube)

- Tim Corey. (2023). *Intro to Blazor in .NET 8*.
 - Elérhető: https://www.youtube.com/watch?v=jw_gkkg9Tw
- Nick Chapsas. (2022). *Unit Testing in .NET Core with xUnit & Moq*.
 - Elérhető: <https://www.youtube.com/watch?v=GwYpuKQJ3vU>

Szoftvertesztelés (bUnit, xUnit, Moq)

- Hansen, E., et al. (2024). *bUnit - A testing framework for Blazor components*. bUnit hivatalos dokumentáció.
 - Elérhető: <https://bunit.dev/docs/getting-started/>
- Clarius Consulting, et al. (2024). *Moq: The most popular and friendly mocking framework for .NET*. GitHub Wiki.
 - Elérhető: <https://github.com/moq/moq4/wiki/Quickstart>
- .NET Foundation. (2024). *xUnit.net Official Documentation*. xUnit.net.
 - Elérhető: <https://xunit.net/>